

Machine Learning avec Python

Joyce MBIGUIDI • Data Scientist



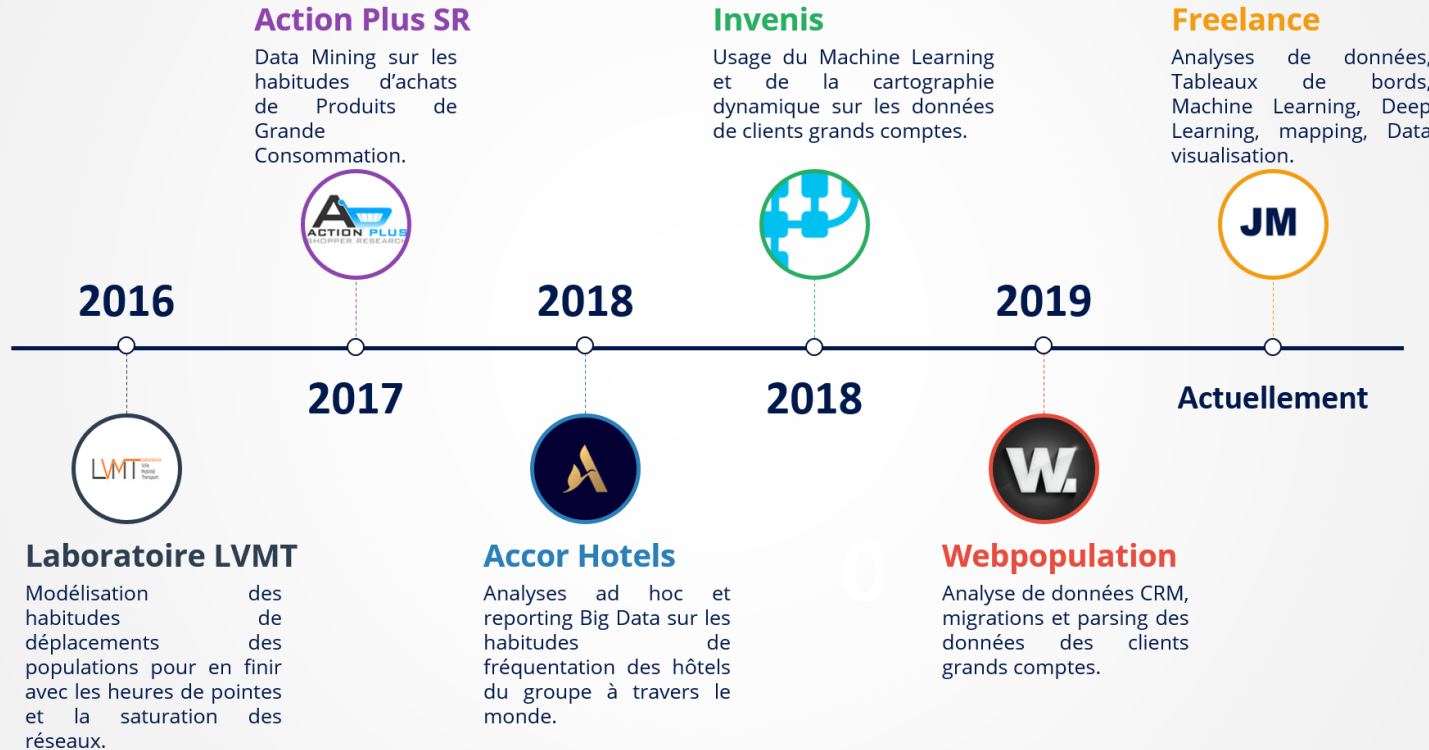
Objectifs pédagogiques

- Identifier les principaux algorithmes supervisés et non supervisés
- Savoir implémenter des algorithmes supervisés et non supervisés
- Traduire un problème réel en solution opérationnelle
- Choisir l'algorithme le plus approprié face à un problème donné



Skills4All
digital learning certifiant

Le formateur





Skills4All
digital learning certifiant

Déroulement des séances

01

- Approche théorique
- Démonstration python
- Conseil & astuces

02

- Exercice pratique
- Correction des exercices





Skills4All
digital learning certifiant

Plan de formation

Plan de formation

- Introduction
- Régression linéaire simple
- Régression linéaire multiple
- Régression logistique
- Arbres de décision pour la régression
- Arbres de décision pour la classification
- Random Forest pour la régression
- Random Forest pour la classification
- XG Boost pour la régression
- XG Boost pour la classification
- K Nearest Neighbor (KNN) pour la régression
- K Nearest Neighbor (KNN) pour la classification
- Support Vector Machine (SVM) pour la régression
- Support Vector Machine (SVM) pour la classification
- Analyse en Composantes Principales (ACP)
- Analyse Factorielle des Correspondances (AFC)
- Analyse en Correspondance Multiples (ACM)
- Classification Ascendante Hiérarchique (CAH)
- K-Means
- DB Scan
- Deep Learning (CNN)



Skills4All
digital learning certifiant

Introduction



Définition



Machine Learning par secteur d'activité



Machine Learning par type d'application



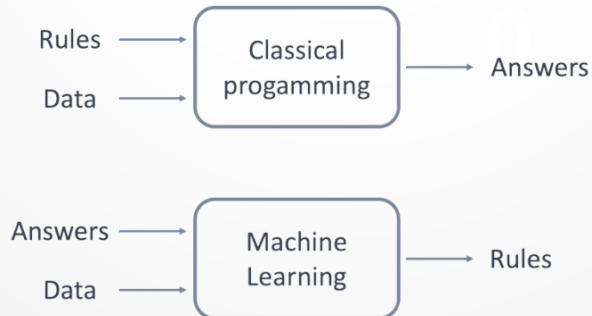
Déroulement d'une étude



Méthodes descriptives et prédictives

Introduction

- Le Machine Learning est un domaine scientifique qui mobilise des algorithmes d'apprentissage automatique pour découvrir des « patterns » dans de petits ou grands ensembles de données.
- Les algorithmes apprennent de façon autonomes à réaliser les tâches qu'on leur demande.
- Pour être autonomes, certains algorithmes ont besoin d'être entraînés à partir d'un ensemble de données labélisées.



Introduction : définition

Qu'est-ce que l'intelligence ?

- L'intelligence au sens général, peut être décrite comme la capacité d'un système ou d'un être à comprendre, apprendre, raisonner et s'adapter de manière autonome à son environnement.
- **Chez les animaux** : l'intelligence animale fait référence à la capacité des animaux à acquérir, comprendre et utiliser des connaissances pour résoudre des problèmes et s'adapter à leur environnement.
- **Chez l'homme** : l'intelligence humaine peut être définie comme la capacité de comprendre, d'apprendre, de raisonner et de s'adapter de manière complexe et créative à différents domaines de la connaissance et de l'expérience.

Introduction : définition

Les animaux intelligents

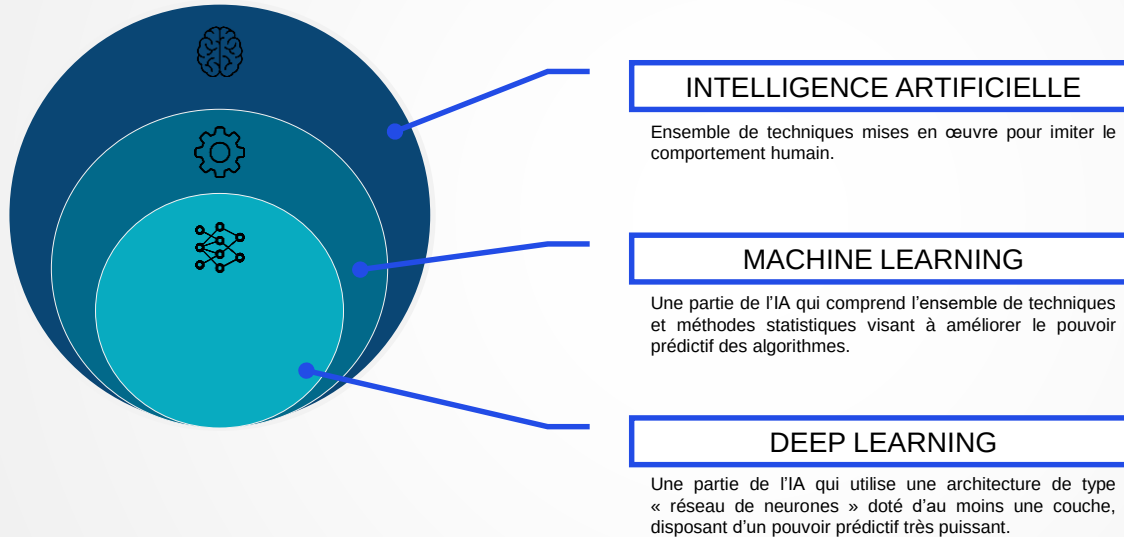
- **Chimpanzé :**
 - Conscience d'eux-mêmes, se reconnaissent dans un miroir
 - Juge l'intention de ses congénères et réagit en conséquence
 - Fabrique et utilise des outils
 - Transmets la culture aux générations suivantes...
- **Pieuvre :**
 - Mémoire
 - Apprentissage
 - Orientation
 - En captivité, il teste plusieurs stratégies pour s'échapper

Introduction : définition

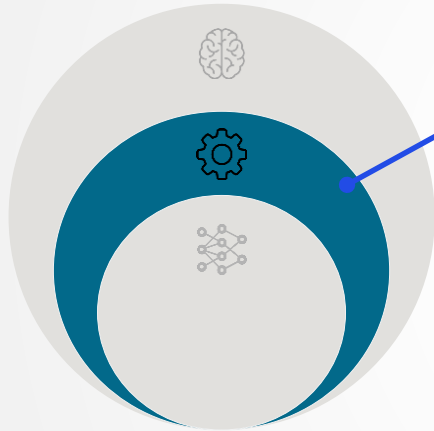
L'intelligence chez l'humain

- « L'intelligence humaine est caractérisée par plusieurs aptitudes, surtout cognitives, qui permettent à l'individu humain d'apprendre, de former des concepts, de comprendre, d'appliquer la logique et la raison » (cf. Wikipédia).
- L'activité neuronale basée sur l'interconnexion complexe de milliards de neurones, qui communiquent entre elles par des signaux électriques et chimiques pour traiter l'information et contrôler les fonctions du corps et de l'esprit.
- Ces signaux électriques, appelés influx nerveux, se propagent le long des neurones via des synapses, permettant ainsi la transmission et le traitement des informations dans le cerveau, ce qui sous-tend les processus cognitifs, émotionnels et moteurs qui caractérisent notre fonctionnement mental et comportemental.

Introduction : définition



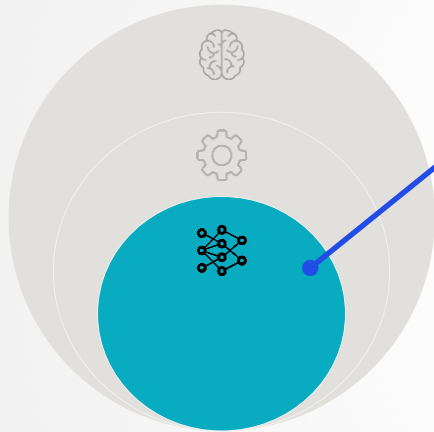
Introduction : définition



MACHINE LEARNING

- Modèles Supervisés :
 - Régression linéaire
 - Régression Logistique
 - Arbres de décisions
 - Random Forest
 - XG Boost
- Modèles Non supervisés :
 - Analyse Multivariée (ACP, AFC, ACM...)
 - K-Means
 - DB SCAN

Introduction : définition

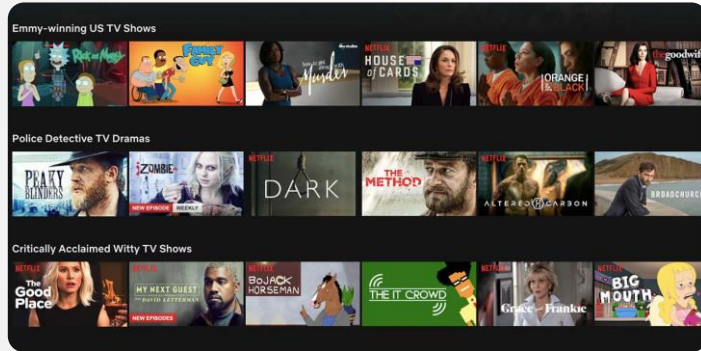


DEEP LEARNING

- ANN (Artificial Neural Network) :
 - Prédire le départ d'un client
 - Prédire la conversion d'un prospect en client
- CNN (Convolution Neural Network) :
 - Reconnaissance faciale
 - Voiture autonome
 - Détection d'incendie par image satellite.
- RNN (Recurrent Neural Network) :
 - Prédire le prix d'une action au CAC40
 - Prédire l'évolution et le prix du baril de pétrole
- SOM (Self Organizing Map) :
 - cartes auto-adaptative pour la détection de fraude bancaire
- BM (Boltzmann machine) :
 - Système de recommandation
- AE (AutoEncoders) :
 - Dé-bruitage d'image (rendre une image floue plus claire)...

Introduction : définition

Moteur de recommandation Netflix



Computer vision



Agent conversationnel



amazon alexa



Introduction : Machine Learning par secteur d'activité

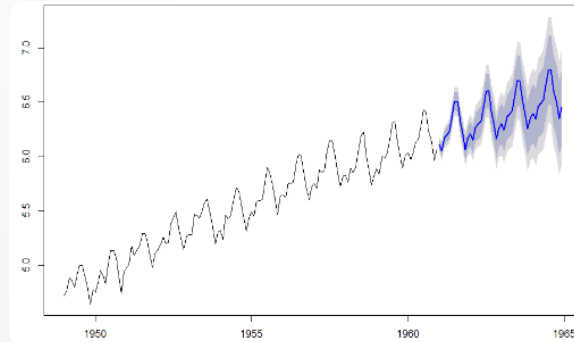
Banque

- Naissance du scoring de risque au début du XXème siècle.
- Plus tard : méthodes de classifications et associations de produits.
- Le Data Mining est rendu possible grâce à l'essor d'internet, la téléphonie mobile (NTIC), le traitement de l'information (entrepôts de données), la concurrence entre les banques...
- La prise en compte des données des réseaux sociaux interviendra bien plus tard.
- Grâce au Big Data, il est désormais possible de détecter les profils de clients risquant par exemple le surendettement.

Introduction : Machine Learning par secteur d'activité

Secteur financier

- Utilise les méthodes de séries temporelles pour effectuer des prévisions de prix, d'indicateurs économiques (PIB, taux d'intérêts directeurs, indice des prix à la consommation, taux de chômage...), d'évolution des marchés.



Introduction : Machine Learning par secteur d'activité

Grande distribution

- Développement des cartes de crédit : donne naissance à l'enrichissement de leurs bases de données, études comportementales provenant des tickets de caisse.
- Cartes de fidélité : détection des associations de produits, profils de clients, optimisation de la disposition des produits en rayon.
- Segmentation de la clientèle et segmentation des tickets de caisses par familles de produits (textile, alimentaire...).

Introduction : Machine Learning par secteur d'activité

Assurances

- **Études de ventes croisées (cross-selling)** : technique de vente croisée ou additionnelle qui consiste à proposer un service/produit complémentaire lors de l'achat pour augmenter le panier moyen du client.
- **Études de montée en gamme (up-selling)** : proposer un produit/service plus cher que le produit initial pour monter en gamme et faire plus de marge.
- Tarification en fonction du risque encourus.

Introduction : Machine Learning par secteur d'activité

Transports

- **Yield Management** : fixation dynamique des prix en fonction des critères de concurrence, taux de remplissage, événements locaux ou mondiaux, préférences touristiques...).
- **Géolocalisation** : amélioration du trafic routier (accidents, bouchons, places libres de stationnement...)

Introduction : Machine Learning par secteur d'activité

Automobile

- Score de ré-achat
- Origine des défauts de construction (minimisation)
- Études de satisfaction pour améliorer la conception des véhicules (qualité, confort...)

Introduction : Machine Learning par secteur d'activité

Vente par correspondance

- Réduction des coûts (catalogues de produits adressés à des millions de clients)
- Segmentation RFM
- Actions marketing (prospection commerciale)
- Si la banque est pionnière en matière de score de risque, la vente par correspondance l'est pour le score d'appétence.

Introduction : Machine Learning par secteur d'activité

Médecine

- Études d'association de médicaments (contre indication)
- Études sur les facteurs de décès ou de survie face à une pathologie (Covid, cancers, infarctus...)
- Décryptage du génome humain : trouver les gènes responsables de la survenue d'une maladie héréditaire
- Reconnaissance d'image en imagerie médicale : cancer du sein

Introduction : Machine Learning par secteur d'activité

Agroalimentaire

- Analyses sensorielles (goût, saveur, texture...)
- Analyse des risques sanitaires : évolution des micro-organismes afin de prévenir des risques liés à la santé (date limite de consommation)
- Classification des espèces vivantes : plantes, animaux

Introduction : Machine Learning par type d'application

Gestion de la relation client

- Conversion de prospects en clients
- Reconversion d'anciens clients en clients
- Détection des clients susceptibles de partir pour la concurrence
- Personnalisation dynamique des pages d'un site web en fonction du profil du visiteur
- Gestion des appels entrants dans une plateforme téléphonique
- Analyse des lettres de réclamation de la clientèle

Introduction : Machine Learning par type d'application

Marketing stratégique

- Aide à la création des packages et des promotions
- Aide à la conception des nouveaux produits
- Politique de fidélisation de la clientèle
- Adaptation de la communication à chaque segment de clientèle
- Associations de produits
- Surveillance de la e-réputation de l'entreprise

Introduction : Machine Learning par type d'application

Gestion du risque

- Recherche des facteurs de risques de sinistre (assurance habitation / automobile)
- Prévention des impayées (téléphonie, factures d'énergie...)
- Autorisation des transactions bancaires (cas de fraudes ou de compte débiteur)
- Optimisation des procédures de recouvrement et de contentieux
- Détection de fraude automatisée

Introduction : déroulement d'une étude

1. Définition des objectifs
2. Inventaire des données existantes
3. Collecte des données
4. Exploration et préparation des données
5. Segmentation de la population
6. Élaboration et validation des modèles prédictifs
7. Déploiement des modèles
8. Suivi des modèles
9. Enrichissement des modèles

Introduction : méthodes descriptives et prédictives

- **Description** : l'objectif est de décrire les tendances et modèles cachés au sein des données. Cela sert à expliquer ou vérifier une hypothèse.
- **Classification** : consiste à créer des sous-ensembles de données similaires, afin d'obtenir une vision générale de l'ensemble à l'aide de l'exploration des données. Les informations sont classifiées selon des critères bien précis, en fonction d'une problématique définie.
- **Association** : cette méthode se concentre sur la possible corrélation entre deux événements. Elle s'efforce d'expliquer de quelle façon un fait est lié à un autre, afin de mieux comprendre des comportements ou des tendances.

Introduction : méthodes descriptives et prédictives

- Les méthodes prédictives visent à expliquer des phénomènes observés ou non dans l'analyse descriptive.
- Si les informations extrapolées sont **qualitatives**, on parle alors de **discrimination** ou classement.
- Si les informations extrapolées sont **quantitatives**, on parle alors de **prédictions**.

Introduction : méthodes descriptives et prédictives

- On recherche une variable « cible », c'est-à-dire une donnée dont on souhaite connaître la valeur.
 - **Estimation** : l'estimation établit le lien entre une combinaison de critères et une valeur cible unique.
 - **Segmentation** : il s'agit de classer des données en fonction de critères qui déterminent leurs valeurs. Les sociétés peuvent ainsi segmenter les consommateurs ou prospects selon leurs caractéristiques comme leur âge, sexe ou lieu d'habitation.
 - **Prévision et prédiction** : cette technique d'analyse vise à estimer et prédire des valeurs et tendances futures. Sa finalité : découvrir des structures qui peuvent mener à des prédictions et modèles vraisemblables. Par exemple pour un site e-commerce : il est intéressant de prédire quel type de produit un client sera amené à choisir en fonction de ses achats précédents.

Vocabulaire

Vocabulaire

- **Données** : les données sont à la base de l'apprentissage automatique. C'est la matière première qu'un algorithme d'apprentissage automatique utilise pour apprendre des modèles ou des relations. Les données peuvent être sous forme de texte, d'images, de nombres ou de tout autre format.
- **Features** : les features sont les caractéristiques ou propriétés mesurables des données. Par exemple, dans un ensemble de données sur les prix des maisons, les caractéristiques peuvent inclure la taille, l'emplacement et l'âge des maisons.
- **Labels** : les labels sont les résultats que nous voulons que l'algorithme prédise. Dans l'exemple du prix de la maison, l'étiquette serait le prix réel de la maison.

Vocabulaire

- **Modèle** : un modèle est la représentation mathématique d'une relation entre des entités et des étiquettes. C'est le "cerveau" qui apprend à partir des données.
- **Entraînement** : l'entraînement est le processus d'introduction de données (caractéristiques et étiquettes) dans un algorithme d'apprentissage automatique pour créer un modèle. L'algorithme analyse les données et essaie de découvrir des modèles ou des relations entre les caractéristiques et les étiquettes.
- **Test** : le test est le processus d'évaluation des performances du modèle formé sur de nouvelles données inédites. Cela nous aide à déterminer dans quelle mesure le modèle se généralise à de nouvelles situations.
- **Évaluation** : L'évaluation est le processus de mesure de la performance d'un modèle. Nous utilisons généralement des mesures telles que l'accuracy, la précision, le rappel et le score F1 pour évaluer les performances du modèle.

Vocabulaire

- **Apprentissage supervisé** : le modèle s'entraîne à partir d'un exemple de données labélisées. Il apprend à identifier certaines tendances, relations, redondances (patterns) dans les données.
- **Apprentissage non supervisé** : le modèle ne dispose pas d'exemples en entrée et les données ne sont pas labélisées. Il cherche tout seul des patterns. Il analyse tout seul les données et essaie de détecter des classements, des densités dans les données.

Environnement de développement

Environnement de développement



Visual Studio Code





Skills4All
digital learning certifiant

Avant de débiter...

Avant de débiter : comprendre les données

- Dans le Machine Learning, les données sont la matière première que nous utilisons pour enseigner nos algorithmes. Considérez les données comme les ingrédients d'une recette. Plus les ingrédients sont bons, meilleur est le plat final. Les données peuvent prendre de nombreuses formes, telles que des nombres, du texte, des images ou de l'audio.
- Imaginez que nous ayons les données suivantes sur les tailles (en pouces) et les poids (en livres) d'un groupe de personnes :
 - Tailles : [62, 64, 68, 70, 72, 74]
 - Poids : [120, 130, 145, 160, 180, 200]
- Notre objectif est de créer un modèle d'apprentissage automatique capable de prédire le poids d'une personne en fonction de sa taille. Dans ce cas, la "hauteur" est la feature et le "poids" est la cible.

Avant de débiter : choix de l'algorithme

- Il existe un large choix d'algorithmes d'apprentissage automatique :
 - Si la variable y est une variable quantitative, dans la plupart des cas, vous aurez à faire à un problème de régression.
 - Si la variable y est qualitative nominale ou ordinale, dans la plupart des cas, vous aurez à faire à un problème de classification.
- Dans notre cas, nous avons une variable indépendante (taille) et une variable dépendante (poids), il s'agit d'un modèle de régression dont la régression linéaire est le choix le plus approprié.

Avant de débiter : préparer les données

- Avant d'introduire les données dans l'algorithme, nous devons les diviser en deux parties : un ensemble d'apprentissage (80%) et un ensemble de test (20%). L'ensemble d'apprentissage est utilisé pour « enseigner » l'algorithme et l'ensemble de test est utilisé pour évaluer ses performances.
- Utilisons les 4 premiers points de données pour la formation et les 2 derniers pour les tests :
 - **Ensemble d'entraînement :**
 - Tailles : [62, 64, 68, 70]
 - Poids : [120, 130, 145, 160]
 - **Ensemble de test :**
 - Hauteurs : [72, 74]
 - Poids : [180, 200]

Avant de débiter : entraînement du modèle

- Il s'agit d'alimenter les données d'entraînement dans l'algorithme de régression linéaire pour créer un modèle.
- L'algorithme analysera les données et tentera de découvrir un modèle (pattern) ou une relation entre la taille et le poids.
- Dans ce cas, l'algorithme trouvera la ligne droite la mieux ajustée qui représente cette relation.

Avant de débiter : test du modèle

- Une fois que nous avons un modèle entraîné, il convient d'utiliser les données de test pour évaluer ses performances.
- Pour chaque « hauteur » de l'ensemble de test, le modèle prédira les poids correspondants.
- Nous pouvons ensuite comparer les poids prédits aux poids réels pour voir à quel point notre modèle est précis.

Avant de débiter : évaluation du modèle

- Il existe plusieurs façons de mesurer les performances d'un modèle, selon qu'il soit un modèle de régression ou de classification.
- Pour la régression linéaire, une métrique commune est appelée « erreur quadratique moyenne » (MSE), qui calcule la différence quadratique moyenne entre les valeurs prédites et réelles. Plus le MSE est bas, meilleures sont les performances du modèle. Il y a également le R^2 , qui mesure la qualité d'ajustement du modèle.
- À ce stade, vous avez appris les étapes de base de la création d'un modèle d'apprentissage automatique à l'aide de la régression linéaire.



Skills4All
digital learning certifiant

Feature scaling

Feature scaling : normalization (Min-Max Scaling)

- La normalisation met à l'échelle les features en les transformant en une plage de [0, 1]. Cela se fait en soustrayant la valeur minimale de la caractéristique et en la divisant par la plage (valeur maximale moins valeur minimale).
 - Formule : $(x - \min(x)) / (\max(x) - \min(x))$
- Par exemple, supposons que nous ayons un ensemble de données avec des tailles en pouces et des âges en années :
 - Tailles : [62, 64, 68, 70]
 - Âges : [20, 25, 30, 35]
- Après normalisation, l'ensemble de données ressemblerait à ceci :
 - Hauteurs normalisées : [0, 0,25, 0,75, 1]
 - Âges normalisés : [0, 0,333, 0,667, 1]

Feature scaling : standardization (Z-score Scaling):

- La standardisation met à l'échelle les caractéristiques en les transformant pour avoir une moyenne de 0 et un écart type de 1. Cela se fait en soustrayant la moyenne de la caractéristique et en la divisant par l'écart type.
 - Formule : $(x - \text{moyenne}(x)) / \text{std}(x)$
- En utilisant le même jeu de données, après normalisation, cela ressemblerait à ceci :
 - Hauteurs normalisées : [-1,341, -0,447, 0,447, 1,341]
 - Âges standardisés : [-1,341, -0,447, 0,447, 1,341]

Feature scaling : standardization (Z-score Scaling):

- Maintenant que nous en savons plus sur la mise à l'échelle des features, voyons comment l'appliquer dans un flux de travail Machine Learning :
 1. Préparez l'ensemble de données en le divisant en ensembles d'apprentissage et de test.
 2. Appliquez la mise à l'échelle des fonctionnalités à l'ensemble d'apprentissage. Assurez-vous d'enregistrer les paramètres de mise à l'échelle (min, max, moyenne, std) pour chaque fonctionnalité.
 3. Entraînez le modèle à l'aide de l'ensemble d'entraînement à l'échelle.
 4. Avant de faire des prédictions avec l'ensemble de test, appliquez la même mise à l'échelle aux données de test à l'aide des paramètres de mise à l'échelle de l'ensemble d'apprentissage. Cela garantit la cohérence du processus de mise à l'échelle.
 5. Évaluez les performances du modèle sur l'ensemble de test à l'échelle.

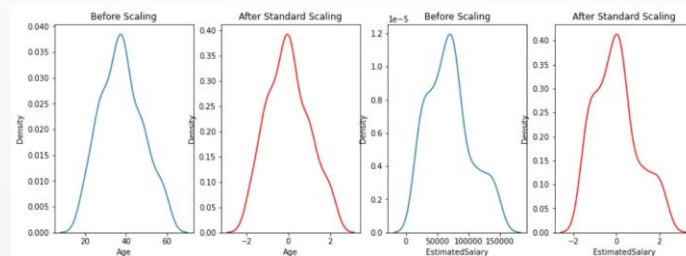
Feature scaling : discrétisation

- Discrétiser une variable aléatoire continue consiste à la découper en un nombre fini de compartiments.
- La nouvelle variable obtenue est une variable catégorielle.
- On peut discrétiser une variable de trois manières :
 - **Quantile** : chaque groupe a la même amplitude dans l'étendue des valeurs.
 - **Uniforme** : chaque groupe a le même nombre de valeurs réparties en centiles.
 - **K-Means** : des clusters sont identifiés et les points sont répartis entre les groupes.
- $age = [1, 5, 9, 4, 7, 11, 14, \dots, 42, 44, 46, 70, 74, 78, 77]$

<i>age_cut</i>	[1 – 11[	[11 – 21[	[21 – 31[	[... [	[67 – 77]
<i>age_discretized</i>	enfant	jeune	Mature	...	Vieux

Feature scaling : résumé

- Les données résultantes après une standardisation auront une moyenne de 0 et un écart type de 1
- Les données résultantes après une normalisation auront une valeur minimale de 0 et une valeur maximale de 1.
- Il n'y a en réalité aucun changement dans la distribution des données avant et après **feature scaling**. Tout ce qui change, c'est l'échelle de mesure.
- Dans certains cas d'usages, la précision du modèle peut significativement augmenter (tout dépendra alors de l'algorithme utilisé).





Skills4All
digital learning certifiant

Valeurs manquantes

Les valeurs manquantes :

- Les données manquantes se produisent lorsque certaines valeurs ne sont pas disponibles ou ne sont pas enregistrées dans votre jeu de données.
- Cela peut se produire pour diverses raisons, telles que des erreurs de saisie de données, des capteurs défectueux ou des participants qui ne fournissent pas d'informations.
- Le fait d'avoir des données manquantes peut entraîner des résultats biaisés ou incorrects lors de la formation d'un modèle d'apprentissage automatique.
- Certains algorithmes ne fonctionnent pas en raison de la présence de données manquantes. Il convient donc, selon les cas, de les supprimer ou de les imputer.
- D'autres algorithmes tels que les arbres de décision et leurs versions ensemblistes (Random Forest, XGBoost, etc.), peuvent accepter les données manquantes sans nécessiter de prétraitement.

Les valeurs manquantes : supprimer les lignes concernées

- Une approche simple consiste à supprimer toutes les lignes de votre ensemble de données contenant des valeurs manquantes.
- Cela peut être une stratégie efficace si le nombre de valeurs manquantes est faible et si leur suppression n'a pas d'impact significatif sur l'ensemble des données.
- Cependant, si une grande partie de vos données contient des valeurs manquantes, cette méthode peut entraîner la perte de trop d'informations précieuses.

Les valeurs manquantes : imputer par une constante

- Une autre approche consiste à remplir les valeurs manquantes avec une valeur constante ou la moyenne, la médiane ou le mode des données disponibles pour cette caractéristique.
- Cette méthode est facile à mettre en œuvre mais ne fournit pas toujours les résultats les plus précis, surtout si les données manquantes ne manquent pas au hasard.
- Attention, n'imputez jamais par une donnée arbitraire.

Les valeurs manquantes : autres méthodes d'imputation

- Les méthodes d'imputation sont des techniques plus avancées qui estiment les valeurs manquantes en fonction des relations entre les entités de votre jeu de données.
- Une méthode d'imputation populaire est k-Nearest Neighbors (KNN), qui remplit les valeurs manquantes en fonction de la valeur moyenne des k points de données les plus similaires.
- Une autre option est l'imputation par la régression, qui estime les valeurs manquantes en ajustant un modèle de régression aux données disponibles.

Les valeurs manquantes : conseils

1. Identifiez les entités avec des données manquantes dans votre jeu de données.
2. Décidez d'une stratégie pour gérer les données manquantes (par exemple, suppression, remplissage ou imputation).
3. Appliquez la stratégie choisie à l'ensemble d'entraînement. Si vous utilisez l'imputation, ajustez l'imputation sur l'ensemble d'apprentissage et enregistrez les paramètres d'imputation.
4. Entraînez votre modèle d'apprentissage automatique à l'aide de l'ensemble d'entraînement traité.
5. Avant de faire des prédictions avec l'ensemble de test, appliquez la même stratégie pour gérer les données manquantes à l'aide des paramètres de l'ensemble d'apprentissage.
6. Évaluez les performances du modèle sur l'ensemble de test traité.



Skills4All
digital learning certifiant

Construction d'un modèle

Construction d'un modèle



- **Prétraitement (data pre-processing)**

- Importation des données (import)
- Nettoyage des données (data wrangling)
- Imputation des valeurs manquantes (data imputation)
- Séparation en données d'entraînement et de test (split)



- **Modélisation**

- Construction du modèle
- Entraînement du modèle
- Prédictions



- **Evaluation**

- Métriques de performances
- Avis sur les métriques



Skills4All
digital learning certifiant

Evaluation d'un modèle

Évaluation d'un modèle

- L'évaluation du modèle est le processus d'évaluation des performances d'un algorithme d'apprentissage automatique sur des données qu'il n'a jamais vu auparavant.
- Il vous aide à déterminer si le modèle se généralise bien et peut faire des prédictions précises sur de nouveaux points de données.
- Il existe différentes métriques et techniques pour évaluer les modèles, selon le type de problème que vous résolvez (classification, régression, etc.).

Évaluation d'un modèle : métriques de classification

- **Accuracy** : la proportion d'instances correctement classées par rapport au nombre total d'instances.
- **Précision** : La proportion de vrais positifs par rapport au total des positifs prédits.
- **Rappel (sensibilité)** : La proportion de vrais positifs par rapport au total des positifs réels.
- **Score F1** : La moyenne harmonique de la précision et du rappel, fournissant un équilibre entre les deux.

Évaluation d'un modèle : métriques de régression

- **Erreur absolue moyenne (MAE)** : la moyenne des différences absolues entre les valeurs prévues et réelles.
- **Erreur quadratique moyenne (MSE)** : la moyenne des différences au carré entre les valeurs prévues et réelles.
- **Root Mean Squared Error (RMSE)** : La racine carrée de l'erreur quadratique moyenne, qui est plus sensible aux grandes erreurs que MAE.
- **R au carré** : proportion de la variance de la variable dépendante qui est prévisible à partir des variables indépendantes, allant de 0 à 1.

Évaluation d'un modèle : pourquoi évaluer ?

- La sélection de modèles est le processus de sélection du meilleur modèle parmi un ensemble de candidats en fonction de leurs performances sur un ensemble de validation.
- C'est essentiel parce que différents modèles peuvent être plus performants sur différents types de données, et il y a souvent un compromis entre la complexité du modèle et les performances.
- Dans les slides suivantes, vous découvrirez les 3 techniques de sélection utilisées en Machine Learning.

Évaluation d'un modèle : train-test split

- La méthode la plus simple consiste à diviser vos données en un ensemble d'apprentissage (80%) et un ensemble de test (20%).
- Entraînez chaque modèle candidat sur l'ensemble d'entraînement et évaluez ses performances sur l'ensemble de test.
- Choisissez le modèle avec les meilleures performances sur l'ensemble de test.
- Vous mettrez en place le split 80-20 dans l'ensemble des modèles supervisés des modules de ce cours.

Évaluation d'un modèle : K-Fold Cross-Validation

- La validation croisée K-Fold est une méthode plus robuste qui divise vos données en k plis de taille égale.
- Pour chaque pli, utilisez-le comme ensemble de test tout en utilisant les plis $k - 1$ restants comme ensemble d'entraînement.
- Entraînez et évaluez chaque modèle candidat k fois et calculez la performance moyenne sur tous les plis.
- Choisissez le modèle avec les meilleures performances moyennes.
- Vous mettrez en pratique la validation croisée dans les modules suivants.

Évaluation d'un modèle : Grid Search et Random Search

- Ces méthodes sont utilisées pour optimiser les hyperparamètres d'un modèle.
- Dans Grid Search, vous définissez un ensemble de valeurs possibles pour chaque hyperparamètre, et l'algorithme essaie toutes les combinaisons possibles.
- Dans Random Search, l'algorithme échantillonne une combinaison aléatoire d'hyperparamètres à partir d'une distribution spécifiée.
- Les deux méthodes peuvent être combinées avec une validation croisée pour des résultats plus précis.

Évaluation d'un modèle : Grid Search et Random Search

- Voici un aperçu du processus d'évaluation et de sélection des modèles :
 1. Divisez votre ensemble de données en ensembles d'apprentissage et de validation (ou utilisez la validation croisée).
 2. Entraînez chaque modèle candidat sur l'ensemble d'entraînement (ou k-1 plis en validation croisée).
 3. Évaluez les performances de chaque modèle sur l'ensemble de validation (ou le k-ième pli dans la validation croisée) à l'aide des métriques d'évaluation appropriées.
 4. Comparez les performances des modèles et sélectionnez le meilleur pour votre problème.
 5. Entraînez le modèle choisi sur l'ensemble du jeu de données et utilisez-le pour faire des prédictions sur de nouvelles données.



Skills4All
digital learning certifiant

Régression linéaire simple

Régression linéaire simple



Intuition



Définition



Représentation graphique



Interprétation des résultats



Code Python

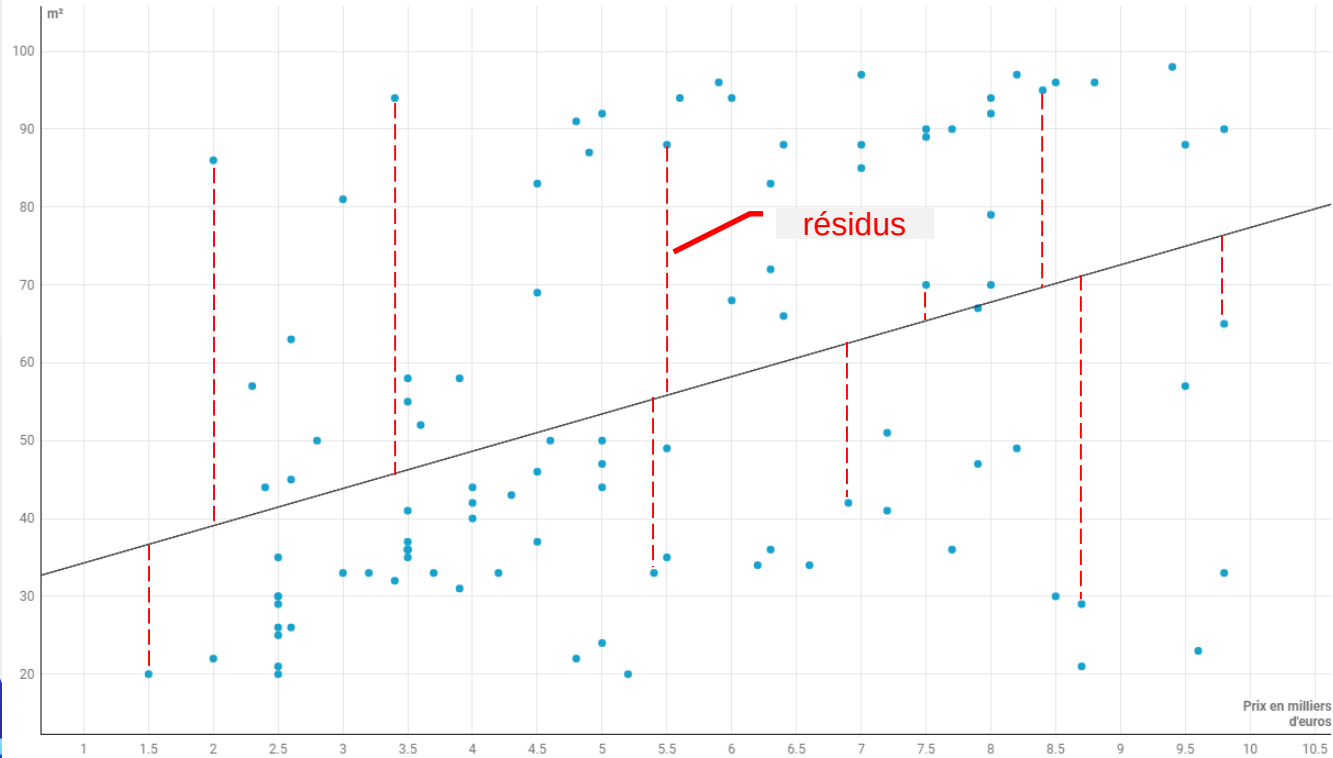
Régression linéaire simple : intuition

- Imaginez que vous ayez un ensemble de données contenant les surfaces des maisons (en m^2) et leurs prix.
- Vous souhaitez créer un modèle capable de **prédire** le prix d'une maison en fonction de sa taille.
- La régression linéaire vous aide à trouver la « ligne de meilleur ajustement » qui représente la relation entre la taille de la maison et son prix.
- Cette ligne peut être utilisée pour faire des **prédictions** pour les nouvelles tailles de maison.
- Intuitivement, la ligne de meilleur ajustement est celle qui minimise la distance totale entre les prix immobiliers réels et les prix immobiliers prédits (l'erreur). En ajustant les coefficients (la pente et l'ordonnée à l'origine de la ligne), nous pouvons trouver la ligne qui représente le mieux la relation entre la caractéristique d'entrée (taille de la maison) et la variable cible (prix de la maison).

Régression linéaire simple : intuition

Simple Linear Regression

Prix en fonction de la surface en m²



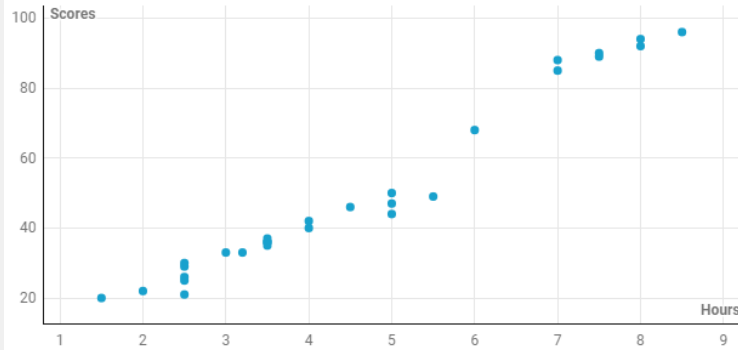
Régression linéaire simple

- L'essentiel à savoir : définition
 - Méthode statistique permettant de modéliser la relation linéaire entre deux variables quantitatives.
 - Par exemple, on dispose du nombre d'heures d'études de quelques étudiants et leurs notes obtenues à l'examen.
 - Il est possible de construire un modèle linéaire qui saurait :
 - Expliquer l'impact du nombre d'heures sur les notes à l'examen
 - Prédire la note qu'un étudiant pourrait obtenir en fonction des heures de révision
 - Objectif double de la régression linéaire :
 - **Explicatif** : montrer à quel point la variable à expliquer impacte la variable explicative.
 - **Prévisionnel** : valeur espérée prise par la variable expliquée.

Régression linéaire simple

Simple Linear Regression

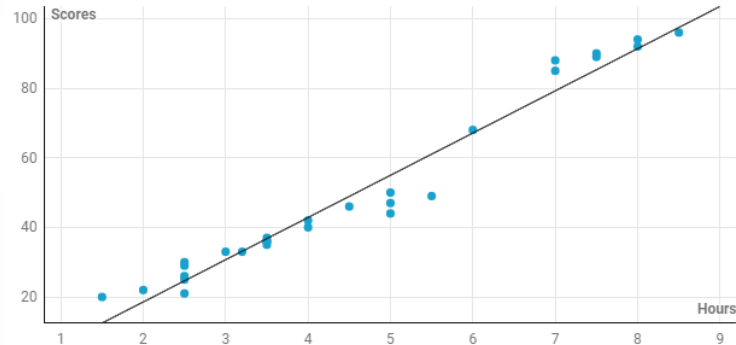
Score en fonction du temps d'étude



- Ici on veut modéliser la relation linéaire entre le nombre d'heures de révisions (X) et le score obtenu à l'examen (Y).

Simple Linear Regression

Score en fonction du temps d'étude

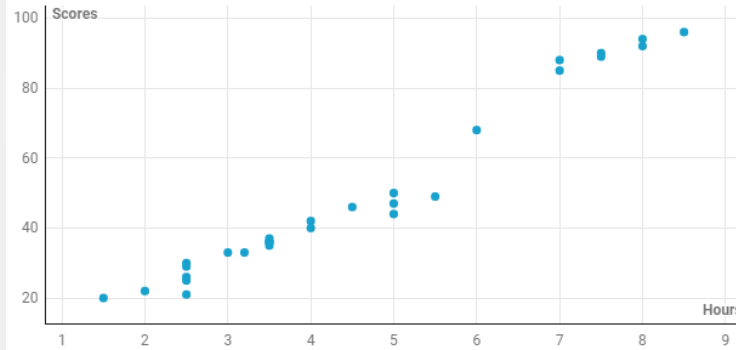


- Idée sous-jacente : Le score à l'examen peut être expliqué par le nombre d'heures de révisions.

Régression linéaire simple

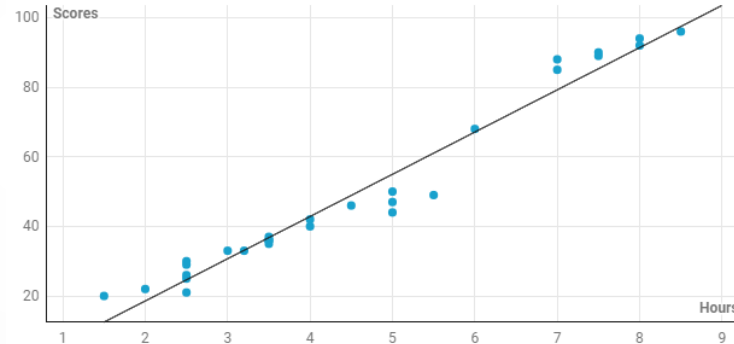
Simple Linear Regression

Score en fonction du temps d'étude



Simple Linear Regression

Score en fonction du temps d'étude



Variable à
expliquer

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

Erreur de mesure

Variable explicative

La pente de la droite

Ordonnée à l'origine

β_0 et β_1 sont des paramètres inconnus qu'on cherche à estimer.

Régression linéaire simple

- L'essentiel à savoir : interprétation des résultats
 - La valeur de b_0 , également appelée interception, indique le point où la ligne de régression estimée croise l'axe y .
 - C'est la valeur de la réponse estimée $f(x)$ pour $x = 0$. La valeur de b_1 détermine la pente de la droite de régression estimée.

Régression linéaire simple

- L'essentiel à savoir : interprétation des résultats
 - D'abord regarder le signe de β_1 . Si positif, alors il influence y de façon à l'augmenter en moyenne.
 - Si négatif, il a tendance à baisser y en moyenne (c'est important de dire EN MOYENNE). Par exemple, si $\beta_1 = +0.21$, alors en moyenne, à chaque fois que x augmente d'une unité (ou baisse selon la valeur de β), y augmente sa valeur de $+0.21$ unités (km, tonnes, litres, degrés...).

Régression linéaire simple

- L'essentiel à savoir : interprétation des résultats
 - La constante β_0 est souvent difficile à interpréter car peut révéler des absurdités.
 - Cette interprétation vaut le coup, lorsque $x = 0$ a du sens. Quoiqu'il en soit, il a son importance pour la construction du modèle général (pente, force).

Régression linéaire simple : prédiction vs prévision

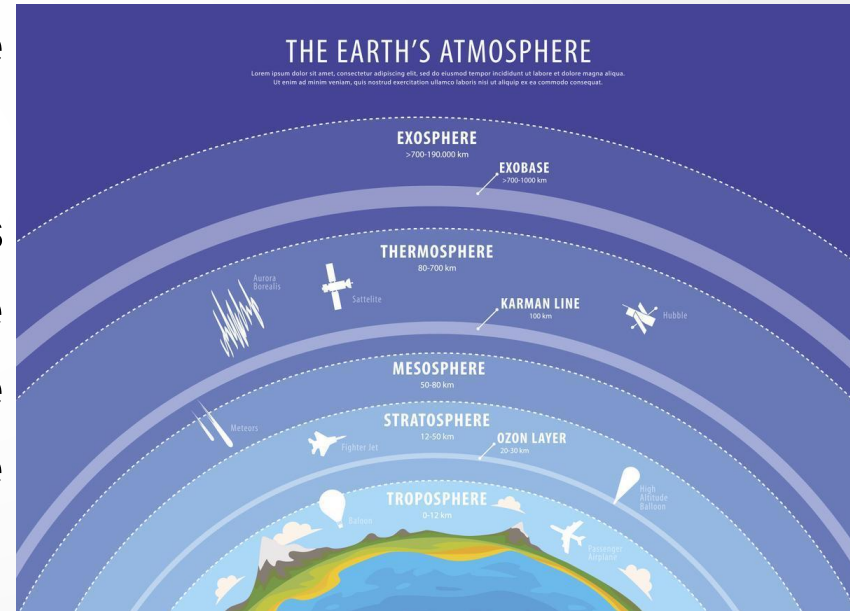
- En science des données, les termes « prédiction » et « prévision » sont souvent utilisés de manière interchangeables, mais ils peuvent avoir des significations légèrement différentes selon le contexte.
- En général, la prédiction est le processus de faire une estimation ou une projection quant à ce qui se produira dans l'avenir en se basant sur des données historiques.
- Les prévisions en revanche sont également basées sur des données historiques, mais elles impliquent souvent l'utilisation de modèles statistiques plus complexes pour estimer les tendances futures et les variations possibles.

Régression linéaire simple : prédiction vs prévision

- Plus précisément, la prédiction peut être considérée comme une estimation de ce qui se passera dans le futur en se basant sur des données historiques, tandis que la prévision prend en compte des facteurs supplémentaires tels que les tendances saisonnières, les effets externes et les fluctuations possibles pour produire une estimation plus précise des résultats futurs.
- En résumé, la prédiction se concentre principalement sur la projection de résultats futurs en se basant sur les données historiques, tandis que la prévision prend en compte les facteurs supplémentaires pour produire une estimation plus précise.

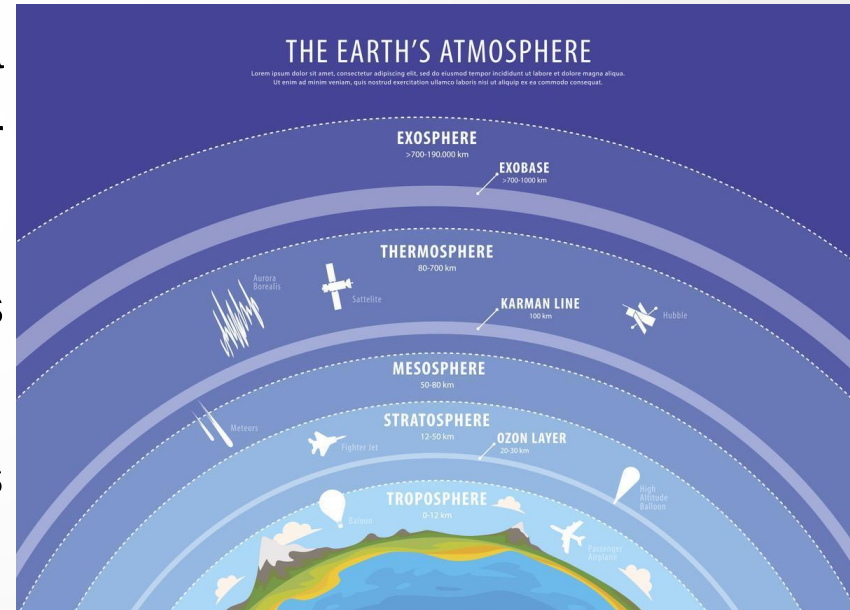
Ozone

- L'ozone est un gaz qui joue un rôle important dans la chimie atmosphérique.
- Une concentration élevée en ozone dans l'atmosphère signifie généralement une quantité importante de molécules d'ozone présentes dans une région donnée de l'atmosphère terrestre.



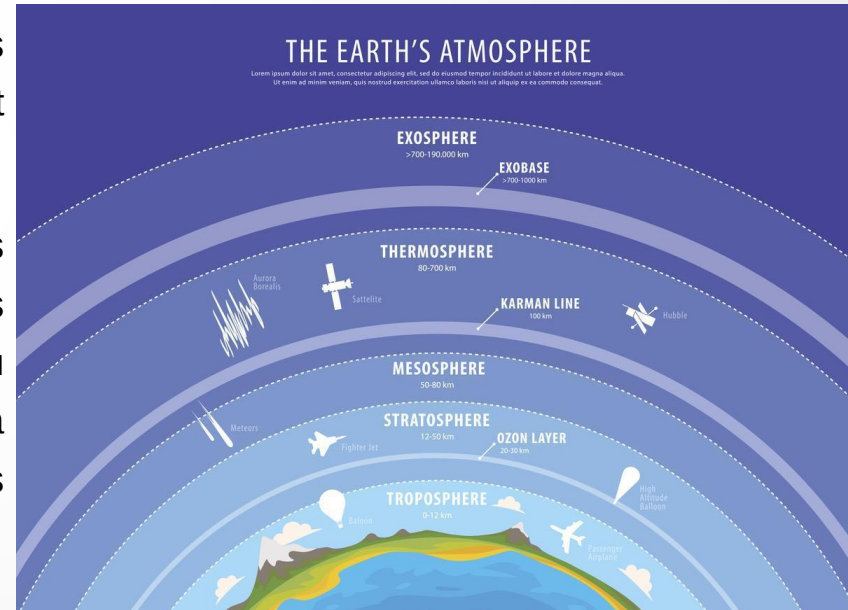
Ozone

- Dans la stratosphère, l'ozone forme la couche d'ozone, qui est essentielle pour filtrer les rayons ultraviolets (UV) du soleil. Cela protège la vie sur Terre des effets nocifs des UV, tels que les coups de soleil, les cancers de la peau et les dommages oculaires.



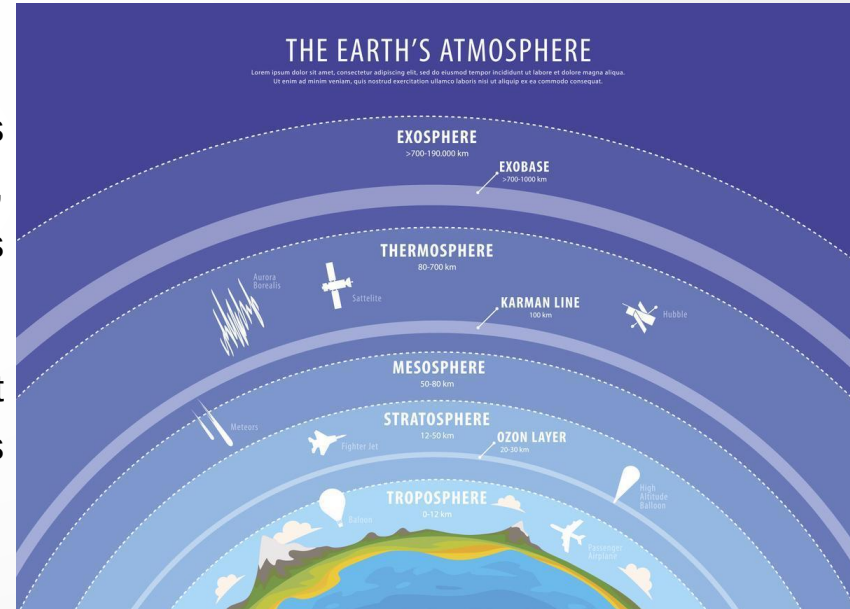
Ozone

- Cependant, à des altitudes plus basses, une concentration élevée en ozone peut être problématique. Dans la troposphère, la couche atmosphérique la plus basse, l'ozone est considéré comme un polluant atmosphérique.
- Il est principalement formé par des réactions chimiques entre les oxydes d'azote (NOx) et les composés organiques volatils (COV) en présence de la lumière du soleil. Les principales sources de NOx et de COV sont la pollution automobile, les émissions industrielles et les produits chimiques émis par certaines plantes.



Ozone

- Une concentration élevée en ozone dans la troposphère peut avoir des effets néfastes sur la santé humaine et l'environnement.
- L'ozone troposphérique est un irritant des voies respiratoires et peut causer des problèmes respiratoires, notamment des difficultés respiratoires, de l'asthme, des irritations pulmonaires et des réactions allergiques.
- Il peut également endommager les cultures, les forêts et les écosystèmes en perturbant les processus biologiques des plantes et en réduisant la photosynthèse.





Skills4All
digital learning certifiant

Régression linéaire multiple

Régression linéaire multiple



Intuition



Définition



Représentation graphique



Interprétation des résultats



Code Python

Régression linéaire multiple

- L'essentiel à savoir : définition
 - Méthode statistique permettant de modéliser la relation linéaire entre une variable quantitative (y) et au moins deux autres variables quantitatives (x).

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon$$

Diagram illustrating the components of the multiple linear regression equation:

- Y : Variable à expliquer
- β_0 : Ordonnée à l'origine
- $\beta_1, \beta_2, \dots, \beta_p$: Coefficient
- $x_1, x_2, \dots, x_p$: Variable explicative
- ε : Erreur de mesure

$\beta_0, \beta_1, \dots, \text{et } \beta_p$ sont des paramètres inconnus qu'on cherche à estimer.

Régression linéaire multiple

- L'essentiel à savoir : représentation graphique



Régression linéaire multiple

- L'essentiel à savoir : interprétation des résultats
 - La valeur de b_0 , également appelée interception, indique le point où la ligne de régression estimée croise l'axe y .
 - C'est la valeur de la réponse estimée $f(x)$ pour $x = 0$. La valeur de b_1 détermine la pente de la droite de régression estimée.

Régression linéaire multiple

- L'essentiel à savoir : interprétation des résultats
 - La constante bêta 0 est souvent difficile à interpréter car peut révéler des absurdités.
 - Un exemple concret sera présenté avec un autre jeu de données dans la partie exercices.
 - Cette interprétation vaut le coup, lorsque $x = 0$ a du sens. Quoiqu'il en soit, il a son importance pour la construction du modèle général (pente, force).



Skills4All
digital learning certifiant

Régression logistique multiple

Régression logistique



Intuition



Définition



Représentation graphique



Interprétation des résultats



Code Python

Régression logistique : intuition

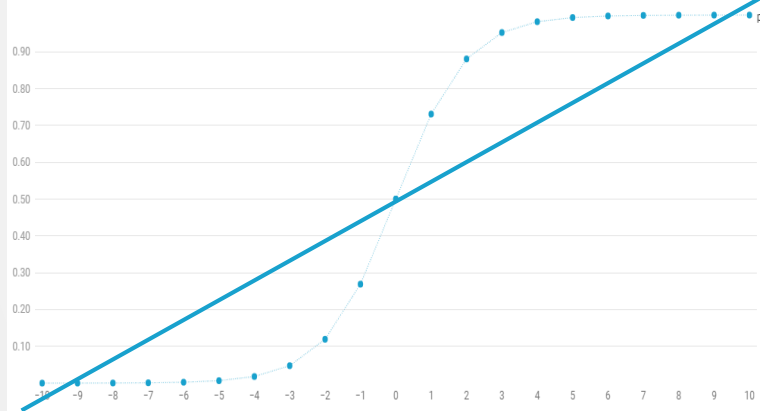
- Imaginez que vous disposez d'un ensemble de données contenant des informations sur les tumeurs, telles que leurs tailles et si elles sont bénignes (non cancéreuses) ou malignes (cancéreuses).
- Vous souhaitez créer un modèle capable de prédire si une tumeur est bénigne ou maligne en fonction de sa taille.
- Puisqu'il s'agit d'un problème de classification binaire, la régression logistique est un choix approprié.

Régression logistique : intuition

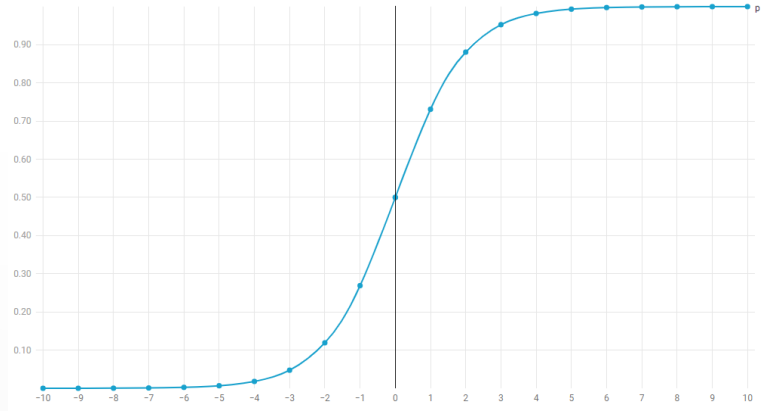
- Au lieu d'ajuster une ligne droite, comme dans la régression linéaire, la régression logistique ajuste une courbe en forme de "S" (la fonction sigmoïde) qui représente la probabilité qu'une tumeur soit maligne.
- La sortie du modèle est une valeur comprise entre 0 et 1, qui peut être interprétée comme la probabilité que la tumeur soit maligne (classe 1).
- En ajustant les coefficients, nous pouvons trouver la courbe qui représente le mieux la relation entre la caractéristique d'entrée (taille de la tumeur) et la variable cible (bénigne ou maligne).

Régression logistique : intuition

Linear function



Sigmoid function



Régression logistique : intuition

- Dans les deux cas, les modèles apprennent les coefficients en minimisant une fonction objectif :
 - Pour la **régression linéaire**, la fonction objectif est l'erreur quadratique moyenne (MSE), qui représente la moyenne des différences au carré entre les valeurs cibles prévues et réelles.
 - Pour la **régression logistique**, la fonction objectif est la perte de log (ou perte d'entropie croisée), qui mesure la différence entre les probabilités prédites et les valeurs cibles binaires réelles.

Régression logistique

- L'essentiel à savoir : définition
 - La régression logistique a pour objectif d'expliquer et de prédire les valeurs d'une variable qualitative binaire (y) à partir de variables explicatives quantitatives et qualitatives (x).

$$\log\left(\frac{p(x)}{1 - p(x)}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Diagram annotations:

- événement (pointing to $p(x)$)
- coefficient (pointing to β_0)
- événement contraire (pointing to $1 - p(x)$)
- une réalisation de x (pointing to x_1)

$\beta_1, \dots, \beta_p$ sont des paramètres inconnus estimés par la méthode du maximum de vraisemblance

Régression logistique

- Ici, y ou $\log(p(x)/1 - p(x))$ est la probabilité prédite.
- $x_1, x_2, x_3, \dots, x_n$ sont les features ou variables explicatives.
- $\beta_0, \beta_1, \beta_2, \dots, \beta_n$ sont les coefficients qui définissent la relation entre les features et la variable cible y , avec $y = 1$.
- Le but de la régression logistique est de trouver les coefficients qui maximisent la vraisemblance des données observées.

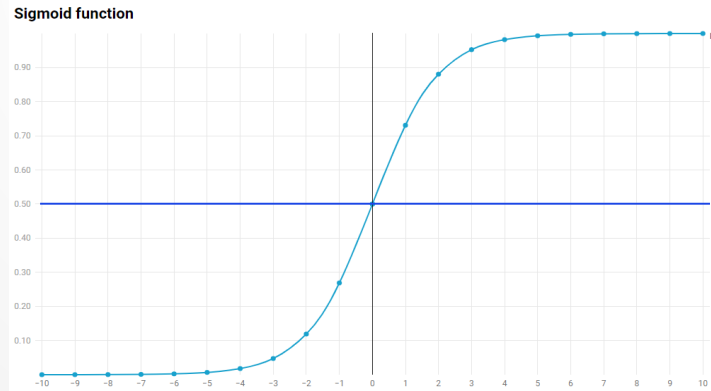
$$\log\left(\frac{p(x)}{1 - p(x)}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Diagram annotations:

- événement (pointing to $p(x)$)
- coefficient (pointing to β_0)
- événement contraire (pointing to $1 - p(x)$)
- une réalisation de X (pointing to x_1)

Régression logistique : fonction sigmoïde

- Cette probabilité est modélisée par une courbe appelée « courbe sigmoïde », bornée par 0 et 1 puisqu'une probabilité est bornée entre 0 et 1.
- Cette courbe est définie par la fonction logistique que nous avons présenté précédemment.



Régression logistique : interprétation des métriques

- **Odds Ratio** : ou rapport des cotes, est le rapport de deux probabilités complémentaires : la probabilité P de survenue d'un événement (risque), divisé par la probabilité $(1 - P)$ que cet événement ne survienne pas (non risque, c'est-à-dire sans l'événement).
- **Regarder le signe** :
 - si $+$ alors la variable x augmente le risque de maladie par exemple.
 - Si $-$ alors, on a moins de chance d'être malade.

Régression logistique : interprétation des métriques

- Si **Odds Ratio** >1 , alors on a plus de chances d'être positif parmi le groupe exposé (cancer = 1) par rapport au groupe témoin (cancer = 0).
- Si **Odds Ratio** $=1$, alors on a autant de chances d'être positif dans un groupe, comme dans l'autre.
- Si **Odds Ratio** <1 , alors on a moins de chances d'être positif parmi le groupe exposé (cancer = 1) par rapport au groupe témoin (cancer = 0).

Régression logistique : interprétation des métriques

- **Odds Ratio :**

$$\log\left(\frac{p(x)}{1-p(x)}\right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$

Passer le coef en
Exponentielle nous
donne les Odds Ratio

Passer le **coef** en
Exponentielle indique donc
que x_1 a $s\%$ de chances
d'appartenir à Cancer = 1.

- **Risque relatif :** c'est le surcroit de chance d'être positif parmi le groupe exposé (cancer = 1) par rapport au groupe témoin (cancer = 0). L'idée est de dire que les personnes qui ont par exemple une tumeur de grande taille, ont $x\%$ de chances d'avoir un cancer.



Skills4All
digital learning certifiant

Arbre de décision

Arbre de décision



Intuition



Représentation graphique



Mesure d'impureté



Interprétation des résultats

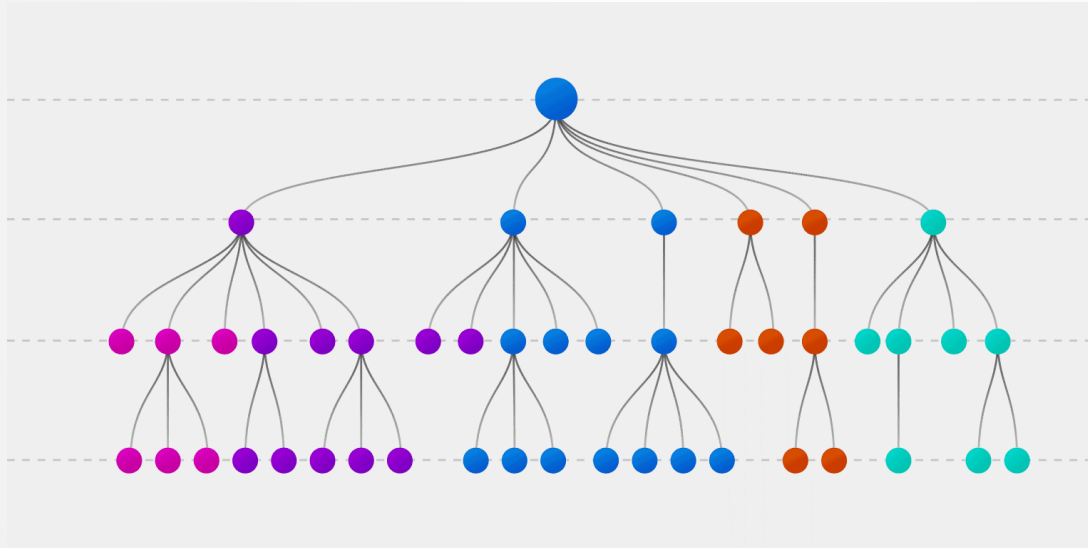


Code Python

Arbre de décision : intuition

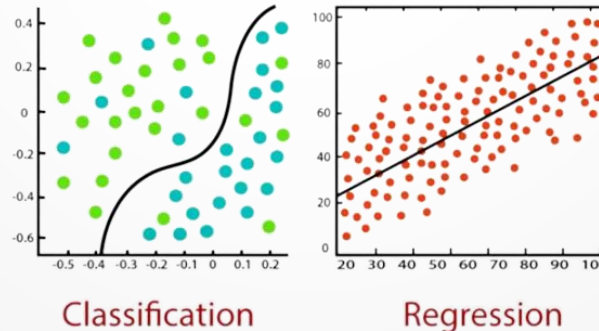
- Un arbre de décision est une structure de type organigramme dans laquelle chaque nœud interne représente une décision basée sur une caractéristique d'entrée, chaque branche représente un résultat de la décision et chaque nœud feuille représente une étiquette de classe ou une valeur prédite.
- Les décisions sont prises en divisant de manière récursive l'ensemble de données en fonction d'une valeur de caractéristique ou d'une condition spécifique, ce qui se traduit par une structure arborescente.

Arbre de décision : graphique



Arbre de décision : intuition

- Les arbres de décisions servent aussi bien pour la régression que pour la classification.
- C'est un outil d'exploration des données et d'aide à la décision qui permet d'expliquer et de prédire une variable quantitative (arbre de régression) ou qualitative (arbre de segmentation) à partir de variables explicatives quantitatives et/ou qualitatives.

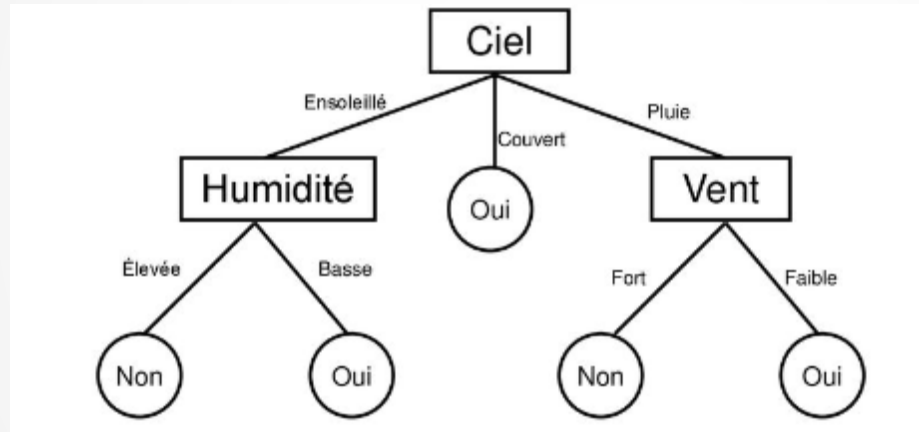


Arbre de décision : la construction d'un arbre

- Pour construire un arbre de décision, nous devons choisir la **meilleure feature** et le meilleur **seuil** pour diviser l'ensemble de données à chaque nœud.
- Le choix dépend de l'objectif du problème (classification ou régression) et de la mesure d'impureté utilisée (impureté de Gini, entropie ou erreur quadratique moyenne).
- Les étapes générales pour construire un arbre de décision sont :
 - a) Commencez avec l'ensemble de données complet au nœud racine.
 - b) Sélectionnez la meilleure caractéristique et le meilleur seuil pour diviser l'ensemble de données.
 - c) Créez des nœuds enfants en divisant l'ensemble de données en fonction de la fonctionnalité et du seuil sélectionnés.
 - d) Répétez les étapes b) et c) pour chaque nœud enfant jusqu'à ce qu'un critère d'arrêt soit satisfait (par exemple, profondeur maximale de l'arbre, nombre minimal d'échantillons par feuille ou nœuds purs)

Arbre de décision : exemple 1

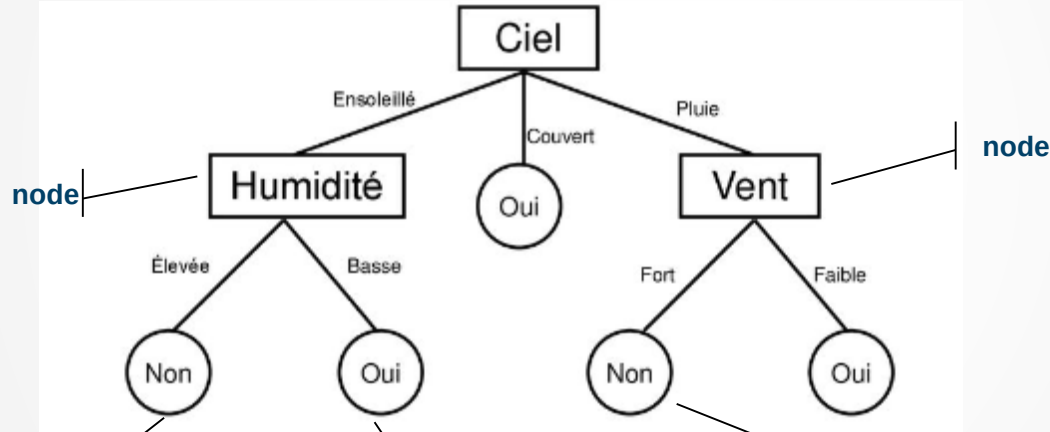
Un tennisman se demande s'il peut aller jouer dehors



Arbre de décision : exemple 1

Un tennisman se demande s'il peut aller jouer dehors

Root node : racine de l'arbre.
Point de départ de notre arbre de décision.



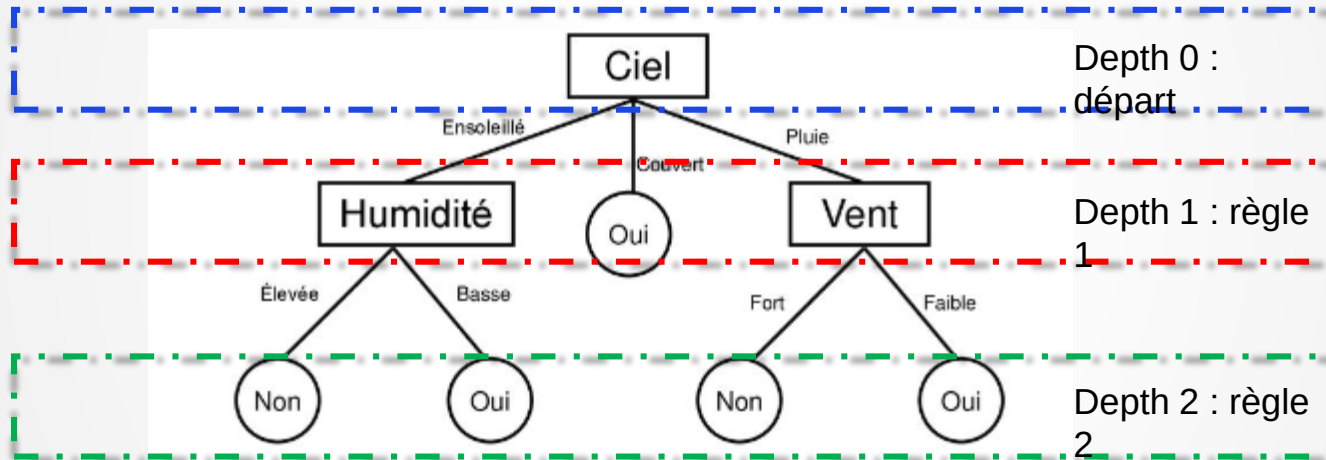
Leaf node : feuille de l'arbre.

Leaf node : feuille de l'arbre.

Leaf node : feuille de l'arbre.

Arbre de décision : exemple 1

Un tennisman se demande s'il peut aller jouer dehors



Arbre de décision : implémentations

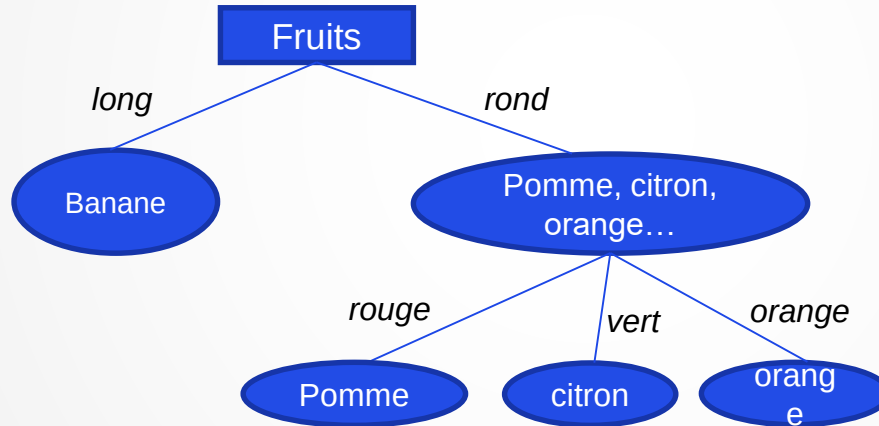
- Il existe différentes implémentations d'arbres de décisions :
 - ID3 : spécialisé sur les variables quantitatives nominales (classement)
 - C4.5 : s'applique aussi bien sur des données quantitatives que qualitatives
 - C5.0 : extension commerciale de C4.5
 - CART : s'applique sur des données quanti et quali, supporte la régression.

Arbre de décision : la construction d'un arbre

- Imaginez un ensemble de données contenant des informations sur les fruits, telles que leurs couleurs, leurs formes et leurs tailles.
- Nous voulons créer un modèle qui peut classer un fruit comme une pomme, une orange ou une banane en fonction de ces caractéristiques.
- Un arbre de décision peut être construit pour représenter les décisions qui conduisent à la classification correcte d'un fruit :
 - Si le fruit est long, c'est une banane.
 - Si le fruit est rond et a une couleur plus proche du rouge, c'est une pomme.
 - Si le fruit est rond et a une couleur plus proche de l'orange, c'est une orange.

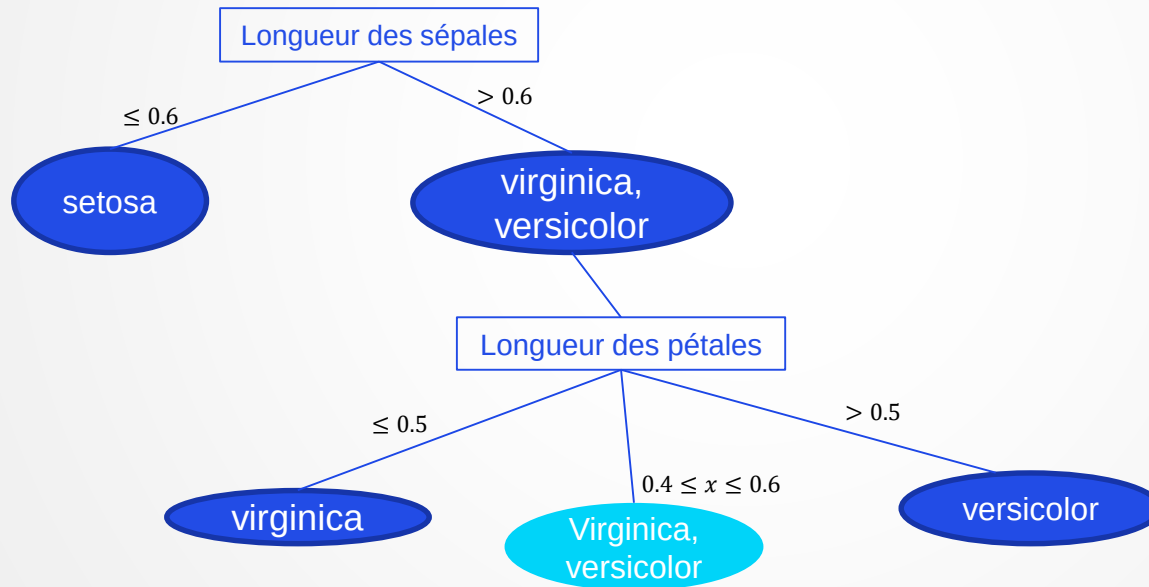
Arbre de décision : exemple 2

Un maraicher se demande comment catégoriser ses fruits



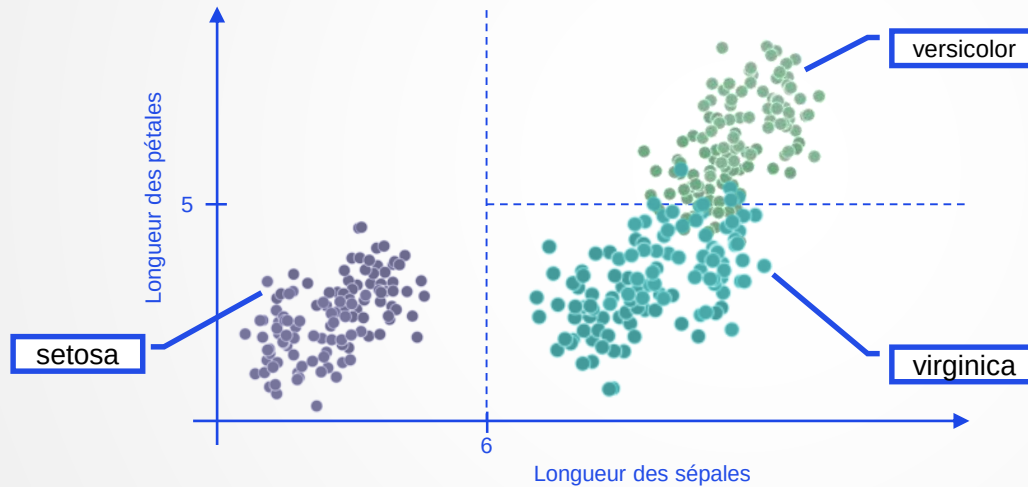
Arbre de décision : exemple 3

Un botaniste se demande comment catégoriser des plantes



Arbre de décision : exemple 3

Un botaniste se demande comment catégoriser des plantes



Arbre de décision : construction d'un arbre

- Synthèse :
 - On commence par trouver la variable qui est la plus appropriée pour la construction de l'arbre (racine) : Entropie (ID3) ou Gini (CART).
 - On réitère l'opération jusqu'à ce qu'on ne peut plus séparer les données

Arbre de décision : mesures d'impureté

- Les mesures d'impuretés sont utilisées pour évaluer la qualité d'une division dans un arbre de décision. Ils aident à quantifier le degré de «mélange» des classes ou des valeurs cibles au sein d'un nœud.
- Une impureté plus faible indique une meilleure répartition. Les mesures d'impuretés courantes comprennent :
 - **Impureté de Gini** : utilisée pour les problèmes de classification, l'impureté de Gini mesure la probabilité de mal classer une instance si elle était étiquetée de manière aléatoire en fonction de la distribution des classes dans le nœud.
 - **Entropie** : également utilisée pour les problèmes de classification, l'entropie mesure la quantité de « désordre » ou « d'incertitude » dans le nœud. Il est basé sur le concept de la théorie de l'information.
 - **Erreur quadratique moyenne (MSE)** : utilisée pour les problèmes de régression, la MSE mesure la différence quadratique moyenne entre les valeurs cibles réelles et prédites.

Arbre de décision : sélection de la meilleure feature avec l'entropie

- Soit un ensemble de classes k données par y :
 - L'entropie est donnée par : $E(S) = -\sum_{i=1}^K p_i \times \log_2(p_i)$
 - En fait p_i revient à diviser le nombre d'échantillon de la classe par le nombre d'échantillon total (cardinalités) : $p_i = \frac{|I_i|}{|S|}$
 - L'entropie est toujours comprise entre 0 et 1
 - L'attribut ayant un meilleur gain d'information (valeur la plus élevée) est sélectionné pour la construction de l'arbre :

$$Gain(S, attribut) = E(S) - \sum_{i=1}^K p_i \times E(S_i)$$

Arbre de décision : sélection de la meilleure feature avec l'entropie

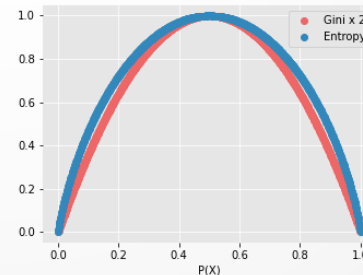
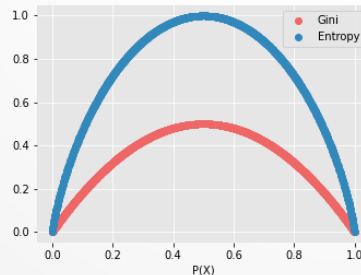
- La division optimale est choisie par la caractéristique ayant moins d'entropie.
- Il obtient sa valeur maximale lorsque la probabilité des deux classes est la même et un nœud est pur lorsque l'entropie a sa valeur minimale, qui est 0 :
 - $Entropy_{min} = -1 \times \log_2(1) = 0$
 - $Entropy_{max} = -0.5 \times \log_2(0.5) - 0.5 \times \log_2(0.5) = 1$

Arbre de décision : sélection de la meilleure feature avec Gini

- On cherchera également quel attribut constituera la racine de l'arbre, et comment seront répartis les nœuds.
- L'impureté Gini mesure la fréquence à laquelle tout élément de l'ensemble de données sera mal étiqueté lorsqu'il est étiqueté de manière aléatoire.
- La valeur minimale de l'indice de Gini est 0. Cela se produit lorsque le nœud est pur, cela signifie que tous les éléments contenus dans le nœud appartiennent à une classe unique. Par conséquent, ce nœud ne sera plus divisé.
- Ainsi, la répartition optimale est choisie sur la base des caractéristiques ayant un indice de Gini faible. De plus, il obtient la valeur maximale lorsque la probabilité des deux classes est la même.
 - $Gini_{min} = 1 - (1^2) = 0$
 - $Gini_{max} = 1 - (0.52 + 0.52) = 0.5$

Arbre de décision : entropie vs Gini

- L'indice de Gini et l'entropie ont deux différences principales :
- **Première différence** : L'indice de Gini a des valeurs à l'intérieur de l'intervalle $[0, 0.5]$ alors que l'intervalle de l'entropie est $[0, 1]$.
- **Seconde différence** : Sur le plan informatique, l'entropie est plus complexe car elle utilise des logarithmes et, par conséquent, le calcul de l'indice de Gini sera plus rapide.
- Dans la figure ci-dessous, les deux sont représentés. L'indice de Gini a également été représenté multiplié par deux pour voir concrètement les différences entre eux, qui ne sont pas très significatives.



Arbre de décision : overfitting et pruning

- Les arbres de décision peuvent devenir très complexes, entraînant un **overfitting** s'ils ne sont pas correctement contrôlés.
- L'overfitting se produit lorsque le modèle capture le bruit dans les données et perd sa capacité à se généraliser sur de nouvelles données.
- Pour éviter l'overfitting, nous pouvons utiliser des techniques comme l'élagage, qui consiste à supprimer certaines branches ou nœuds de l'arbre en fonction de critères spécifiques.

Arbre de décision : choix de la feature et du seuil

- Lors de la construction d'un arbre de décision, nous devons choisir la feature la plus pertinente et le seuil optimal pour diviser l'ensemble de données à chaque nœud.
- La meilleure feature est celle qui fournit le plus de gain d'informations (ou la plus grande réduction d'impuretés) lors de la division de l'ensemble de données (cf. mesures d'impuretés).
- Le seuil est la valeur utilisée pour diviser les données en deux groupes en fonction de la fonctionnalité choisie.
- Par exemple, dans le problème de classification des fruits, nous pourrions choisir la caractéristique « forme » et un seuil de « rond » pour diviser l'ensemble de données, ce qui signifie que tous les fruits ronds vont à un nœud enfant et tous les fruits non ronds vont à l'autre nœud enfant.

Arbre de décision : critère d'arrêt

- Les critères d'arrêt sont des conditions qui déterminent le moment où l'algorithme de l'arbre de décision doit arrêter la croissance de l'arbre.
- Les critères d'arrêt courants comprennent :
 - **Profondeur maximale de l'arborescence** : limitez la profondeur maximale de l'arborescence pour éviter tout ajustement excessif (overfitting).
 - **Échantillons minimum par feuille** : exigez un nombre minimum d'instances dans chaque nœud feuille pour garantir que l'arbre ne devienne pas trop spécifique aux données d'apprentissage.
 - **Nœuds purs** : arrêtez de développer l'arborescence lorsque toutes les instances d'un nœud appartiennent à la même classe (pour la classification) ou ont la même valeur cible (pour la régression).

Arbre de décision : pruning

- L'élagage est une technique utilisée pour simplifier un arbre de décision et réduire l'overfitting en supprimant les branches ou les nœuds qui fournissent peu ou pas d'informations supplémentaires.
- Deux types courants d'élagage sont:
 - **Pré-élagage** : arrêtez la croissance de l'arbre en fonction des critères d'arrêt lors de la construction de l'arbre.
 - **Post-élagage** : créez d'abord l'arbre complet, puis supprimez les branches ou les nœuds en fonction d'un ensemble de données de validation ou de mesures de complexité (par exemple, le nombre de nœuds ou la profondeur de l'arbre).

Arbre de décision : avantages

- Facile à comprendre et à interpréter.
- Peut gérer à la fois les caractéristiques numériques et catégorielles.
- Nécessite peu de prétraitement des données (par exemple, pas besoin de mise à l'échelle des fonctionnalités).

Arbre de décision : inconvénients

- Enclin à l'overfitting, en particulier lorsqu'il s'agit de données bruyantes ou d'un grand nombre de features.
- Peut être sensible à de petits changements dans les données, conduisant à des arborescences instables.
- La nature gourmande de l'algorithme peut ne pas aboutir à l'arbre optimal.

Arbre de décision : classification

- On utilise les arbres de classification pour expliquer et/ou prédire l'appartenance d'individus à une classe (ou modalité ou catégorie) d'une variable qualitative, sur la base de variables explicatives quantitatives et/ou qualitatives.

Arbre de décision : classification

- **Jeu de données IRIS :**
 - Pour chaque type de plante, on dispose de quelques caractéristiques.
 - A partir de ces dernières, on veut prédire à quel type de plante elles appartiennent.

iris setosa



petal sepal

iris versicolor



petal sepal

iris virginica



petal sepal

Arbre de décision : classification

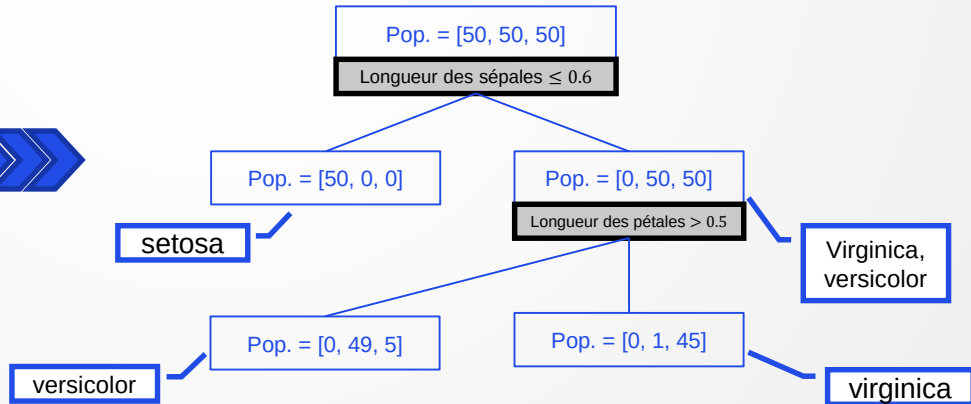
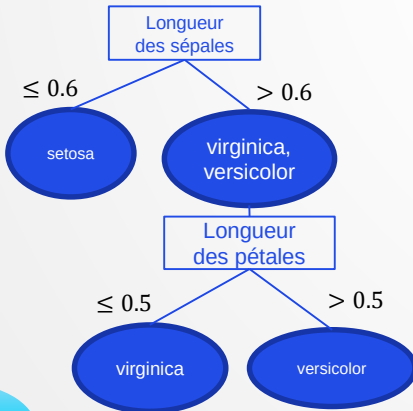
- Jeu de données IRIS :

Sepal length ⇅	Sepal width ⇅	Petal length ⇅	Petal width ⇅	Species ⇅
5.2	3.5	1.4	0.2	<i>I. setosa</i>
4.9	3.0	1.4	0.2	<i>I. setosa</i>
4.7	3.2	1.3	0.2	<i>I. setosa</i>
4.6	3.1	1.5	0.2	<i>I. setosa</i>
5.0	3.6	1.4	0.3	<i>I. setosa</i>
5.4	3.9	1.7	0.4	<i>I. setosa</i>
4.6	3.4	1.4	0.3	<i>I. setosa</i>
7.0	3.2	4.7	1.4	<i>I. versicolor</i>
6.4	3.2	4.5	1.5	<i>I. versicolor</i>

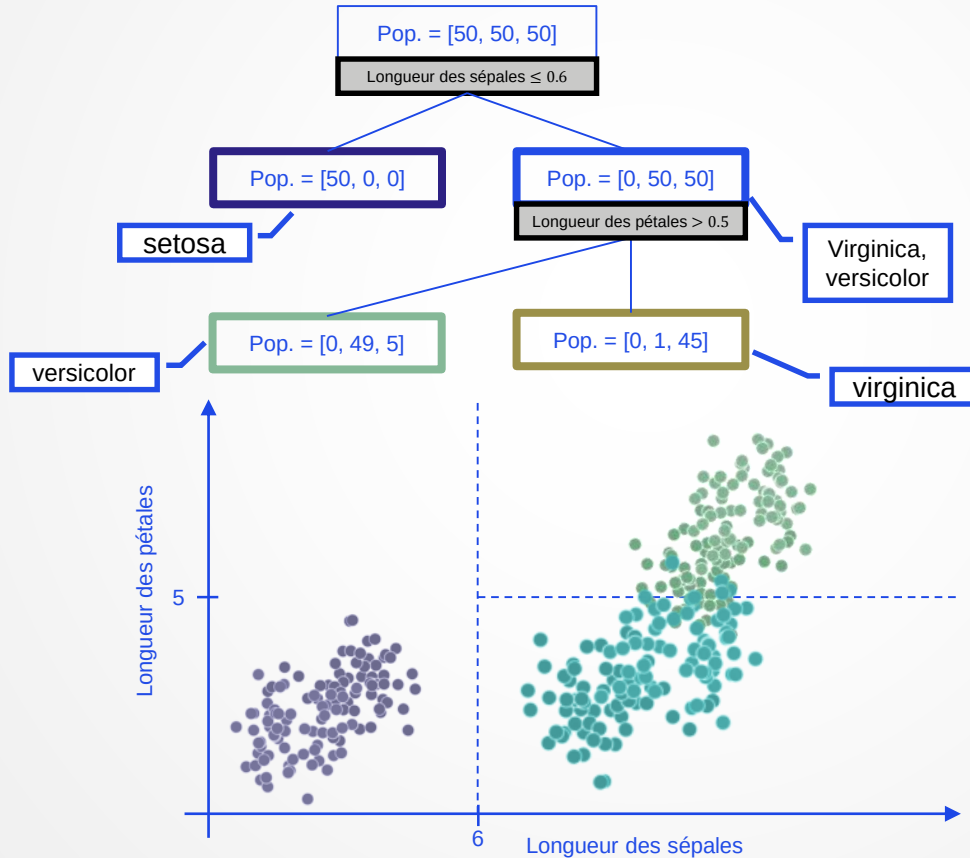
Arbre de décision : pureté de l'arbre

- **Jeu de données IRIS :**

- On cherche la pureté dans l'arbre
- Dans l'idéal, on voudrait que la classification {setosa, versicolor, virginica} s'opère de façon à obtenir un type de fleur par nœud.
- Le but est de créer un arbre avec des nœuds les plus purs possibles.



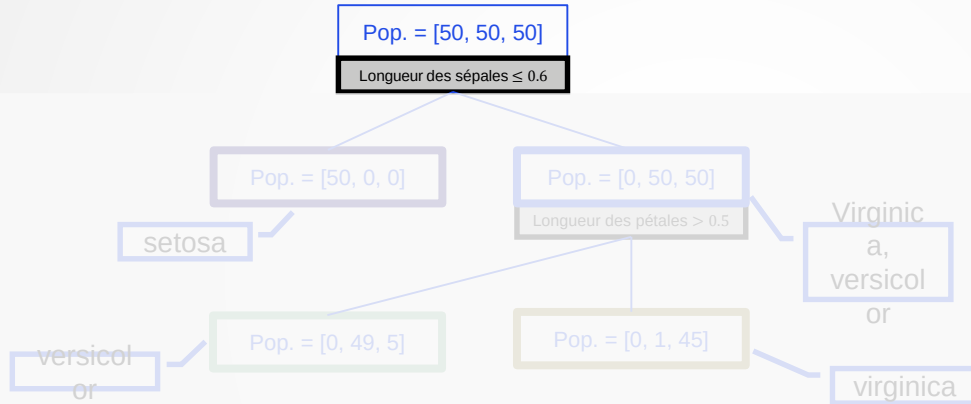
Arbre de décision : pureté de l'arbre



Arbre de décision : pureté de l'arbre, GINI

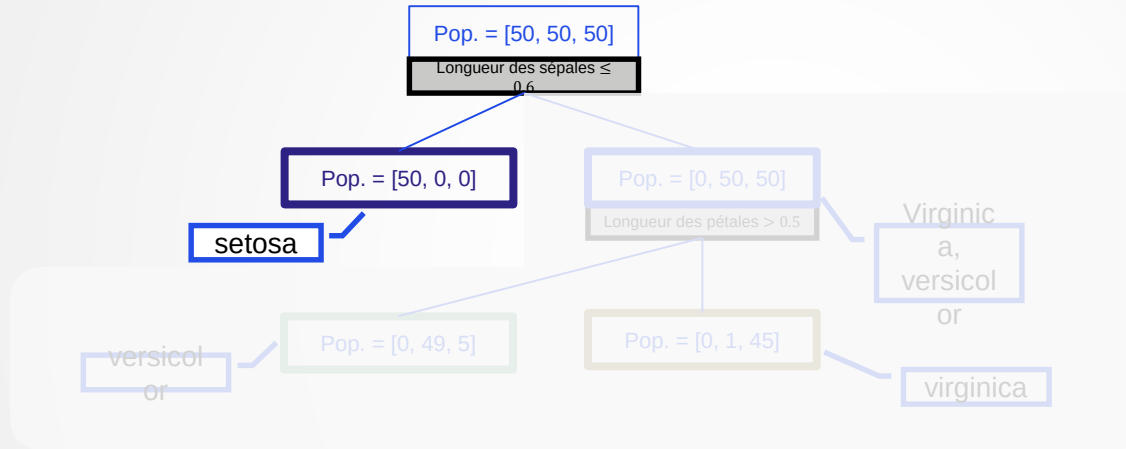
- Gini est un critère permettant de calculer la pureté des nœuds.
- C'est la somme de la probabilité d'appartenance à chaque classe élevée au carré.
- Plus le Gini est petit (proche de 0), plus le nœud est pur.
- Gini est donc un critère à minimiser.
- $G_i = 1 - \sum_{k=1}^n p_{i,k}^2$
- $p_{i,k}$ est le ratio du nombre d'individus de la classe k parmi la population du $i^{ème}$ nœud.

Arbre de décision : pureté de l'arbre, GINI



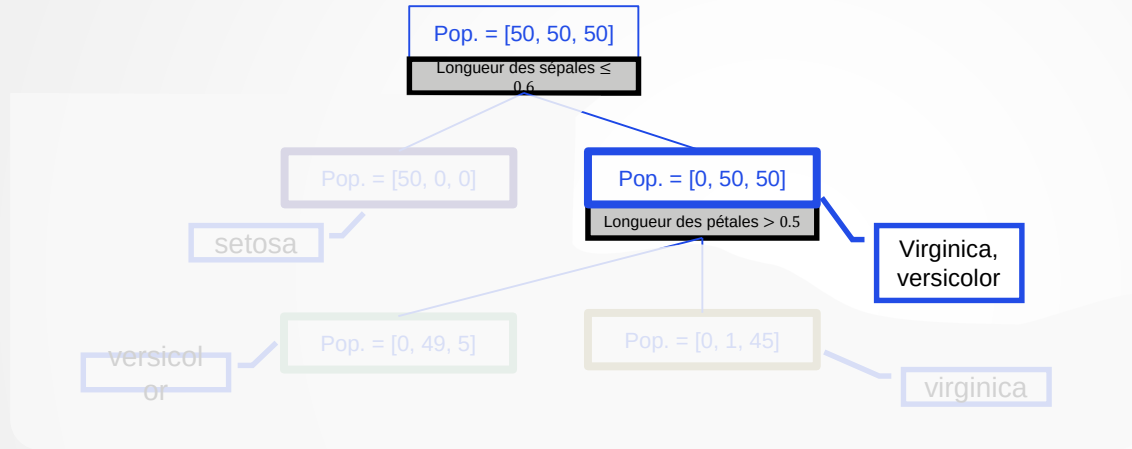
$$gini = 1 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 - \left(\frac{50}{150}\right)^2 = 0.667$$

Arbre de décision : pureté de l'arbre, GINI



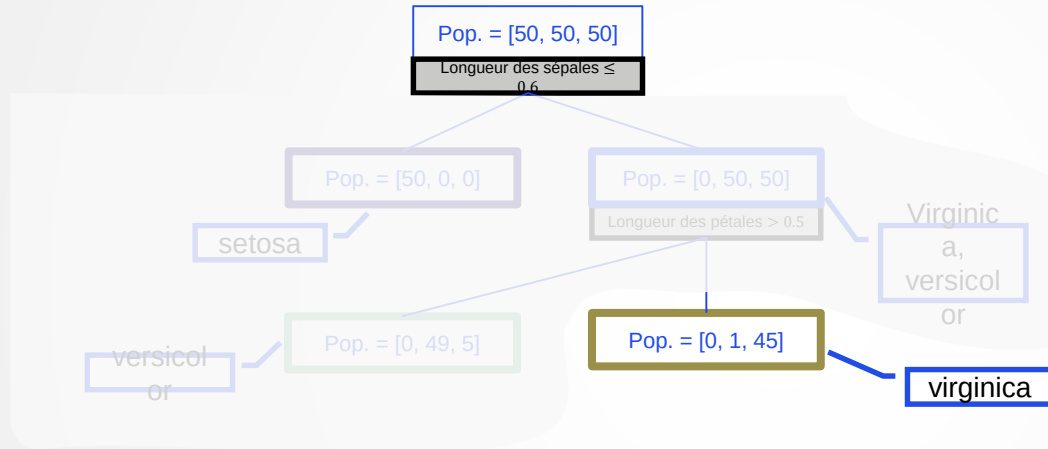
$$gini = 1 - \left(\frac{50}{50}\right)^2 - \left(\frac{0}{50}\right)^2 - \left(\frac{0}{50}\right)^2 = 0$$

Arbre de décision : pureté de l'arbre, GINI



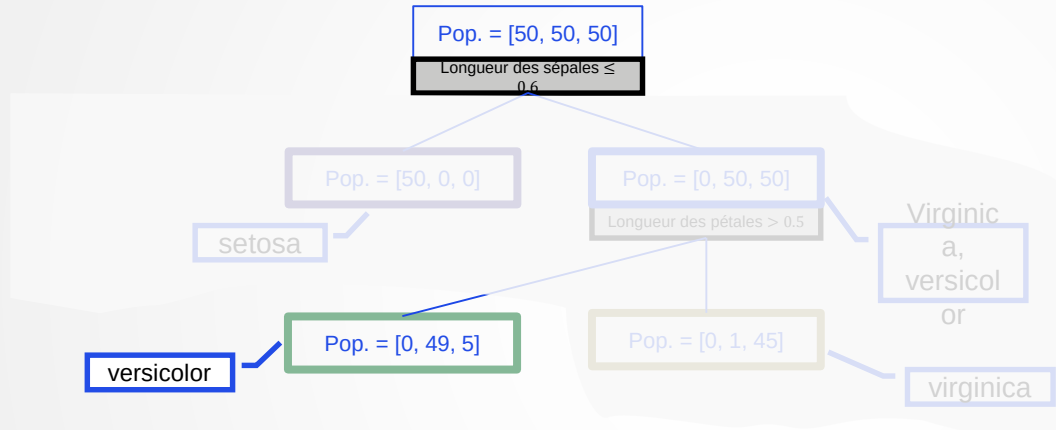
$$gini = 1 - \left(\frac{0}{50}\right)^2 - \left(\frac{50}{100}\right)^2 - \left(\frac{50}{100}\right)^2 = 0.5$$

Arbre de décision : pureté de l'arbre, GINI



$$gini = 1 - \left(\frac{0}{46}\right)^2 - \left(\frac{1}{46}\right)^2 - \left(\frac{45}{46}\right)^2 = 0.043$$

Arbre de décision : pureté de l'arbre, GINI



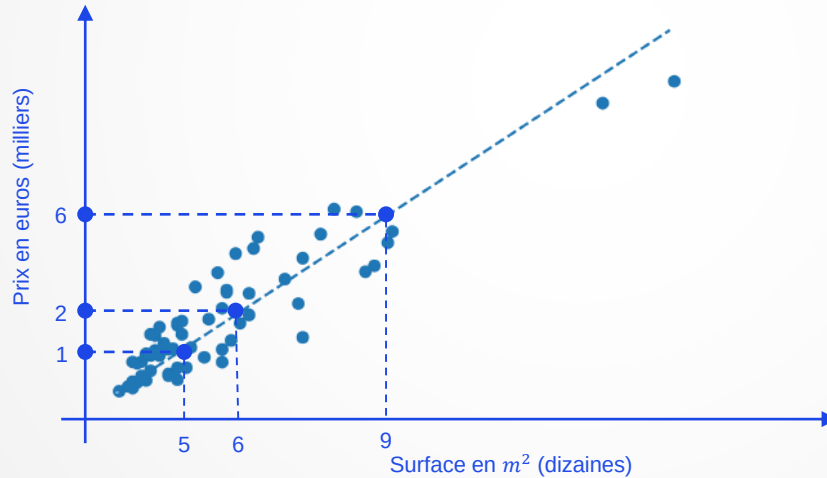
$$gini = 1 - \left(\frac{0}{54}\right)^2 - \left(\frac{49}{54}\right)^2 - \left(\frac{5}{54}\right)^2 = 0.168$$

Arbre de décision : régression

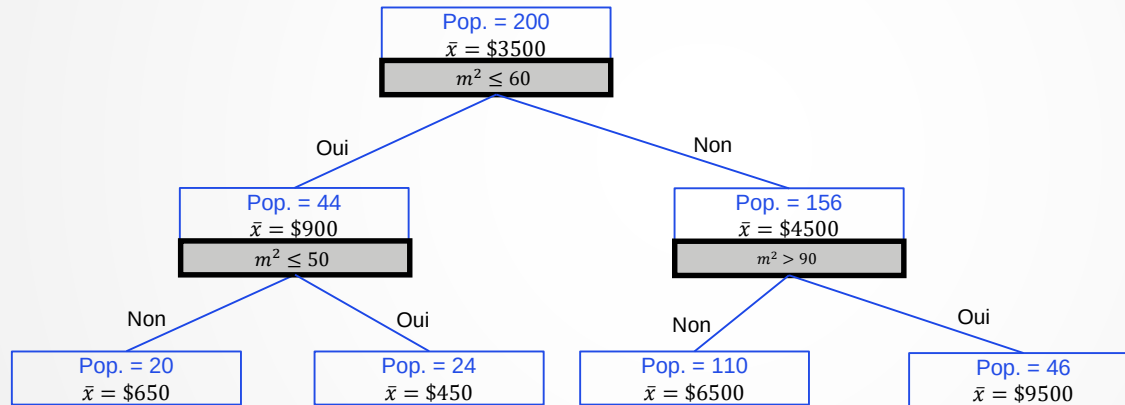
- On utilise les arbres de régression pour expliquer et/ou prédire les valeurs prise par une variable dépendante quantitative, en fonction de variables explicatives quantitatives et/ou qualitatives.
- Ils peuvent par exemple aider à prédire le niveau des prix de loyers en fonction de certaines caractéristiques d'un logement.

Arbre de décision : régression

Boston House



Arbre de décision : régression Boston House



x : valeurs réelles

$\hat{x}$: prédiction

$\bar{x}$: moyenne

Arbre de décision : pureté de l'arbre, MSE

- Ici, pas de critère Gini.
- On calcule l'erreur moyenne de prédiction.
- $MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$
- Cette métrique nous indique dans quelle mesure les prédictions s'écartent des valeurs réelles.
- Ce critère est à minimiser.



Skills4All
digital learning certifiant

Random Forest

Random Forest



Intuition



Représentation graphique



Fine tuning



Avantages et inconvénients



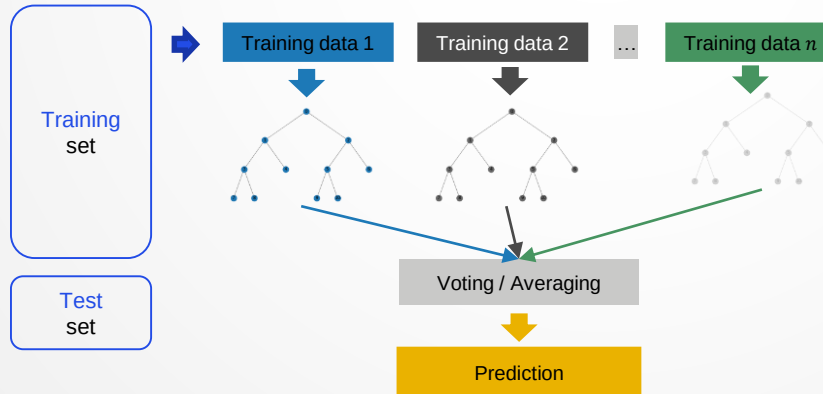
Code Python

Random Forest : intuition

- Random Forest est un algorithme de Machine Learning utilisée pour la classification, la régression et d'autres tâches.
- Il fonctionne en construisant une multitude d'arbres de décision au moment de l'entraînement et prédit la classe d'appartenance d'un individu (classification) ou la prédiction moyenne (régression) des arbres individuels.

Random Forest : intuition

- L'algorithme de forêt aléatoire combine plusieurs algorithmes d'arbres de décision pour améliorer la stabilité et la précision de la prise de décision. C'est un algorithme d'apprentissage automatique populaire car il peut gérer une grande quantité de données et il est relativement précis.



Random Forest : intuition

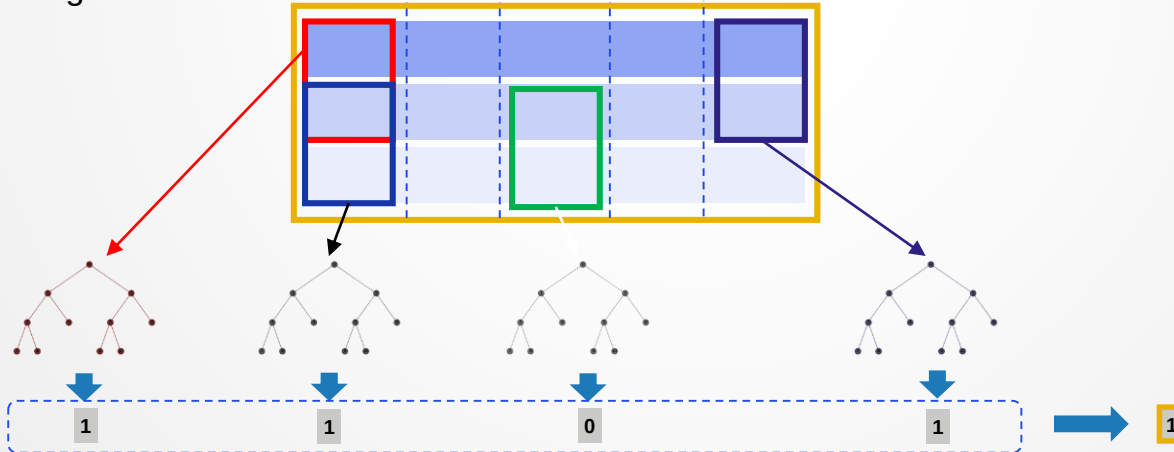
- **Modèle ensembliste (ou ensemble learning)** : technique qui repose sur la combinaison de multiples algorithmes de Machine Learning afin d'accroître les performances du modèle d'apprentissage, et parvenir à un niveau de précision supérieur à celui qui serait réalisé si on utilisait un de ces algorithmes pris séparément.

Random Forest : fonctionnement

1. L'algorithme sélectionne des échantillons aléatoires de données à partir du dataset d'origine (bagging) pour créer de nombreux ensembles d'apprentissage différents.
2. Pour chaque échantillon, un modèle d'arbre de décision est construit.
3. Pour chaque arbre, un sous-ensemble aléatoire des features est choisi pour la construction et le fractionnement de l'arbre.
4. L'algorithme combine les arbres de décision individuels dans un modèle final en prenant un vote majoritaire pour la classification ou en faisant la moyenne pour la régression.

Random Forest : bagging

- Aussi appelé bootstrap aggregating, consiste à sous-échantillonner les données, en créant un data set pour chaque modèle.
- Pour déterminer le résultat final, on procède par un vote des résultats de chaque modèle pour une classification, ou par une moyenne pour une régression.



Random Forest : fine tuning

Certaines étapes supplémentaires qui peuvent être prises en compte pour affiner le modèle :

1. Régler des paramètres de l'algorithme de l'arbre de décision, tels que la profondeur maximale de l'arbre et le nombre minimal d'échantillons requis pour diviser un nœud.
2. Élaguer les arbres pour enlever les branches qui n'apportent pas d'amélioration significative de la précision.
3. Utiliser la validation croisée pour évaluer les performances du modèle et identifier la meilleure combinaison de paramètres.
4. Utiliser les « **features importance** » pour sélectionner les variables pertinentes, permettant ainsi de réduire la dimension des données et avoir de meilleurs résultats.

Random Forest : avantages

- Il s'agit d'un algorithme polyvalent qui peut être utilisé à la fois pour des tâches de classification et de régression.
- Il peut gérer des données de grande dimension et un grand nombre de features.
- Il est moins sujet à l'overfitting par rapport à un arbre de décision unique.
- Il fournit une estimation de l'importance des features dans le modèle.
- Il peut être utilisé pour les variables catégorielles et continues.
- Il peut gérer les données manquantes.

Random Forest : inconvénients

- Il peut être difficile d'interpréter le modèle final car il s'agit d'une combinaison de nombreux arbres de décision.
- Il est moins performant avec des données de très petite taille.
- Il peut être coûteux en temps de calcul de former un grand nombre d'arbres, en particulier si les données sont de grande dimension.

Random Forest : cas d'applications

Classification :

- Les forêts aléatoires sont largement utilisées pour les tâches de classification dans divers domaines tels que la classification d'images, la reconnaissance vocale, la bio-informatique et le diagnostic médical.
- Par exemple, des forêts aléatoires peuvent être utilisées pour classer des images de chiffres manuscrits, pour identifier l'espèce d'une fleur d'iris en fonction de ses caractéristiques physiques et pour diagnostiquer un patient atteint d'une certaine maladie en fonction des symptômes et des résultats des tests.

Random Forest : cas d'applications

Régression :

- Random Forest peut être utilisé pour des tâches de régression telles que la prévision du cours des actions, les prévisions météorologiques et la prévision de la consommation d'énergie.

Features selection :

- Pour identifier les features les plus importantes d'un jeu de données. La mesure de l'importance des variables peut être utilisée pour sélectionner les variables les plus pertinentes pour une tâche particulière ou pour réduire la dimensionnalité des données.

Random Forest : cas d'applications

Détection d'anomalies :

Les forêts aléatoires peuvent être utilisées pour détecter des anomalies ou des valeurs aberrantes dans des ensembles de données en identifiant des modèles qui s'écartent de la norme.

Credit scoring :

Les forêts aléatoires peuvent être utilisées pour prédire le risque de défaut de remboursement d'un crédit, en identifiant des patterns dans les données qui indiquent un risque de défaut élevé.

Random Forest : cas d'applications

Détection des fraudes :

Les forêts aléatoires peuvent être utilisées pour détecter les transactions frauduleuses en identifiant les modèles qui s'écartent du comportement normal.

Systèmes de recommandation :

Les forêts aléatoires peuvent être utilisées pour créer des systèmes de recommandation en identifiant des patterns dans les données qui indiquent les préférences d'un utilisateur et en recommandant des éléments susceptibles de l'intéresser.

Random Forest : hyper paramètre Grid Search

- Le nombre d'hyper paramètres à optimiser varie d'un algorithme à l'autre.
- Par exemple pour Random Forest, on peut choisir le nombre d'arbres à créer et le nombre de variables à utiliser à chaque division d'un nœud.
- Si on le paramètre à la main, cela peut vite s'avérer très coûteux en temps (et pas forcément très intéressant).
- Grid Search va créer un modèle pour chaque combinaison de paramètres.
- Dans notre exemple nous aurons 20 modèles à construire. Vous comprenez donc rapidement qu'il ne faut pas abuser du Grid Search parce qu'il augmente considérablement les temps de calcul.

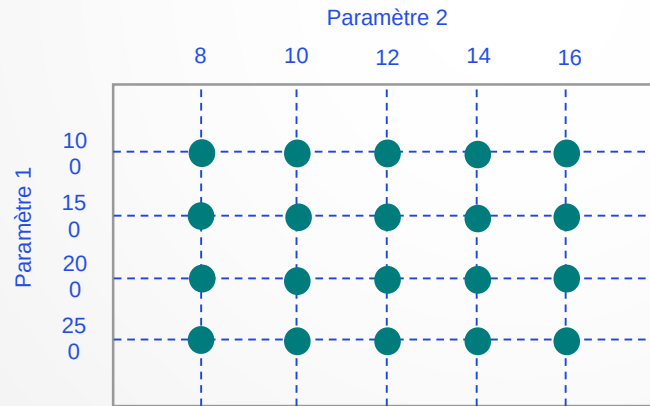
nombre d'arbres : {100, 150, 200, 250}

nombre de variables sélectionnées : {8, 10, 12, 14, 16}

Random Forest : hyper paramètre Grid Search

nombre d'arbres : {100, 150, 200, 250}

nombre de variables sélectionnées : {8, 10, 12, 14, 16}



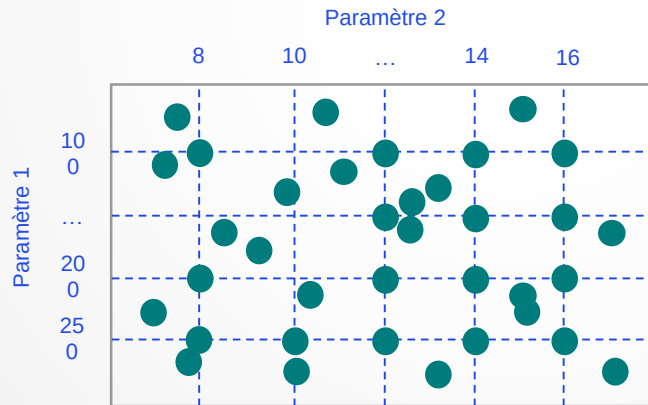
Random Forest : hyper paramètre Random Search

- Une des options pour pallier la limitation de Grid Search, est d'utiliser la méthode Random Search.
- On garde le « système de grille » mais on change notre façon de sélectionner les combinaisons de paramètres à chaque étape.
- Cette fois, plutôt que de fixer un ensemble de valeurs prises par un hyper paramètre donné, on va fixer un intervalle de valeurs et on va associer à chaque hyper paramètre une distribution de probabilités.
- Ensuite, à chaque itération, on entraîne le modèle et on garde les résultats de performances en mémoire.

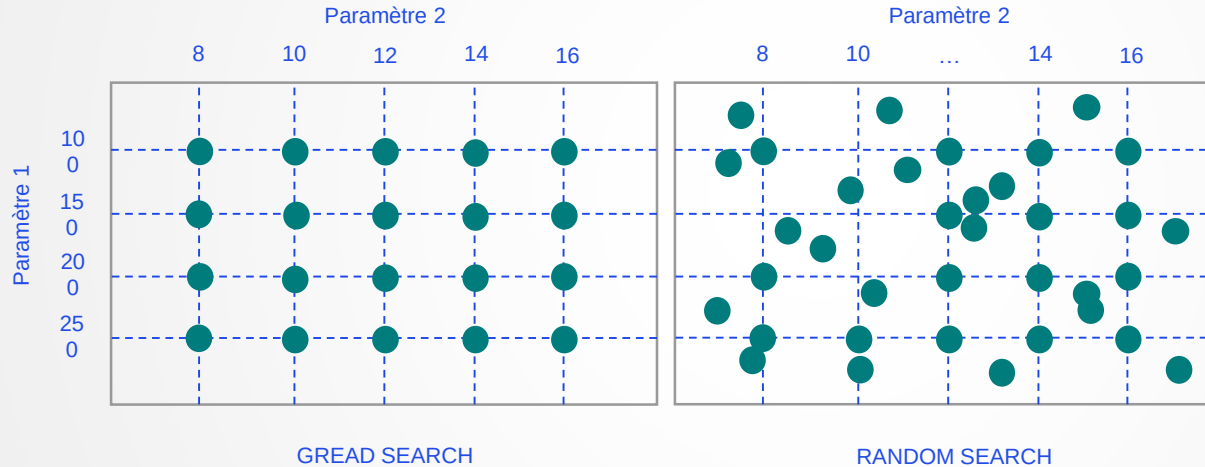
Random Forest : hyper paramètre Random Search

nombre d'arbres aléatoires : {...}

nombre de variables sélectionnées de façon aléatoire : {...}



Random Forest : Greed Search vs Random Search



Random Forest : validation croisée

- La validation croisée permet d'évaluer les performances des modèles d'apprentissage automatique.
- Elle permet également d'éviter le phénomène de sur apprentissage (over fitting).
- La validation croisée K-Fold est une méthode qui divise vos données en k plis de taille égale.
- Comprenez un « pli » comme étant un échantillon de vos données initiales.
- Chaque pli est utilisé comme ensemble de test tout en utilisant les plis $k - 1$ restants comme ensemble d'entraînement.
- Chaque modèle candidat est entraîné et évalué k fois et on obtient une performance moyenne sur tous les plis.
- Choisissez le modèle avec les meilleures performances moyennes.

Random Forest : validation croisée

Dataset initial avant split



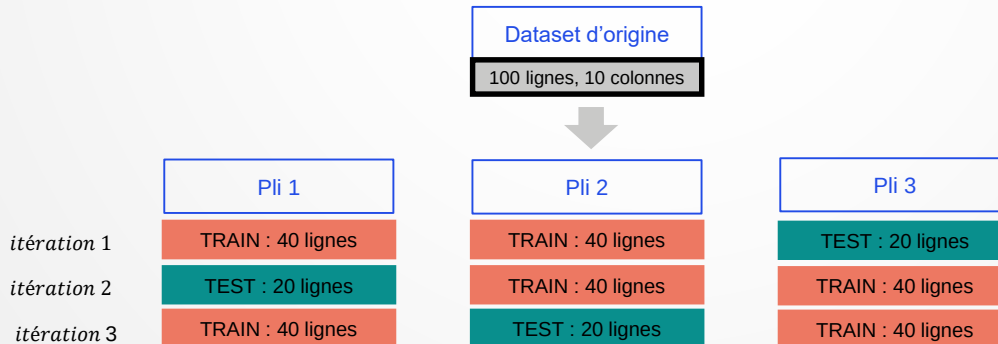
Dataset initial après split
80-20



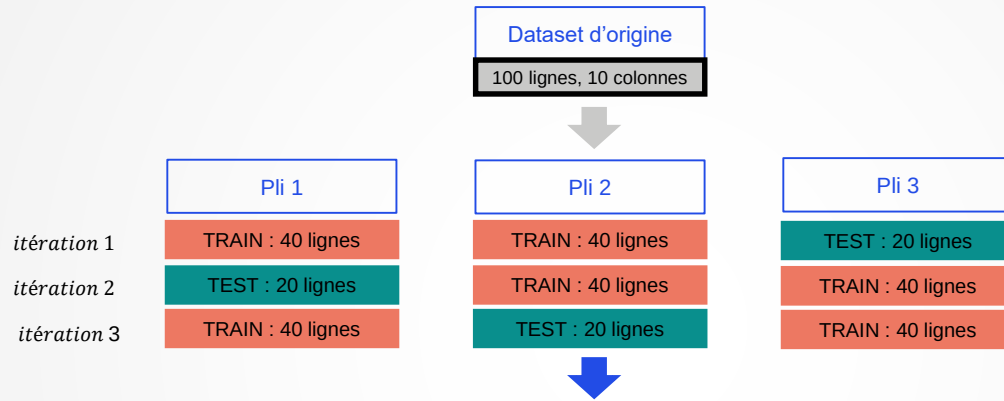
- Data set initial
- Objectif : le séparer en 80-20 (train, test)
- Le problème c'est qu'avec ce procédé, on évalue le modèle qu'une seule fois.

Random Forest : validation croisée

- La validation croisée permet d'évaluer le modèle plusieurs fois.
 - Il présente donc au modèle la totalité de l'échantillon.
1. A partir du dataset initial on définit 3 plis (folds). Ici $k=3$.
 2. Chaque pli contient un ensemble d'entraînement et un ensemble de test.
 3. 3 plis impliquent donc 3 itérations et donc 3 modèles évalués.



Random Forest : validation croisée



itération 1

Sepal length	Sepal width	Petal length	Petal width
5.2	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.3
5.4	3.9	1.7	0.4
4.6	3.4	1.4	0.3
7.0	3.2	4.7	1.4
6.4	3.2	4.5	1.5

itération 2

Sepal length	Sepal width	Petal length	Petal width
5.2	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.3
5.4	3.9	1.7	0.4
4.6	3.4	1.4	0.3
7.0	3.2	4.7	1.4
6.4	3.2	4.5	1.5

itération 3

Sepal length	Sepal width	Petal length	Petal width
5.2	3.5	1.4	0.2
4.9	3.0	1.4	0.2
4.7	3.2	1.3	0.2
4.6	3.1	1.5	0.2
5.0	3.6	1.4	0.3
5.4	3.9	1.7	0.4
4.6	3.4	1.4	0.3
7.0	3.2	4.7	1.4
6.4	3.2	4.5	1.5

Random Forest : validation croisée

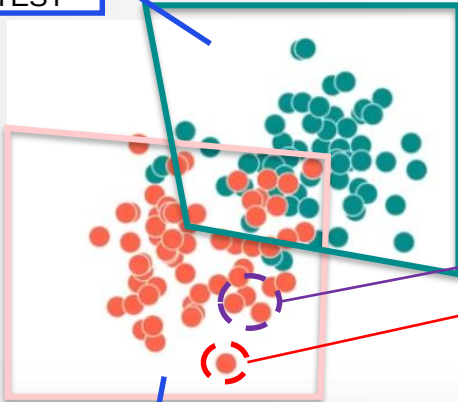
itération 1

TRAIN : 40 lignes

TRAIN : 40 lignes

TEST : 20 lignes

TEST



TRAIN

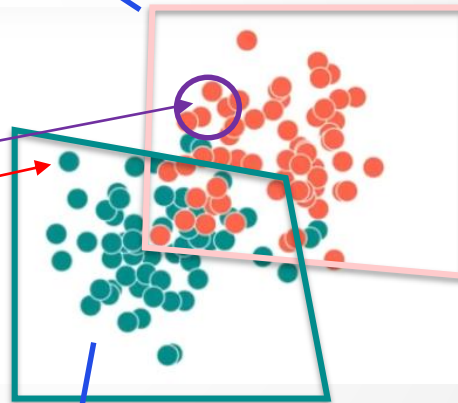
itération 2

TEST : 20 lignes

TRAIN : 40 lignes

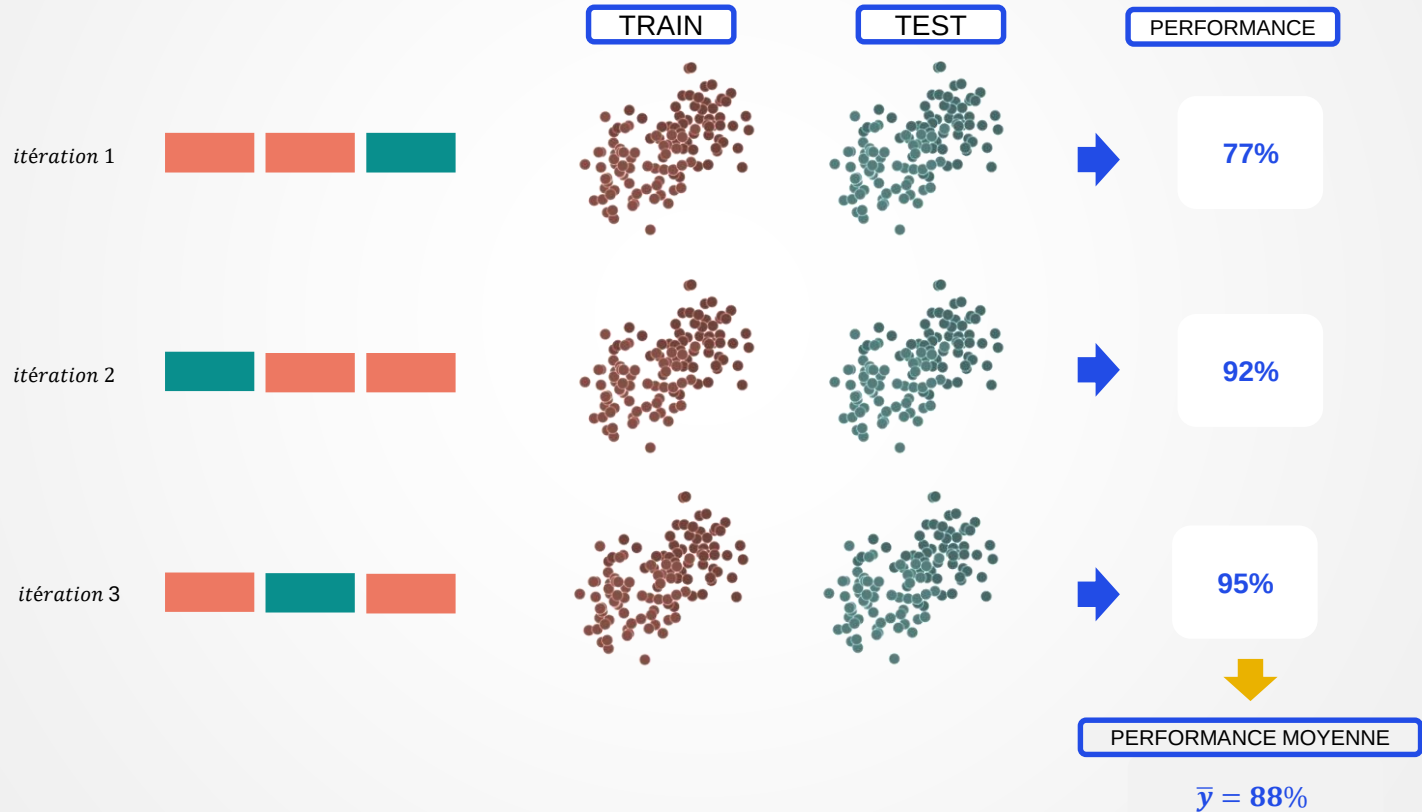
TRAIN : 40 lignes

TRAIN



TEST

Random Forest : validation croisée



XG Boost

XG Boost



Intuition



Historique



La descente du gradient



Avantages et inconvénients



Code Python

XG Boost : intuition

- Extreme Gradient Boosting est un algorithme d'apprentissage automatique basé sur le gradient boosting.
- XG Boost peut être utilisé pour résoudre des problèmes de régression et de classification.
- Avant d'aller plus loin, faisons un tour d'horizon sur les différentes évolutions des algorithmes basés sur les arbres, afin de mieux comprendre XG Boost et les raisons de son succès.

XG Boost : historique

1. Arbres de décision :

- Au départ ils avaient le vent en poupe. On pouvait ainsi représenter graphiquement plusieurs solutions possibles basées sur un certain nombre de conditions.
- Par exemple, un responsable de recrutement choisira ses meilleurs candidats en fonction d'un certain nombre de critères (niveau d'éducation, années d'expérience, performances pendant l'entretien...).

XG Boost : historique

2. Bagging :

- Ensuite il y a eu le Bagging ou Bootstrap aggregating.
- C'est une technique d'apprentissage ensembliste qui permet d'améliorer la classification des arbres de décision (stabilité et précision).
- Le résultat final est trouvé par un mécanisme de vote de la classe majoritaire, à l'issue d'une confrontation de plusieurs modèles d'arbres de décision.
- Il réduit la variance et permet donc d'éviter le surapprentissage.
- Son fonctionnement fait penser à un mécanisme de vote aux urnes, par analogie au processus de vote dans les pays démocratiques.

XG Boost : historique

3. Random Forest :

- Il s'agit d'un algorithme basé sur le bagging avec une différence clé dans laquelle seul un sous-ensemble de features est sélectionné au hasard.
- Par exemple, un entretien technique pour tester les compétences en programmation et un entretien comportemental pour évaluer les compétences non techniques des candidats.

XG Boost : historique

4. Boosting :

- Contrairement aux arbres de décisions qui construisent des modèles parallèles, les modèles de Boosting sont construits de façon séquentielle.
- L'avantage des modèles construits de façon séquentielle est qu'ils permettent de minimiser les erreurs, car chaque modèle tente de compenser les faiblesses de son prédécesseur.
- Cela fait penser au postulant qui questionne ses concurrents à la sortie de l'entretien d'embauche, afin de compenser son manque d'assurance et d'être mieux préparé aux questions. Cela a pour effet de booster son efficacité lors du processus de recrutement.

XG Boost : historique

5. Gradient boosting :

- Le Gradient boosting utilise l'algorithme de descente du gradient pour minimiser les erreurs dans les modèles séquentiels.
- La descente de gradient est un algorithme d'optimisation qui permet de trouver le minimum de n'importe quelle fonction convexe en convergeant progressivement vers celui-ci.
- En Machine Learning, on l'utilise dans les problèmes d'apprentissage supervisé pour minimiser la fonction coût (convexe).
- C'est grâce à cet algorithme que la machine apprend, c'est-à-dire trouve le meilleur modèle.
- Par analogie, les cabinets de conseil tirent parti en utilisant des entretiens de cas pour éliminer les candidats les moins qualifiés.

XG Boost : historique

6. XG Boost :

- Enfin XG Boost est arrivé !
- Il s'agit d'un algorithme de Gradient Boosting super optimisé qui fonctionne de façon séquentielle.
- Il est utilisé pour résoudre des problèmes de classification ou de régression.
- Il élague ses arbres, prend en compte les données manquantes et utilise la régularisation pour éviter des biais ou le surapprentissage.

XG Boost : la descente de gradient

- La descente de gradient est un algorithme d'optimisation utilisé pour entraîner des modèles d'apprentissage automatique et des réseaux neuronaux.
- Les données d'entraînement aident ces modèles à apprendre au fil du temps, et la fonction de coût dans la descente de gradient agit spécifiquement comme un baromètre, évaluant sa précision à chaque itération des mises à jour des paramètres.
- Jusqu'à ce que la fonction soit proche de zéro ou égale à zéro, le modèle continue à ajuster ses paramètres pour obtenir l'erreur la plus faible possible.

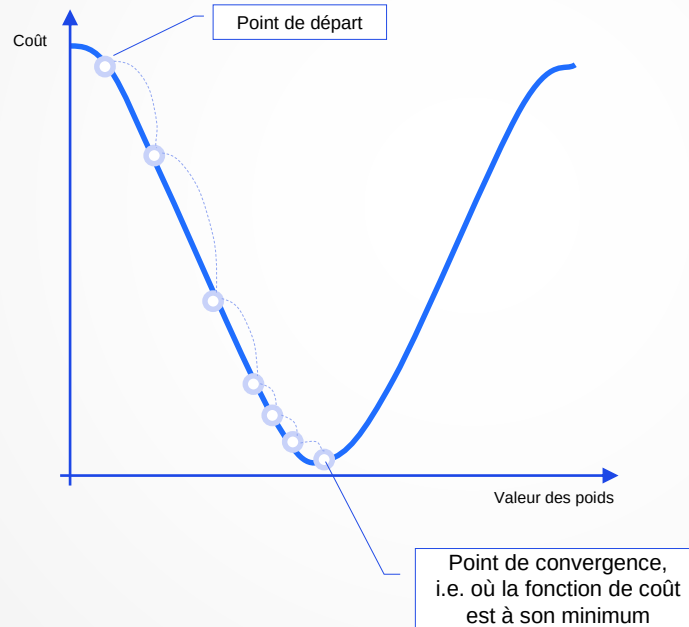
XG Boost : la descente de gradient

- En régression linéaire, souvenez-vous de la formule suivante pour la pente d'une ligne, soit $y = mx + b$, où m représente la pente et b l'ordonnée à l'origine sur l'axe des y .
- Souvenez-vous aussi du nuage de points et de la ligne qui trouve le meilleur ajustement, ce qui nécessitait de calculer l'erreur entre le résultat réel et le résultat prédit à l'aide de la formule de l'erreur quadratique moyenne.
- L'algorithme de descente de gradient se comporte de manière similaire, mais il est basé sur une fonction convexe.

XG Boost : la descente de gradient

- Le point de départ est juste un point arbitraire qui nous permet d'évaluer la performance.
- À partir de ce point de départ, nous trouverons la dérivée (ou la pente) et, à partir de là, nous pourrons utiliser une ligne tangente pour observer la raideur de la pente.
- La pente informera les mises à jour des paramètres, c'est-à-dire les pondérations et le biais.
- La pente au point de départ sera plus forte, mais au fur et à mesure que de nouveaux paramètres sont générés, la pente devrait progressivement diminuer jusqu'à atteindre le point le plus bas de la courbe, appelé point de convergence.

XG Boost : la descente de gradient



XG Boost : la descente de gradient

- Comme pour la recherche de la ligne de meilleur ajustement dans la régression linéaire, l'objectif de la descente de gradient est de minimiser la fonction de coût, ou l'erreur entre la prédiction et le y réel.
- Pour ce faire, elle a besoin de deux points de données : **une direction** et un **taux d'apprentissage**.
- Ces facteurs déterminent les calculs de dérivée partielle des itérations futures, ce qui lui permet d'arriver progressivement au minimum local ou global (c'est-à-dire au point de convergence).

XG Boost : le taux d'apprentissage (learning rate)

- Également appelé taille d'étapes ou alpha c'est la taille des étapes choisies pour atteindre le minimum.
- Il s'agit généralement d'une petite valeur, qui est évaluée et mise à jour en fonction du comportement de la fonction de coût.
- Un taux d'apprentissage élevé se traduit par des étapes plus grandes, qui risquent de dépasser le minimum.
- À l'inverse, un taux d'apprentissage faible a des étapes de petite taille. S'il présente l'avantage d'une plus grande précision, le nombre d'itérations compromet l'efficacité globale, car il faut plus de temps et de calculs pour atteindre le minimum.
- Il n'existe pas d'astuces pour trouver le taux d'apprentissage optimal. On tâtonne en modifiant les hyperparamètres.

XG Boost : la fonction de coût

- La fonction de coût mesure la différence, ou l'erreur, entre le y réel et le y prédit à sa position actuelle.
- Elle améliore l'efficacité du modèle d'apprentissage automatique en fournissant un retour d'information au modèle afin qu'il puisse ajuster les paramètres pour minimiser l'erreur et trouver le minimum local ou global.
- Il itère continuellement, en se déplaçant dans la direction de la descente la plus raide (ou du gradient négatif) jusqu'à ce que la fonction de coût soit proche de zéro ou égale à zéro.
- Le modèle cesse d'apprendre lorsque la fonction de coût est proche de zéro ou égale à zéro.

XG Boost : avantages et inconvénients

- **Avantages :**
 - Traite aussi bien la régression que la classification
 - Gère les valeurs manquantes et la régularisation
 - Rapide et précis du fait de son fonctionnement
 - Facilement interprétable et industrialisable
- **Inconvénients :**
 - Peut se montrer peu efficace dans un contexte de Big Data
 - Peu efficace lorsque le nombre de colonnes est supérieur au nombre de lignes



Skills4All
digital learning certifiant

K Nearest Neighbors

K Nearest Neighbors



Intuition



Fonctionnement



Représentation graphique



Avantages et inconvénients



Code Python

K Nearest Neighbors : intuition

- k-nearest neighbors ou k-plus proches voisins (KNN) est un type d'algorithme d'apprentissage supervisé non paramétrique, dans lequel un objet est classé sur la base du vote majoritaire de ses k plus proches voisins parmi l'ensemble d'apprentissage, où k est un entier positif .
- En cas de problème de régression, il peut renvoyer la moyenne des k voisins les plus proches ou la moyenne pondérée avec le poids comme distance.

K Nearest Neighbors : intuition

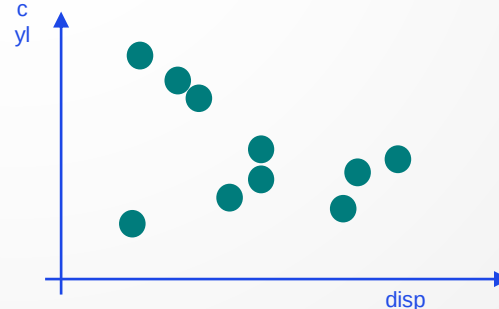
- Il est important de noter que l'algorithme KNN est un algorithme paresseux, ce qui signifie qu'il n'apprend pas un modèle pour faire des prédictions.
- L'idée clé derrière KNN est que des points similaires ont tendance à avoir des étiquettes similaires. L'algorithme KNN est souvent utilisé dans les problèmes de classification et de régression.
- KNN pour la régression calcule la moyenne de la variable réponse ($\bar{y}$) pour les k plus proches voisins. Le principe est le même pour les problèmes de classification, mais l'algorithme est légèrement modifié en raison du type de valeurs catégorielles.

K Nearest Neighbors : fonctionnement

KNN pour la régression

- Nous disposons de caractéristiques techniques d'un liste de véhicules automobiles.
- Nous souhaitons trouver les k plus proches voisins, i.e. les voitures qui ont des caractéristiques très proches (similaires).
- Il faudra calculer la distance euclidienne entre toutes les caractéristiques.

Marque	Mpg	Cyl	Disp
Mazda RX4	21	6	160
Mazda RX4 Wag	21	6	160
Datsun 710	22.8	4	108
Hornet 4 Drive	21.4	6	258
Hornet Sportabout	18.7	8	360
Valiant	18.1	6	225



K Nearest Neighbors : fonctionnement

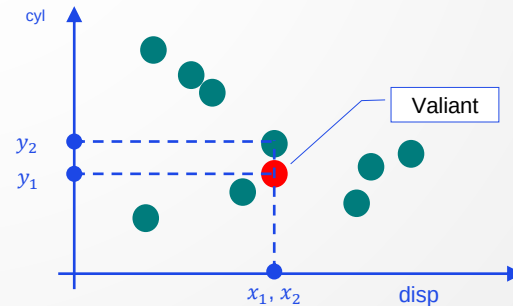
KNN pour la régression

- La distance euclidienne est donnée par la formule :

$$d = \sqrt{\text{colonne}_1^2 + \text{colonne}_2^2 + \text{colonne}_3^2 \dots + \text{colonne}_p^2}, \text{ soit } d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- On choisit par exemple la voiture Valiant (il faut bien commencer quelque part).
- On retire y qui ne doit surtout pas entrer en compte pendant le calcul de la distance euclidienne.
- Il nous restera donc comme features : x_1 et x_2

Marque	Mpg	Cyl	Disp
Valiant	18.1	6	225
Mazda RX4 Wag	21	6	160
Datsun 710	22.8	4	108
Hornet 4 Drive	21.4	6	258
Hornet Sportabout	18.7	8	360
Mazda RX4	21	6	160



K Nearest Neighbors : fonctionnement

KNN pour la régression

- La distance entre Valiant et lui-même est nulle :

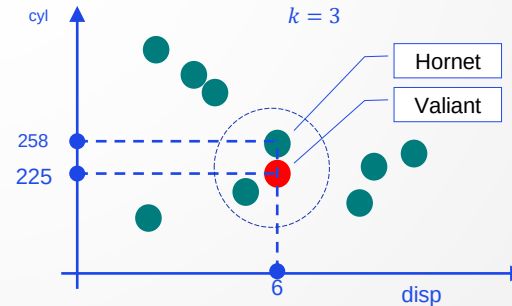
$$d = \sqrt{(6 - 6)^2 + (225 - 225)^2} = 0.00$$

- La distance entre valiant et Hornet 4 Drive :

$$d = \sqrt{(6 - 6)^2 + (258 - 225)^2} = 33.00$$

- Après calcul de la distance euclidienne, triez le tableau dans l'ordre croissant de d .

Marque	Mpg	Cyl	Disp	d
Valiant	18.1	6	225	0.00
Hornet 4 Drive	21.4	6	258	33.00
Mazda RX4	21	6	160	65.00
Mazda RX4 Wag	21	6	160	65.00
Datsun 710	22.8	4	108	117.01
Hornet Sportabout	18.7	8	360	135.01



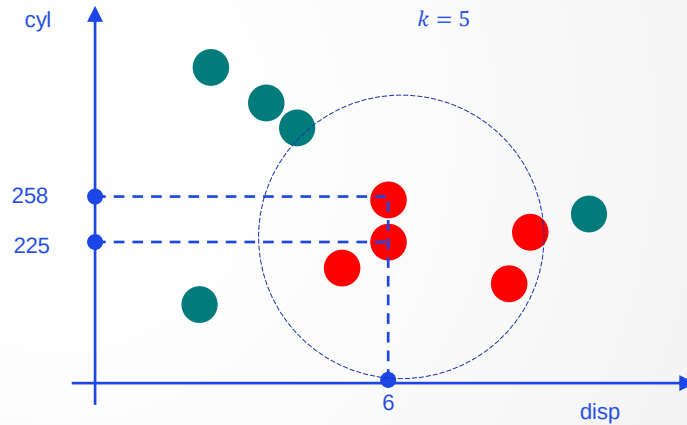
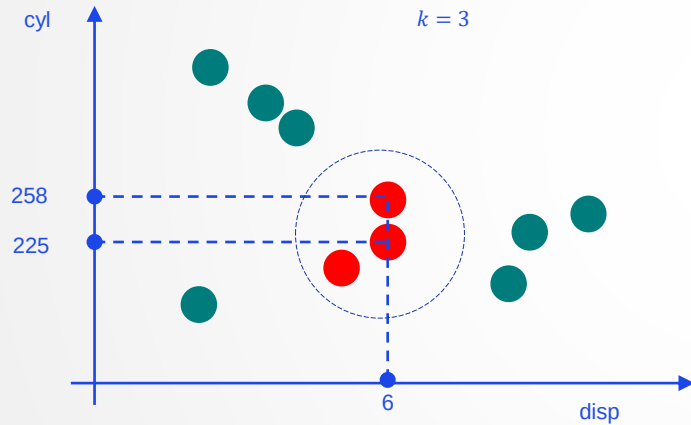
K Nearest Neighbors : fonctionnement

KNN pour la régression

- Déterminer la valeur k , qui est le nombre de voisins les plus proches à considérer.
- Trouver les k voisins les plus proches du nouveau point de données dans l'espace des caractéristiques.
- Déterminer la classe majoritaire parmi les k plus proches voisins.
- Attribuer le nouveau point de données à la classe majoritaire.

K Nearest Neighbors : fonctionnement

KNN pour la régression



K Nearest Neighbors : fonctionnement

KNN pour la régression

Il existe différentes méthodes pour déterminer la distance entre deux points dans l'espace :

- **Distance euclidienne** : Il s'agit de la mesure de distance la plus couramment utilisée et elle est définie comme la racine carrée de la somme des carrés des différences entre les coordonnées de deux points. En d'autres termes, c'est la distance en ligne droite entre deux points de l'espace euclidien.

$$d(x, y) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

K Nearest Neighbors : fonctionnement

KNN pour la régression

- **Distance de Manhattan** : Cette mesure de distance est également connue sous le nom de distance de « taxi », car il s'agit de la distance qu'un taxi devrait parcourir pour se rendre d'un point à l'autre s'il ne peut se déplacer que le long des lignes de quadrillage d'une ville. Elle est définie comme la somme des différences absolues entre les coordonnées de deux points.

$$d(x, y) = |x_1 - y_1| + |x_2 - y_2| + \dots + |x_n - y_n|$$

K Nearest Neighbors : fonctionnement

KNN pour la régression

- **Similitude du cosinus** : Cette métrique de distance est utilisée pour mesurer la similarité entre deux vecteurs non nuls d'un espace de produit intérieur qui mesure le cosinus de l'angle entre eux.

$$\cos(x, y) = \frac{(x \cdot y)}{(\|x\| \cdot \|y\|)}$$

- **Distance de Minkowsky** : Cette métrique de distance est une généralisation de la distance euclidienne et de Manhattan.

$$d(x, y) = ((|x_1 - y_1|^p) + (|x_2 - y_2|^p) + \dots + (|x_n - y_n|^p))^{1/p}$$

K Nearest Neighbors : fonctionnement

KNN pour la classification

- La classification consiste à trouver la classe d'appartenance d'un individu à l'aide de ses caractéristiques.
- Par exemple, on peut classer un courrier électronique comme « spams » ou « e-mail », en fonction de son contenu (objet, expéditeur, mots, pièces jointes...)
- On peut aussi classer un patient comme « sain » ou « malade » en fonction de son âge, son genre, ses analyses sanguines et urinaires, son poids, sa tension artérielle...
- KNN pour la classification utilise la distance de Minkowsky, qui est une généralisation de la distance euclidienne et de Manhattan que vous avons vu avec KNN pour la régression.

K Nearest Neighbors : fonctionnement

KNN pour la classification

- **Distance de Minkowsky** : Cette métrique de distance est une généralisation de la distance euclidienne et de Manhattan.

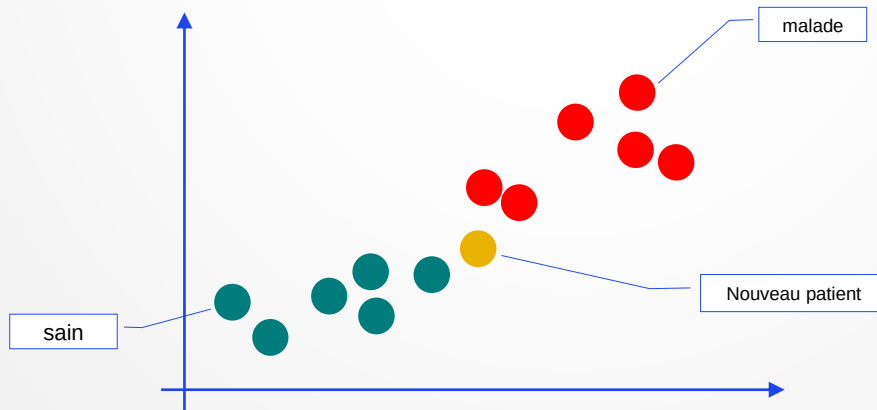
$$d(x, y) = ((|x_1 - y_1|^p) + (|x_2 - y_2|^p) + \dots + (|x_n - y_n|^p))^{1/p}$$

- Lorsque $p = 1$, on obtient la distance de Manhattan.
- Lorsque $p = 2$, on obtient la distance euclidienne.
- Lorsque $p = \infty$, on obtient la distance de Chebychev.
- Pour obtenir la classe d'appartenance d'un point, nous étudions les groupes de classes environnants.
- Si ce point est plus proche d'un groupe qu'un autre, alors ce point appartiendra au groupe considéré.

K Nearest Neighbors : fonctionnement

KNN pour la classification

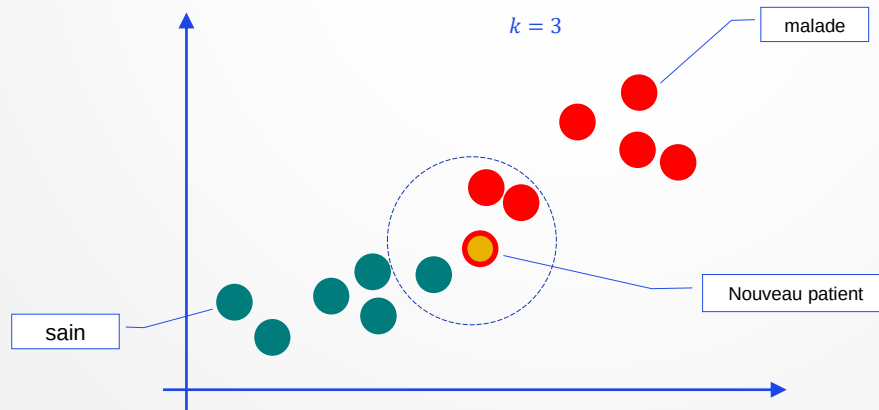
- Supposons que nous souhaitons classer des patients dans deux catégories : « sain » ou « malade ».
- L'idée est de repérer rapidement les patients identifiés comme malades, afin de leur apporter un traitement avant l'aggravation de la maladie.



K Nearest Neighbors : fonctionnement

KNN pour la classification

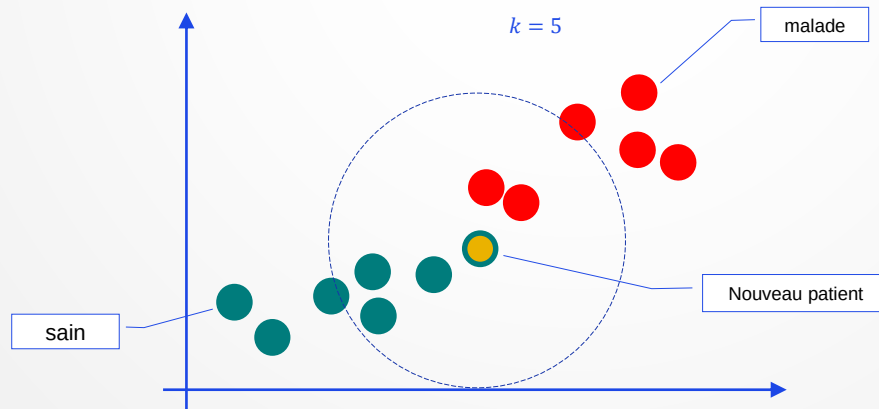
- Si on choisit $k = 3$ avec des poids constants pour tous les points, on obtient deux patients malades et un patient en bonne santé.
- Le nouveau patient appartiendra à la classe rouge, étant donné que les rouges sont majoritaires.



K Nearest Neighbors : fonctionnement

KNN pour la classification

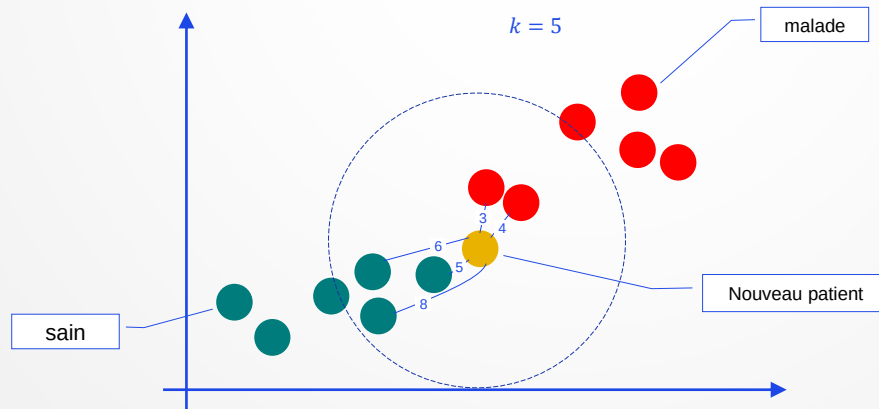
- Si on choisit $k = 5$ avec des poids constants pour tous les points, on obtient deux patients malades et trois patients sains.
- Le nouveau patient appartiendra à la classe verte, étant donné que les patients sains y sont majoritaires.



K Nearest Neighbors : fonctionnement

KNN pour la classification

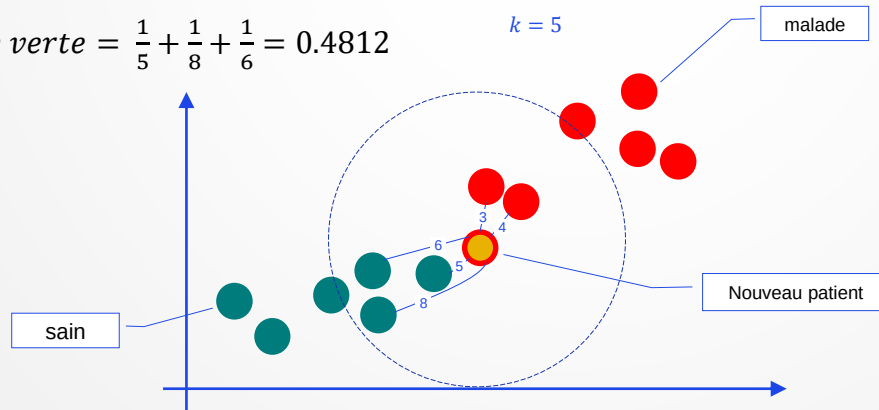
- En revanche, si on choisit $k = 5$ et qu'on utilise la distance euclidienne pour calculer les **poids** de chaque points, alors on aura trois patients sains et deux malades dans le voisinage.
- Les distances euclidiennes sont illustrées comme ci-dessous :



K Nearest Neighbors : fonctionnement

KNN pour la classification

- Pour calculer les **poids** à partir des distances euclidiennes, on prend l'inverse des distances afin que les points les plus proches aient des poids plus élevés.
- Pour chaque classe, on prend la somme des poids calculés, et la classe avec un poids total plus élevé devient la classe prédite.
- $classe\ rouge = \frac{1}{3} + \frac{1}{4} = 0.5833$
- $classe\ verte = \frac{1}{5} + \frac{1}{8} + \frac{1}{6} = 0.4812$



K Nearest Neighbors : avantages et inconvénients

- Avantages :
 - Principes mathématiques simples
 - Facile à mettre en œuvre
 - Calcul efficace sur machine
- Désavantages :
 - Le bruit ou l'asymétrie des données peuvent conduire à une mauvaise classification.
 - Pas toujours efficace dans les grands ensembles de données d'entraînement.



Skills4All
digital learning certifiant

Support Vector Machine

Support Vector Machine : intuition



Intuition



Fonctionnement



Représentation graphique



Avantages et inconvénients



Code Python

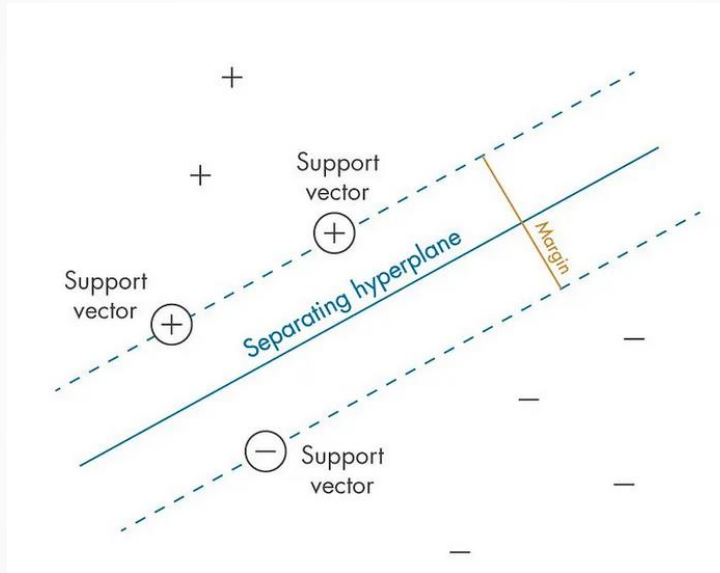
Support Vector Machine : intuition

- Les machines à vecteurs de support sont un ensemble de méthodes d'apprentissage supervisé utilisées pour la classification, la régression et la détection des valeurs aberrantes.
- L'objectif du SVM est de trouver l'hyperplan de séparation optimal qui maximise la séparabilité des données d'apprentissage :
 - L'hyperplan doit être aussi éloigné que possible des points de données.
 - L'hyperplan à marge étroite conduit à une mauvaise classification.
 - Notre objectif est de maximiser la marge.

Support Vector Machine : fonctionnement

- Un classificateur SVM linéaire simple fonctionne en créant une ligne droite entre deux classes.
- Les SVM sont différents des autres algorithmes de classification en raison de la façon dont ils choisissent la limite de décision qui maximise la distance par rapport aux points de données les plus proches de toutes les classes.
- La frontière de décision créée par les SVM est appelée classificateur de marge maximale ou hyperplan de marge maximale (ou encore **séparateur à vaste marge**).

Support Vector Machine : hyperplan optimal



- Marge :
 - C'est la distance entre l'hyperplan et les points de données les plus proches.
 - Une marge plus grande réduit les risques d'erreur de classification.

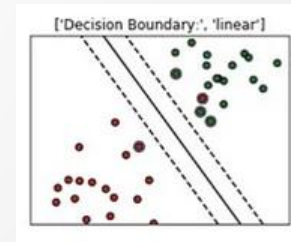
Support Vector Machine : fonction noyau (kernel)

Linéaire

- Recommandé pour la classification de texte car la plupart de ces types de problèmes de classification sont linéairement séparables.
- Le noyau linéaire fonctionne très bien lorsqu'il y a beaucoup de features, et les problèmes de classification de texte en ont beaucoup.
- Les fonctions linéaires du noyau sont plus rapides que la plupart des autres et il y a moins de paramètres à optimiser.

$$f(X) = w^T \times X + b$$

- Dans cette équation, w est le vecteur de poids que vous souhaitez minimiser, X est les données que vous essayez de classer et b est le coefficient linéaire estimé à partir des données d'apprentissage. Cette équation définit la limite de décision renvoyée par le SVM.



Support Vector Machine : fonction noyau (kernel)

- Lorsque les données sont linéairement séparables, SVM trouve une frontière de décision linéaire qui sépare le plan en deux ou trois dimensions; voire un hyperplan de dimensions supérieures.
- En revanche, la plupart du temps, les données d'entreprises ne sont pas toutes linéairement séparables.
- Pour gérer ce type de cas, on utilise le kernel.
- L'astuce du noyau permet aux SVM de projeter les données d'entrée dans un espace de dimension supérieure où elles deviennent linéairement séparables.
- Différentes fonctions de noyau peuvent être utilisées, telles que les noyaux linéaires, polynomiaux, de fonction de base radiale (RBF) et sigmoïdes.

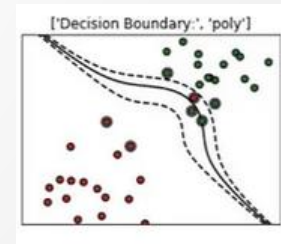
Support Vector Machine : fonction noyau (kernel)

Polynomial

- Le noyau polynomial n'est pas utilisé très souvent dans la pratique car il n'est pas aussi efficace en termes de calcul que les autres noyaux et ses prédictions ne sont pas aussi précises.

$$f(x_1, x_2) = (a + x_1^T \times x_2)^b$$

- C'est l'une des équations de noyau polynomiales simples que vous pouvez utiliser. $f(x_1, x_2)$ représente la limite de décision polynomiale qui séparera vos données. x_1 et x_2 représentent vos données.



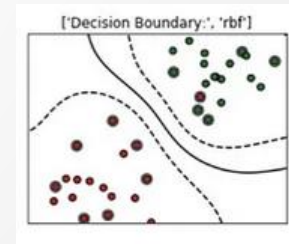
Support Vector Machine : fonction noyau (kernel)

Gaussienne (RBF)

- L'un des noyaux les plus puissants et les plus couramment utilisés dans les SVM. Généralement choisi pour les données non linéaires.

$$f(x_1, x_2) = \exp(-\gamma \times ||x_1 - x_2||^2)$$

- Dans cette équation, gamma spécifie combien un seul point d'entraînement a sur les autres points de données qui l'entourent. $||x_1 - x_2||$ est le produit scalaire entre les features.



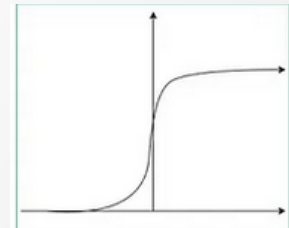
Support Vector Machine : fonction noyau (kernel)

Sigmoid

- Plus utile dans les réseaux de neurones que dans les machines à vecteurs de support, mais il existe parfois des cas d'utilisation spécifiques.

$$f(x, y) = \tanh(\alpha \times x_1^T \times y + C)$$

- Dans cette fonction, *alpha* est un vecteur de poids et *C* est une valeur de décalage pour tenir compte d'une mauvaise classification des données qui peut se produire.



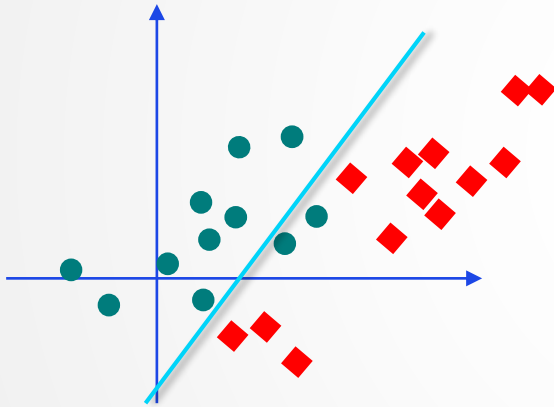
Support Vector Machine : paramètres

Régularisation

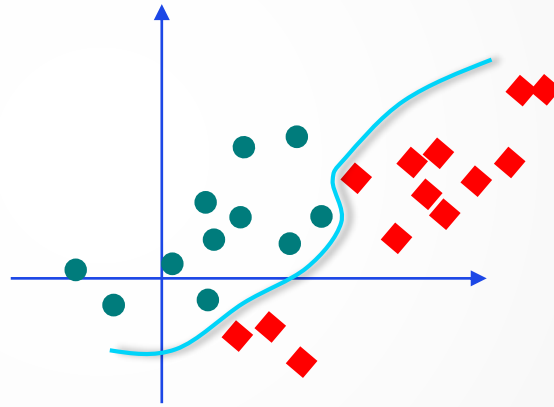
- Le paramètre de régularisation (souvent appelé C ou λ) sert de degré d'importance accordé aux erreurs de classification.
- SVM cherche à maximiser la marge entre les deux classes et à minimiser le nombre d'erreurs de classification.
- Si on choisit de grandes valeurs de C , l'optimisation choisira un hyperplan à plus petite marge si celui-ci parvient à classer correctement tous les points d'apprentissage.
- À l'inverse si C est petit, l'optimiseur recherchera un hyperplan de séparation à vaste marge, même si cet hyperplan classe mal plus de points.

Support Vector Machine : paramètres

Régularisation



Faible régularisation : C est petit

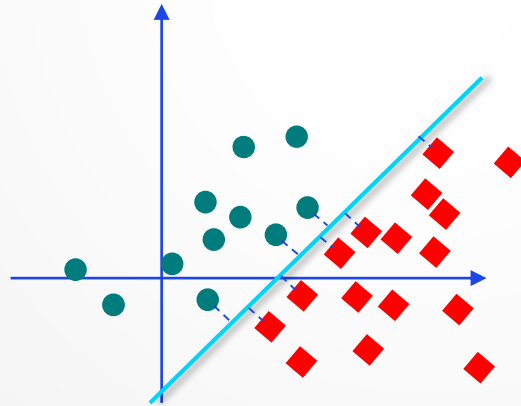


Forte régularisation : C est grand

Support Vector Machine : paramètres

Gamma

- Le paramètre *gamma* définit l'étendue de l'influence d'un seul exemple d'entraînement.
- *gamma* élevé signifie « proche », i.e. que seuls les points proches de la ligne séparatrice seront pris en compte.

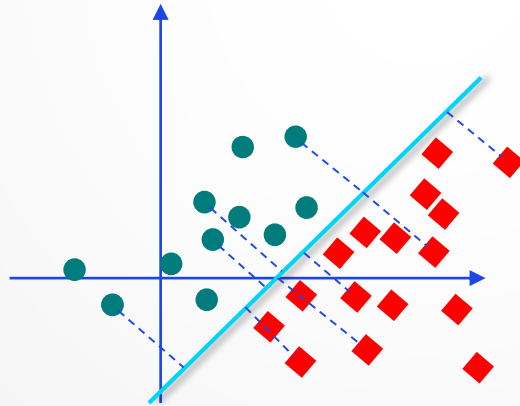


gamma élevé

Support Vector Machine : paramètres

Gamma

- En revanche, un *gamma* faible signifie « loin », i.e. que les points éloignés de la ligne séparatrice seront également pris en compte.

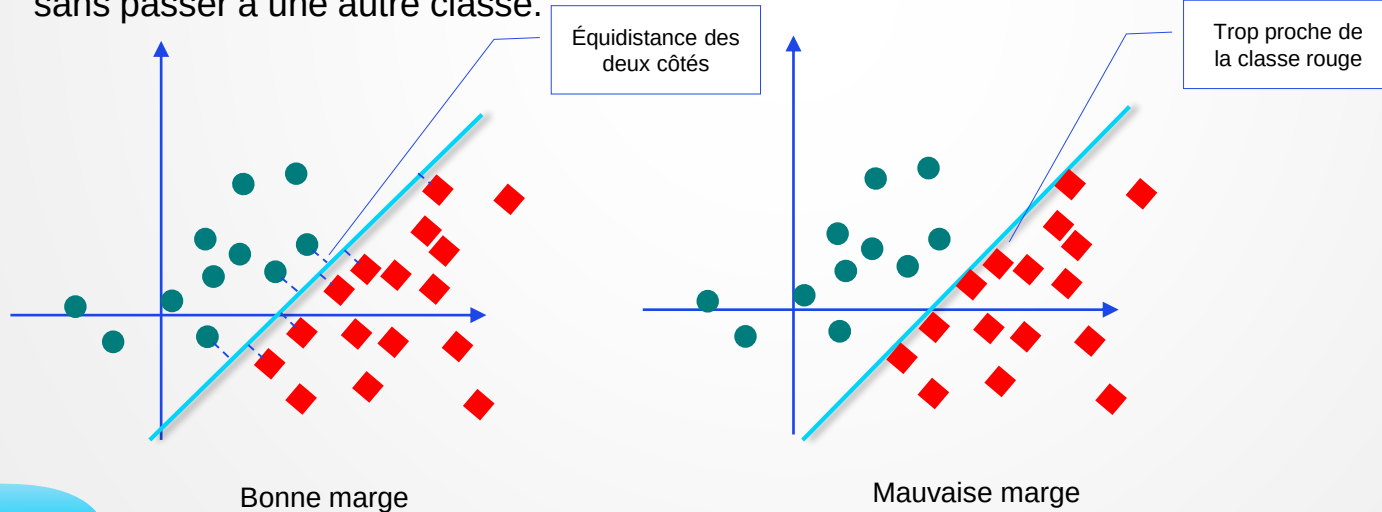


gamma élevé

Support Vector Machine : paramètres

Marge

- Une bonne marge est une marge dont la séparation est la plus grande pour les deux classes.
- Une bonne marge permet aux points d'être dans leurs classes respectives sans passer à une autre classe.



Support Vector Machine : avantages

- Efficace sur les ensembles de données avec plusieurs variables, comme les données financières ou médicales.
- Efficace dans les cas où le nombre de colonnes est supérieur au nombre de lignes.
- Il utilise un sous-ensemble d'individus pendant la phase d'apprentissage dans la fonction de décision du vecteur de support, ce qui le rend efficace en termes de mémoire.
- Différentes fonctions du noyau peuvent être spécifiées pour la fonction de décision. Vous pouvez utiliser le noyau par défaut, mais il est également possible de spécifier des noyaux personnalisés.

Support Vector Machine : inconvénients

- Si le nombre de features est beaucoup plus grand que le nombre de lignes, il est crucial d'éviter l'overfitting lors du choix des fonctions du noyau et du terme de régularisation.
- Les SVM ne fournissent pas directement des estimations de probabilité. Ceux-ci sont calculés à l'aide d'une validation croisée coûteuse en temps de calcul.
- Fonctionne mieux sur de petits ensembles d'échantillons en raison de son temps de d'entraînement élevé.



Skills4All
digital learning certifiant

Analyse en Composantes Principales

Analyse en Composantes Principales



Intuition



Fonctionnement



Représentation graphique



Code Python



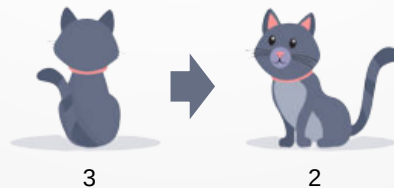
Interprétations

Analyse en Composantes Principales

- L'ACP permet d'étudier les ressemblances entre individus du point de vue des variables et dégage des profils d'individus.
- L'ACP permet aussi de faire le bilan des liaisons linéaires entre variables à partir des coefficients de corrélations.
- On veut représenter au mieux dans un espace plus réduit des observations issues d'un espace plus grand en nombres de dimensions.
- L'idée est de partir d'une réalité trop complexe pour résumer l'information.
- Généralement cette réalité complexe est résumée sur deux dimensions nouvellement créées appelées composantes principales.
- Ces composantes principales sont calculées à partir des variables originelles.

Analyse en Composantes Principales

- Par analogie, dans la réalité, le chat est un « objet » représenté en 3 dimensions (3D).
- Bien que nous évoluons dans un univers à 4 dimensions (3 dimensions d'espace et une dimension temporelle), nous ferons ici abstraction du temps.
- Selon la position du chat, on aura une image plus ou moins reconnaissable.
- Ici, l'angle du chat de droite est la meilleure façon de se le représenter en conservant un maximum d'information.

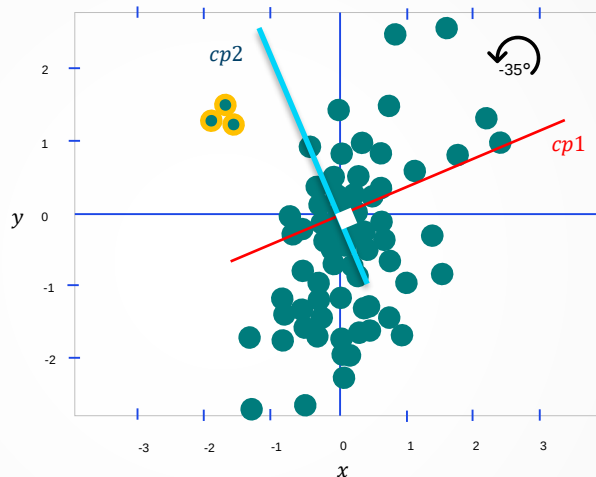
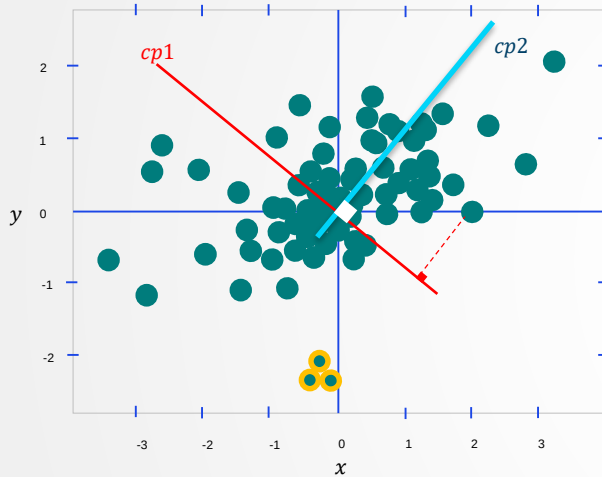


Analyse en Composantes Principales

- Le but recherché en entreprise sera de résumer de grands ensembles de données de façon à y dégager des structures, des formes sur les données qui n'auraient pas été possibles à appréhender.
- En d'autres termes, on veut résumer les données sur des composantes principales de façon à identifier les variables qui marquent une « similarité » ou « dissimilarité » entre les individus (variance).
- L'autre objectif consiste à examiner la position des n individus sur la plan factoriel, i.e. autour des composantes principales afin d'y dégager des typologies d'individus (cf. structures / formes).
- Enfin, on veut étudier les relation des p variables le long des composantes principales afin d'y dégager des typologies de variables.

Analyse en Composantes Principales

Notions de projections et de rotation



1. Orthogonalité de $cp1$ par rapport à $cp2$.
2. La rotation de la structure factorielle dans n'importe quelle direction n'impacte pas la position relative des points, les uns par rapport aux autres; toutefois, les coordonnées des points, c'est-à-dire les poids factoriels seraient bien évidemment modifiés.

Analyse en Composantes Principales

Notions de projections et de rotation

- La rotation de la structure factorielle n'est pas toujours nécessaire.
- Il existe plusieurs méthodes de rotation : Varimax, Oblimin, Quartimax, Equamax, Promax...
- L'objectif de ces méthodes est d'obtenir une structure claire des poids factoriels, c'est-à-dire des facteurs qui sont clairement marqués par de fortes corrélations avec certaines variables et par de faibles corrélations avec d'autres variables.
- Autrement dit, nous souhaitons trouver une rotation qui maximise la variance sur les nouvelles composantes; en d'autres termes, nous cherchons à obtenir une structure des poids pour chaque facteur qui soit aussi différente que possible, permettant une interprétation aussi facile que possible.

Analyse en Composantes Principales

Options de rotation

- **Aucune** : option par défaut.
- **Varimax** : Méthode de rotation orthogonale permettant de minimiser le nombre de variables qui pondèrent fortement chacun des facteurs. Cette méthode permet de simplifier l'interprétation des facteurs.
- **Oblimin** : Méthode de rotation oblique (non orthogonale).
- **Quartimax** : Méthode orthogonale qui minimise le nombre de facteurs requis pour expliquer chaque variable. Cette méthode permet de simplifier l'interprétation des variables observées.
- **Equamax** : Méthode de rotation qui combine la méthode Varimax, simplifiant les facteurs, et la méthode Quartimax, simplifiant les variables. Le nombre de variables pondérant fortement un facteur et le nombre de facteurs nécessaires pour expliquer une variable sont minimisés.
- **Promax** : Rotation oblique qui permet la corrélation des facteurs. Son calcul étant plus rapide que celui d'une rotation Oblimin, cette méthode s'avère particulièrement utile pour traiter des volumes importants de données.

Analyse en Composantes Principales

Covariance et ACP non normée

- Une ACP non normée est une ACP dans laquelle les variables quantitatives du tableau sont uniquement centrées (moyenne = 0).
- Pour obtenir les composantes principales, on calcule la covariance entre les variables.
- La covariance entre deux variables x_1 et x_2 indique si à un écart positif de x_1 pour un individu i par rapport à la moyenne sur x_1 , correspond un écart positif ou négatif de x_2 pour ce même individu i par rapport à la moyenne sur x_2 .

$$\text{cov}(x_1, x_2) = \frac{\sum_{i=1}^n (x_{i,1} - \bar{x}_1)(x_{i,2} - \bar{x}_2)}{n - 1}$$

- La covariance indique donc si les valeurs de x_1 augmentent ou diminuent avec les valeurs croissantes de x_2 . Elle indique le degré de variation conjointe entre les deux variables.
- Regarder le signe de la covariance :
 - $\text{cov}(x_1, x_2) > 0$, x_1 augmente quand x_2 augmente (météo vs consommation de glaces)
 - $\text{cov}(x_1, x_2) < 0$, x_1 augmente quand x_2 baisse (chauffage vs météo)
- Si x_1 et x_2 sont centrées, alors $\bar{x}_1 = \bar{x}_2 = 0$

Analyse en Composantes Principales

Covariance et ACP non normée

- La covariance est un indicateur de la direction linéaire entre x_1 et x_2 :
 - Si sa valeur est positive, alors il y a une liaison croissante
 - Si sa valeur est négative, alors il y a une liaison décroissante
 - Si sa valeur est nulle, alors il y a une absence de liaison
- D'ailleurs si $x_1 \perp x_2 \Rightarrow cov(x_1, x_2) = 0$, i.e. que si deux variables sont indépendantes alors leur covariance est nulle.
- La covariance dépend de l'unité de mesure de x_1 et x_2 .
- La covariance n'a pas de valeur minimale, ni de valeur maximale.
- Attention : $cov(x_1, x_1) = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n} = V(x)$, i.e. la covariance d'une variable avec elle-même est tout simplement la variance (autocovariance).
- Ne confondez pas covariance et corrélation (évitons toute digression pour l'instant).

Analyse en Composantes Principales

Covariance et ACP non normée

- Si x_1 et x_2 sont centrées, alors leurs espérances sont nulles :

$$x_1 - E(x_1) = 0 \text{ et } x_2 - E(x_2) = 0, \text{ donc } \overline{x_1} = \overline{x_2} = 0$$

- $cov(x_1, x_2) = cov(x_2, x_1)$
- Pour $p \geq 2$, on calcule la covariance pour toutes les paires de variables possibles. Soit la matrice de covariances C :

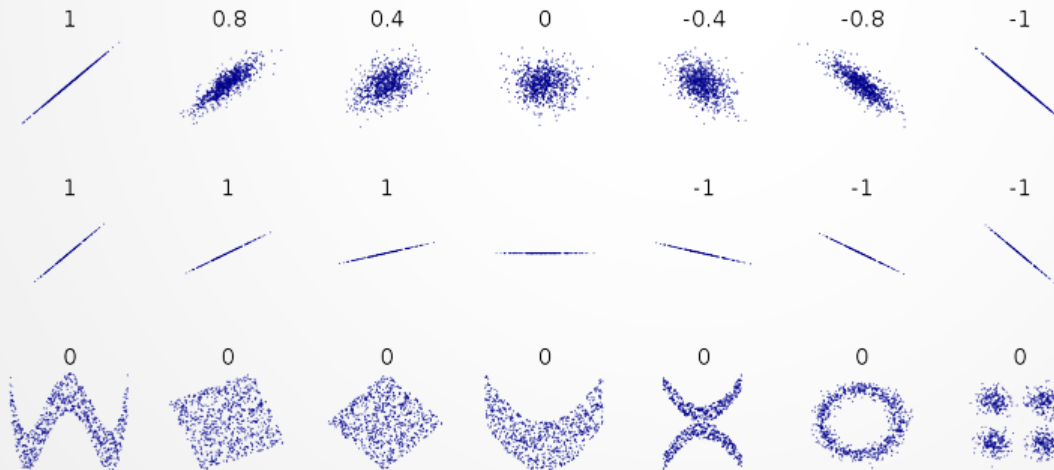
$$C = \begin{pmatrix} \text{var}(x_1) & cov(x_1, x_2) & \cdots & cov(x_1, x_j) & \cdots & cov(x_1, x_p) \\ cov(x_2, x_1) & \text{var}(x_2) & \cdots & cov(x_2, x_j) & \cdots & cov(x_2, x_p) \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ cov(x_j, x_1) & cov(x_j, x_2) & \cdots & \text{var}(x_j) & \cdots & cov(x_j, x_p) \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ cov(x_p, x_1) & cov(x_p, x_2) & \cdots & cov(x_p, x_j) & \cdots & \text{var}(x_p) \end{pmatrix}$$

- C est une matrice carrée de taille $p \times p$, i.e. le nombre de lignes est égal au nombre de colonnes.
- C est une matrice symétrique, i.e. une matrice carrée qui est égale à sa transposée.

Analyse en Composantes Principales

Corrélation et ACP normée

- Une ACP normée est une ACP dans laquelle les variables quantitatives du tableau sont centrées réduites (moyenne = 0 et variance = 1).
- La corrélation mesure l'intensité de la liaison linéaire entre deux variables x_1 et x_2 .



Analyse en Composantes Principales

Corrélation et ACP normée

- L'ACP normée est basée sur la matrice de corrélations.

- Soit la matrice de covariances C :

$$C = \begin{pmatrix} 1 & \rho(x_1, x_2) & \cdots & \rho(x_1, x_j) & \cdots & \rho(x_1, x_p) \\ \rho(x_2, x_1) & 1 & \cdots & \rho(x_2, x_j) & \cdots & \rho(x_2, x_p) \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \rho(x_j, x_1) & \rho(x_j, x_2) & \cdots & 1 & \cdots & \rho(x_j, x_p) \\ \cdots & \cdots & \cdots & \cdots & \cdots & \cdots \\ \rho(x_p, x_1) & \rho(x_p, x_2) & \cdots & \rho(x_p, x_j) & \cdots & 1 \end{pmatrix}$$

- C est une matrice carrée de taille $p \times p$, i.e. le nombre de lignes est égal au nombre de colonnes.
- C est une matrice symétrique, i.e. une matrice carrée qui est égale à sa transposée.
- C a une diagonale de 1

Analyse en Composantes Principales

ACP non normée :

- Il faut toujours centrer vos données par défaut. On utilise une ACP non normée lorsque les données s'expriment toutes dans la même unité de mesure et varient dans des gammes de valeurs identiques.

ACP normée :

- On utilise une ACP normée lorsque les données s'expriment dans des unités différentes et varient dans des gammes de valeurs différentes.

Analyse en Composantes Principales

Interprétations

Composantes principales :

- Les composantes principales sont les nouvelles variables créées par l'ACP, qui sont des combinaisons linéaires des variables d'origine. Chaque composante principale représente une part de la variance totale des données. L'interprétation des composantes principales implique d'examiner les coefficients attribués à chaque variable dans chaque composante principale pour comprendre quelles variables contribuent le plus à la variance observée.

Contribution des variables :

- Les contributions des variables permettent d'évaluer l'importance relative de chaque variable dans la formation des composantes principales. Plus la contribution d'une variable à une composante principale est élevée, plus cette variable a d'influence sur cette composante.

Analyse en Composantes Principales

Interprétations

Le cercle des corrélations :

- Appelé également « biplot », peut être utilisé pour représenter graphiquement les variables et les individus dans l'espace des composantes principales. Cette visualisation permet de voir les relations entre les variables, ainsi que la proximité des individus par rapport aux variables et vice versa.

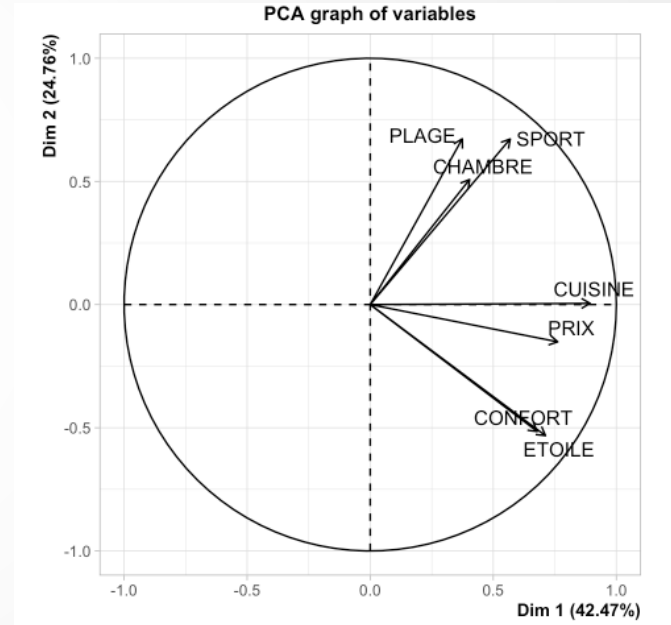
Variance expliquée :

- L'ACP fournit également des informations sur la variance expliquée par chaque composante principale. Cela permet de déterminer quelle proportion de la variance totale est expliquée par chaque composante, ce qui peut aider à sélectionner les composantes principales les plus significatives pour l'interprétation des données.

Analyse en Composantes Principales

Effet taille

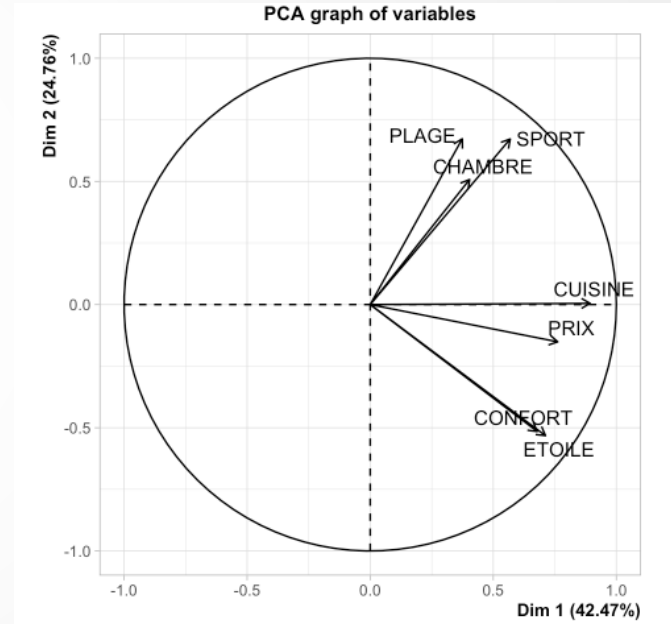
- Lorsqu'on applique l'analyse en composantes principales (ACP), l'effet de la taille des variables peut jouer un rôle important dans l'interprétation des résultats.
- L'effet de la taille se réfère au fait que certaines variables peuvent avoir une plus grande variabilité en raison de leur échelle de mesure, ce qui peut influencer les résultats de l'ACP.
- Si certaines variables sont mesurées sur des échelles différentes ou ont des unités de mesure différentes, elles peuvent avoir une variance plus élevée simplement en raison de leur échelle de mesure, et cela peut biaiser les résultats de l'ACP.



Analyse en Composantes Principales

Effet taille

- Les variables avec des variances plus élevées auront tendance à avoir un poids plus important dans la formation des composantes principales.
- Pour éviter cet effet de taille, il est courant de standardiser les variables avant d'appliquer l'ACP.
- La standardisation consiste à centrer les variables autour de zéro en soustrayant la moyenne et à les diviser par l'écart type.
- Cela permet de mettre toutes les variables sur une même échelle et de donner une importance équivalente à chacune d'entre elles lors de la construction des composantes principales.





Skills4All
digital learning certifiant

Analyse Factorielle des Correspondances

Analyse Factorielle des Correspondances



Intuition



Fonctionnement



Représentation graphique



Code Python



Interprétations

Analyse Factorielle des Correspondances

- L'AFC permet de résumer et de visualiser un tableau de contingence, i.e. qui croise deux variables qualitatives.
- Les objectifs de l'AFC sont les suivants :
 - Comparer les profils-lignes entre eux
 - Comparer les profils-colonnes entre eux
 - Interpréter les proximités entre les lignes et les colonnes

Analyse Factorielle des Correspondances

Interprétations

- **Représentation graphique :**
 - Les individus et les catégories de variables sont positionnés sur ce plan en fonction de leurs relations mutuelles. L'interprétation consiste à analyser la proximité ou l'éloignement des catégories et des individus afin de détecter des tendances ou des structures sous-jacentes.
- **Contribution des variables :**
 - L'AFC fournit des mesures de contribution des variables et des individus. Les contributions des variables indiquent dans quelle mesure chaque variable contribue à la formation des axes principaux de l'AFC.
 - Les contributions des individus permettent d'évaluer l'importance de chaque individu dans la structure globale des données.
 - Ces mesures aident à identifier les variables et les individus qui ont une influence significative sur les résultats.

Analyse Factorielle des Correspondances

Interprétations

- **Liens entre les catégories de variables :**
 - L'interprétation de l'AFC implique également l'examen des liens entre les différentes catégories de variables.
 - Les catégories proches les unes des autres sur le plan sont associées de manière similaire, tandis que les catégories éloignées sont moins liées.
 - L'analyse des proximités entre les catégories permet de dégager des tendances, des regroupements ou des oppositions entre les variables.



Skills4All
digital learning certifiant

Deep Learning : CNN

Objectifs pédagogiques

- Comprendre les concepts de base du Deep Learning.
- Explorer les différents algorithmes du Deep Learning couramment utilisés.
- Acquérir des compétences pratiques en extraction et transformation des données en vue d'implémenter une solution de Deep Learning.
- Comprendre les bonnes pratiques pour implémenter des réseaux de neurones robustes..
- Appliquer les connaissances acquises pour résoudre des cas d'utilisation spécifiques en utilisant des exemples concrets.



Skills4All
digital learning certifiant

Plan de formation

Plan du cours

Partie 1 – Introduction au Deep Learning

- Généralités
 - Qu'est ce que le Deep Learning ?
 - Quelques librairies utiles au Deep Learning
 - TensorFlow
 - Keras
 - Pytorch
 - NumPy
 - Scikit Learn
 - Matplotlib
 - Seaborn
 - Pandas
 - Jeux de données : iris, MNIST, CIFAR-10, cats vs dogs, ImageNet...

Plan du cours

Partie 2 – Environnement du Deep Learning

- Préparer son environnement de travail
 - Installation de Python
 - Python sur Google Colab
 - Créer un environnement virtuel
 - Activer un environnement virtuel
 - Installer Jupyter Notebook
 - Installer TensorFlow

Plan du cours

Partie 3 – Prérequis au Deep Learning

- Réseaux de neurones à propagation avant
 - Perceptrons
 - Fonctions de perte d'entropie croisée
 - Dérivées et descente de gradient stochastique

Plan du cours

Partie 4 – Réseaux de neurones convolutifs (CNN)

- Classification d'images avec CNN
 - Comment les ordinateurs appréhendent les images
 - Le processus de convolution
 - L'opération ReLu
 - L'opération Pooling
 - Flattening et couches entièrement liées
 - Implémenter CNN avec TensorFlow et Keras



Skills4All
digital learning certifiant

01. Introduction au Deep Learning

01 – Introduction au Deep Learning

Partie 1 – Introduction au Deep Learning

- Généralités
 - Qu'est ce que le Deep Learning ?
 - Quelques librairies utiles au Deep Learning
 - TensorFlow
 - Keras
 - Pytorch
 - NumPy
 - Scikit Learn
 - Matplotlib
 - Seaborn
 - Pandas
 - Jeux de données : iris, MNIST, CIFAR-10, cats vs dogs, ImageNet...

Introduction au DL : généralités

Qu'est ce que le Deep Learning ?

- L'apprentissage profond est un des composants de l'Intelligence Artificielle.
- Il s'inspire de la structure et du fonctionnement des réseaux de neuronaux du cerveau humain.
- Les algorithmes du Deep Learning tentent d'apprendre des représentations des données à travers des couches (layer) abstraites.

Introduction au DL : généralités

Qu'est ce que le Deep Learning ?

Composants du Deep Learning :

- Réseaux neuronaux
- Réseaux neuronaux profonds
- Données d'entraînement
- Processus d'entraînement
- Rétropropagation
- Fonctions d'activation

Introduction au DL : généralités

Qu'est ce que le Deep Learning ?

1. Réseaux neuronaux :

- Ce sont les éléments fondamentaux du Deep Learning.
- Ils sont composés de neurones interconnectés organisés en couches.
- Chaque couche traite les données d'entrée et les transmet à la couche suivante.

2. Réseaux neuronaux profonds :

- Ce sont des réseaux de neurones avec de nombreuses couches.
- Ils ont une grande capacité à comprendre des motifs et des représentations complexes à partir d'une grande quantité de données.

Introduction au DL : généralités

Qu'est ce que le Deep Learning ?

3. Données d'entraînement :

- Les modèles de Deep Learning nécessitent une grande quantité de données d'entraînement étiquetées pour apprendre efficacement les motifs.
- Plus il y a de données disponibles, meilleur est généralement le modèle.

4. Processus d'entraînement :

- Ils apprennent de manière itérative à travers un processus appelé entraînement.
- Pendant l'entraînement, le modèle ajuste ses paramètres internes (poids et biais) pour minimiser la différence entre ses prédictions et les valeurs cibles réelles.

Introduction au DL : généralités

Qu'est ce que le Deep Learning ?

5. Rétropropagation :

- Il s'agit d'un algorithme clé utilisé pour entraîner les réseaux neuronaux profonds.
- Il calcule le gradient de la fonction de perte par rapport aux paramètres du modèle
- Ce qui permet au modèle de mettre à jour ses paramètres dans la direction qui minimise la perte.

6. Fonctions d'activation :

- Ce sont des fonctions non linéaires appliquées à la sortie de chaque neurone dans une couche de réseau neuronal.
- Elles introduisent une non-linéarité dans le réseau, permettant d'apprendre des relations complexes dans les données.

Introduction au DL : quelques librairies utiles au DL

TensorFlow

- TensorFlow est une bibliothèque open-source d'apprentissage automatique.
- Elle est utilisée pour la **création** et le **déploiement** de modèles d'apprentissage automatique, en particulier dans le domaine du Deep Learning.
- Il offre une grande flexibilité et une grande puissance pour la conception et l'entraînement de réseaux neuronaux.

Introduction au DL : quelques librairies utiles au DL

Keras

- Keras est une bibliothèque open-source d'apprentissage automatique écrite en Python.
- Elle fournit une interface conviviale et **modulaire** pour la création, la configuration et l'entraînement de réseaux neuronaux, en particulier les réseaux de neurones profonds.
- **Modularité**: Keras permet de construire des modèles d'apprentissage automatique en assemblant des blocs de construction modulaires appelés couches. Ces couches peuvent être facilement combinées pour créer des architectures de réseau complexes.

Introduction au DL : quelques librairies utiles au DL

Pytorch

- IPyTorch est une bibliothèque open-source d'apprentissage automatique développée principalement par Facebook's AI Research lab (FAIR).
- Elle fournit une infrastructure flexible et efficace pour la création et le déploiement de modèles d'apprentissage automatique, en particulier dans le domaine du Deep Learning.

Introduction au DL : quelques librairies utiles au DL

Numpy

- NumPy est une bibliothèque open-source pour le langage de programmation Python, destinée à faciliter le calcul numérique.
- Elle fournit des structures de données et des fonctionnalités pour travailler efficacement avec des tableaux multidimensionnels ainsi que des fonctions mathématiques pour opérer sur ces tableaux.

Introduction au DL : quelques librairies utiles au DL

Scikit Learn

- Scikit-learn, également connu sous le nom sklearn, est une bibliothèque open-source de Machine Learning pour le langage de programmation Python.
- Elle offre une gamme complète d'outils pour la construction et l'évaluation de modèles d'apprentissage automatique dans divers domaines tels que la classification, la régression, le clustering et la réduction de dimensionnalité.

Introduction au DL : quelques librairies utiles au DL

Matplotlib

- Matplotlib est une bibliothèque de visualisation de données en Python, largement utilisée pour créer des graphiques et des visualisations de qualité professionnelle dans divers domaines, y compris la science des données, la recherche académique, l'ingénierie et bien d'autres.

Introduction au DL : quelques librairies utiles au DL

Seaborn

- Seaborn est une bibliothèque de visualisation de données en Python.
- Elle fournit une interface de haut niveau pour créer des graphiques statistiques attrayants et informatifs.
- Les graphiques de Seaborn sont bien plus esthétiques que ceux de son homologue Matplotlib.

Introduction au DL : quelques librairies utiles au DL

Pandas

- Pandas est une bibliothèque open-source en Python, spécialisée dans la manipulation et l'analyse de données.
- Elle offre des structures de données et des outils performants pour effectuer des opérations de nettoyage, de transformation, de filtrage et d'analyse de données.



Skills4All
digital learning certifiant

02. Environnement du Deep Learning

02 – Environnement du DL

Partie 2 – Environnement du Deep Learning

- Préparer son environnement de travail
 - Installation de Python
 - Python sur Google Colab
 - Créer un environnement virtuel
 - Activer un environnement virtuel
 - Installer Jupyter Notebook
 - Installer TensorFlow

Préparer son environnement de travail

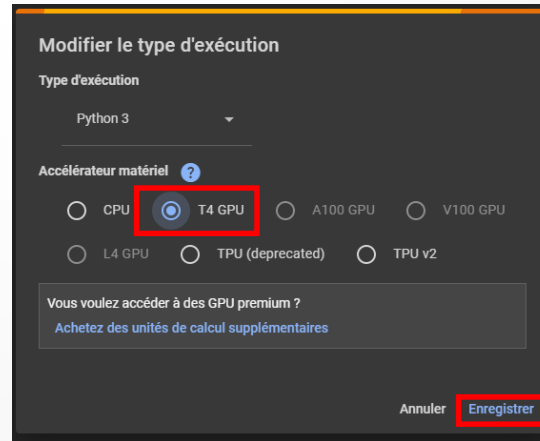
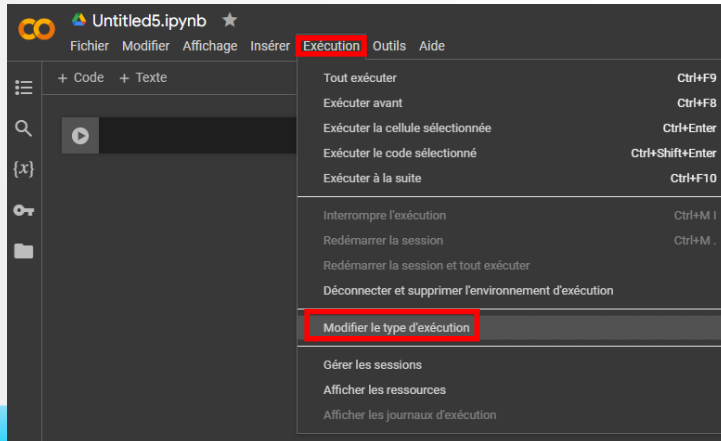
Installation de Python

- Téléchargez et installez Anaconda : <https://www.anaconda.com/>
- Anaconda est une distribution Python utilisée pour le calcul scientifique et l'analyse de données.
- Cette distribution comprend une vaste collection de packages Python pré-installés qui sont couramment utilisés dans les domaines de la science des données, de l'apprentissage automatique, de l'analyse de données et bien plus.
- **Environnements virtuels** : Conda permet de créer des environnements virtuels isolés, ce qui permet aux utilisateurs de travailler sur différents projets avec des configurations de packages distinctes sans conflits de dépendances.
- **Anaconda Navigator** : Anaconda Navigator est une interface graphique conviviale qui permet aux utilisateurs de gérer leurs environnements et packages, ainsi que de lancer des applications, des éditeurs de code et des notebooks Jupyter.

Préparer son environnement de travail

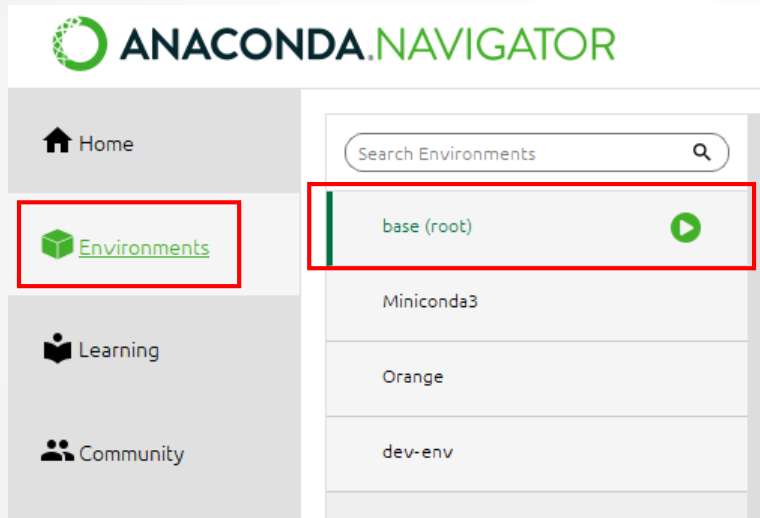
Python sur Google Colab

- Vous pouvez faire du Machine et du Deep Learning directement dans Google Colab (support GPU).
- Pour accéder à Google Colab, suivez le lien : <https://colab.research.google.com/>
- Pour utiliser le GPU, suivez les étapes suivantes :



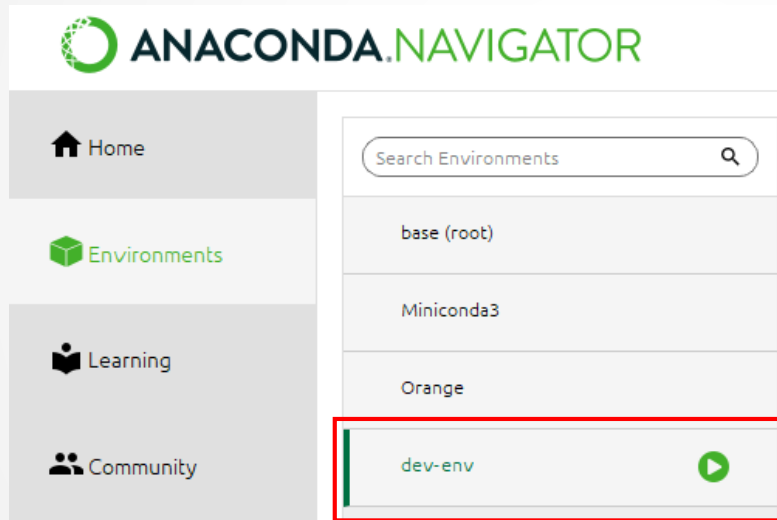
Préparer son environnement de travail

Créer un environnement virtuel



Préparer son environnement de travail

Activer un environnement virtuel



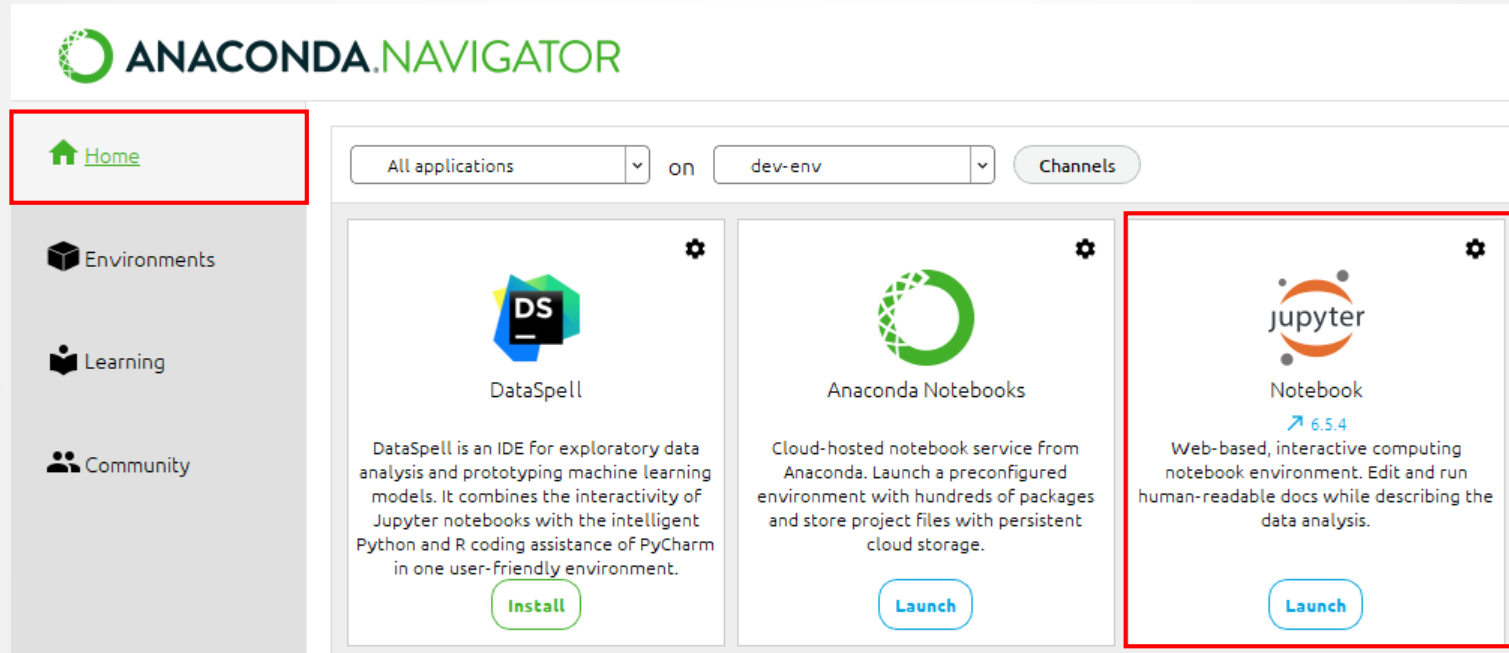
Préparer son environnement de travail

Installer Jupyter Notebook

- **Notebook** : Interface utilisateur web qui permet de combiner du texte, du code exécutable, des visualisations et d'autres éléments interactifs dans un seul document.

Préparer son environnement de travail

Installer Jupyter Notebook



Préparer son environnement de travail

Installer TensorFlow

```
# installation et mise à jour de tensorflow  
!pip install --upgrade tensorflow
```

```
# installation et mise à jour de keras  
!pip install --upgrade tf-keras
```

```
# dernière version de tensorflow  
import tensorflow as tf  
print (tf.__version__)
```



Skills4All
digital learning certifiant

03. Prérequis au Deep Learning

03 – Prérequis au Deep Learning

Partie 3 – Prérequis au Deep Learning

- Réseaux de neurones à propagation avant
 - Perceptrons
 - Fonctions de perte d'entropie croisée
 - Dérivées et descente de gradient stochastique

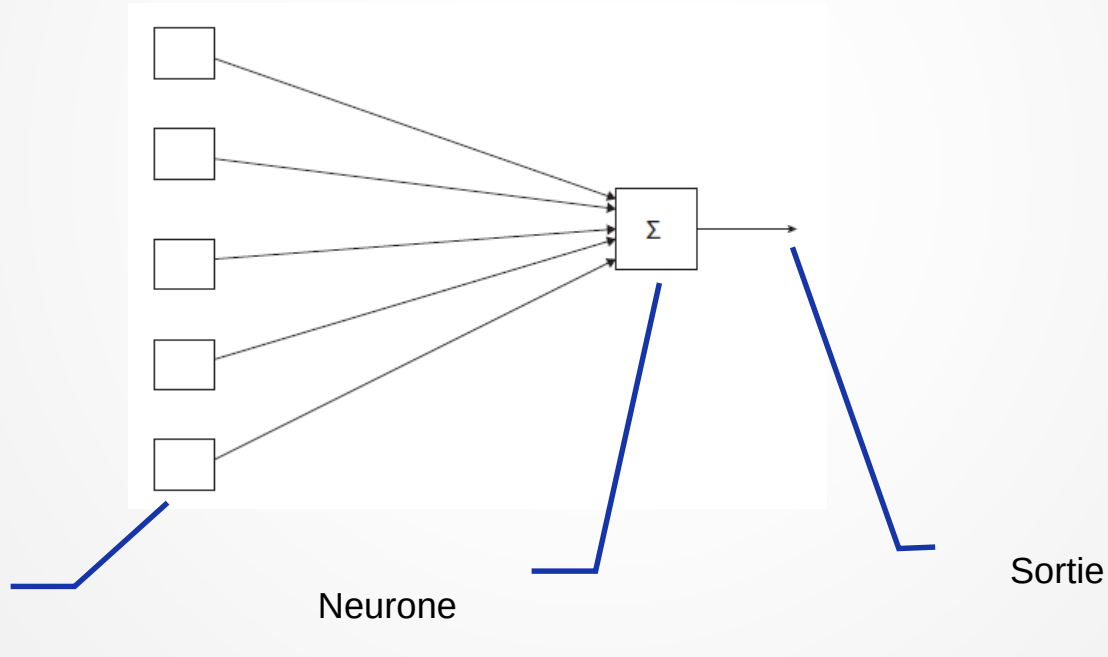
03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons

- Les perceptrons (1957, Frank Rosenblatt) sont une modélisation informatique simple des neurones.
- Un neurone humain comporte plusieurs entrées (dendrites), un corps de cellule et une seule sortie (axone).
- De même, le perceptron informatique reçoit de nombreuses entrées et possède une seule sortie.
- Par exemple, on peut créer un programme qui décidera si une image est un 0 ou non (problème de classification binaire).
- Pour y répondre, un perceptron simple qui se base sur une image 28×28 aurait 784 entrées, une par pixel, et une sortie.

03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons



03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons : fonctionnement

- Deux petites questions :
 - *À quoi reconnaît-on un tigre ?*
 - *Comment faites-vous la différence entre un lapin et un lièvre ?*

03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons : fonctionnement, entrées et poids

- Un perceptron prend un vecteur d'entrée $x = (x_1, x_2, \dots, x_n,)$.
- x peut être l'image d'un tigre 28×28 par exemple.
- Chaque x_i est une caractéristique (variable) de l'entrée.
- x_i peut être une caractéristique du pixel (admettons une rayure)
- Chaque caractéristique x_i peut être pondérée par un poids qui lui est propre w_i .
- Ces poids déterminent l'importance relative de chaque caractéristique x_i dans la décision du perceptron.



03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons : fonctionnement, combinaison linéaire

- Les entrées x_i sont combinées linéairement avec les poids w_i correspondants pour former une somme pondérée $z = \sum_{i=1}^n w_i \cdot x_i$
- $w_i \cdot x_i$ est donc un produit scalaire de deux vecteurs de longueur l

03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons : fonctionnement, **fonction d'activation**

- La somme pondérée z est ensuite passée à travers une fonction d'activation (ϕ) notée $\varphi_{(z)}$.
- Cette fonction d'activation est généralement une fonction non linéaire qui introduit de la non-linéarité dans le modèle, permettant ainsi au perceptron d'apprendre des relations complexes dans les données.
- La fonction est donnée par :
$$\varphi_{(z)} = \begin{cases} 1 & \text{si } b + \sum_{i=1}^n w_i \cdot x_i > 0 \\ 0 & \text{sinon} \end{cases}$$
- b est un poids particulier, appelé biais.

03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons : fonctionnement, **sortie**

- La sortie du perceptron est le résultat de la fonction d'activation appliquée à la somme pondérée z .
- Cette sortie peut être interprétée comme la prédiction du perceptron pour une classe particulière, par exemple, dans une tâche de classification binaire, la sortie peut être soit 0 (classe négative) soit 1 (classe positive).
- Dans notre exemple, on peut noter : $\varphi_{(z)} = \begin{cases} 1 & \text{tigre} \\ 0 & \text{pas tigre} \end{cases}$

03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons : fonctionnement, entraînement

- Pendant la phase d'entraînement, les poids w_i du perceptron sont ajustés itérativement afin de minimiser une fonction de perte qui mesure la différence entre les prédictions du perceptron et les étiquettes réelles des données d'entraînement.
- Cela se fait généralement à l'aide d'algorithmes d'optimisation comme la descente de gradient.

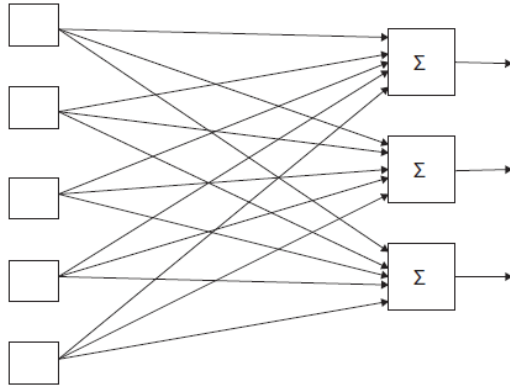
03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons : fonctionnement, **test et validation**

- Une fois que le perceptron est entraîné, il peut être utilisé pour faire des prédictions sur de nouvelles données en alimentant ces données dans le perceptron et en observant la sortie.

03 – Prérequis au DL : réseaux de neurones à propagation avant

Perceptrons multiples



- Trois perceptrons distincts qui partagent les mêmes entrées.
- Il s'agit de perceptrons multiples pour l'identification de classes multiples.
- Tous les perceptrons sont entraînés indépendamment les uns des autres.

03 – Prérequis au DL : réseaux de neurones à propagation avant

Fonction de perte d'entropie croisée

- La fonction de perte (ou fonction d'erreur) est une mesure utilisée pour évaluer la performance d'un modèle par rapport aux données d'entraînement.
- Elle mesure la différence entre les prédictions du modèle et les valeurs réelles des données d'entraînement, et est utilisée comme guide pour ajuster les poids du réseau neuronal lors de l'entraînement afin de minimiser cette différence.
- Le choix de la fonction de perte dépend du type de problème que vous essayez de résoudre (classification, régression, etc.) et des caractéristiques de vos données.
- Notre objectif est de minimiser la perte.

03 – Prérequis au DL : réseaux de neurones à propagation avant

Fonction de perte d'entropie croisée

- La fonction de perte d'entropie croisée est utilisée principalement pour les problèmes de classification.
- Il existe deux variantes principales : la perte de l'entropie croisée binaire (BCE) pour les problèmes de classification binaire et la perte de l'entropie croisée catégorielle (CCE) pour les problèmes de classification multi-classes.
- La fonction de perte mesure la divergence entre la distribution de probabilité prédite par le modèle et la distribution de probabilité réelle des étiquettes de classe.

03 – Prérequis au DL : réseaux de neurones à propagation avant

Dérivée et descente de gradient stochastique

- La dérivée d'une fonction représente son taux de variation instantanée en un point donné.
- Autrement dit, la dérivée de la fonction $f(x)$ en un point x est définie comme la limite de la variation de $f(x)$ divisée par la variation de x lorsque la variation de x tend vers 0.
- On note $f'(x)$ ou $\frac{df(x)}{dx}$
- La dérivée d'une fonction donne la direction dans laquelle la fonction croît ou décroît le plus rapidement au voisinage du point considéré.

03 – Prérequis au DL : réseaux de neurones à propagation avant

Dérivée et descente de gradient stochastique

- La descente de gradient est un algorithme d'optimisation utilisé pour minimiser une fonction objective en ajustant itérativement les paramètres du modèle dans la direction opposée du gradient de la fonction.
- L'idée principale est de suivre la direction dans laquelle la fonction décroît le plus rapidement.
- Le terme de descente de gradient tire son nom du fait qu'on regarde la pente de la fonction de perte (son gradient) et qu'on demande au système de minimiser sa perte (descente) en suivant ce gradient.
- Cette méthode d'apprentissage s'appelle la **rétropropagation**.

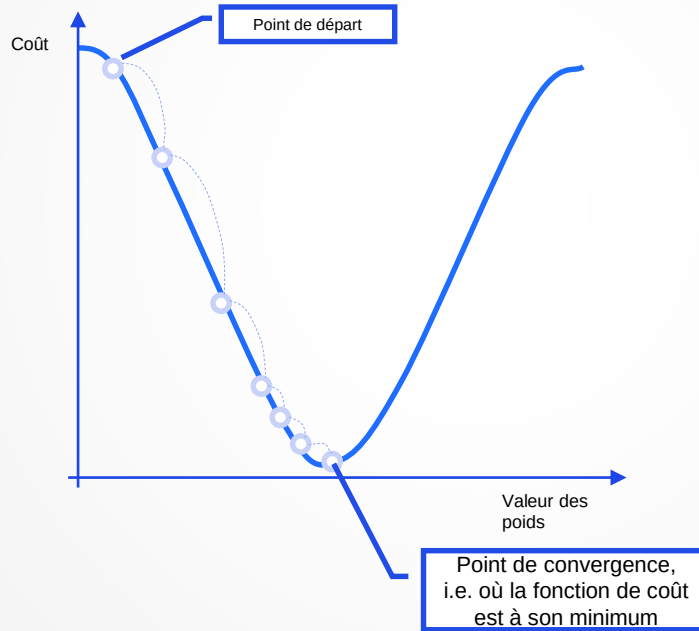
03 – Prérequis au DL : réseaux de neurones à propagation avant

Dérivée et descente de gradient stochastique

- La descente de gradient stochastique est une variante de la descente de gradient classique dans laquelle les mises à jour des paramètres du modèle sont calculées non pas sur l'ensemble complet des données d'entraînement, mais sur un seul exemple (ou un petit sous-ensemble d'exemples) choisi aléatoirement à chaque itération.
- Cela permet d'accélérer l'apprentissage et de gérer efficacement les grands ensembles de données.
- En pratique, la SGD converge souvent plus rapidement que la descente de gradient classique, bien que sa convergence puisse être plus bruitée en raison de la sélection aléatoire des exemples.

03 – Prérequis au DL : réseaux de neurones à propagation avant

Dérivée et descente de gradient stochastique





Skills4All
digital learning certifiant

04. Réseaux de neurones convolutifs

04 – Réseaux de neurones convolutifs

Partie 4 – Réseaux de neurones convolutifs (CNN)

- Classification d'images avec CNN
 - Comment les ordinateurs appréhendent les images
 - Le processus de convolution
 - L'opération ReLu
 - L'opération Pooling
 - Flattening et couches entièrement liées
 - Implémenter CNN avec TensorFlow et Keras

04 – Réseaux de neurones convolutifs : classification d'images (CNN)

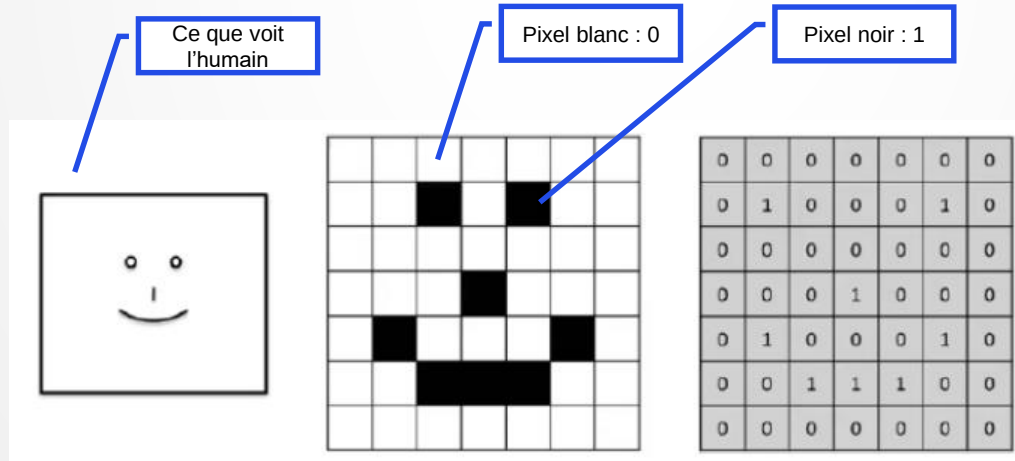
Comment les ordinateurs appréhendent les images

- Quand un humain regarde une image, il y voit des lignes, des cercles, des carrés et d'autres formes.
- La preuve : l'exercice précédent sur le tigre.
- En revanche, un ordinateur ne le « voit » pas de la même façon.
- Une image est représentée comme un assemblage de pixels en deux dimensions.
- Pour les images avec nuances gris, la valeur du pixel peut être comprise entre 0 et 255
- Tandis que pour la couleur, on a trois canaux : rouge, vert et bleu.
- Chaque canal peut avoir une valeur de pixel comprise entre 0 et 255.

04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Comment les ordinateurs appréhendent les images

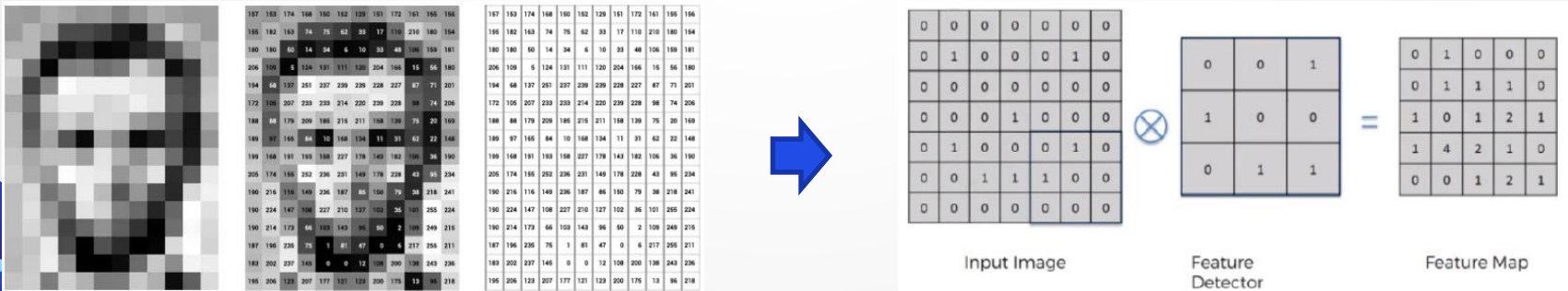
Comment un ordinateur « voit »
une image.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Le processus de convolution

- La convolution est une étape cruciale pour les CNN.
 - Elle sert à extraire des caractéristiques significatives des données d'entrée (images).
- Filtre (Kernel) :** le noyau (filtre ou kernel) est une petite matrice de poids initialisée de façon aléatoire. Ce kernel est appliqué à une région de l'image en cours d'analyse. Il est de petite taille, généralement 3×3 ou 5×5 et est conçu pour détecter des caractéristiques spécifiques telles que les bords, les textures, des motifs, etc.

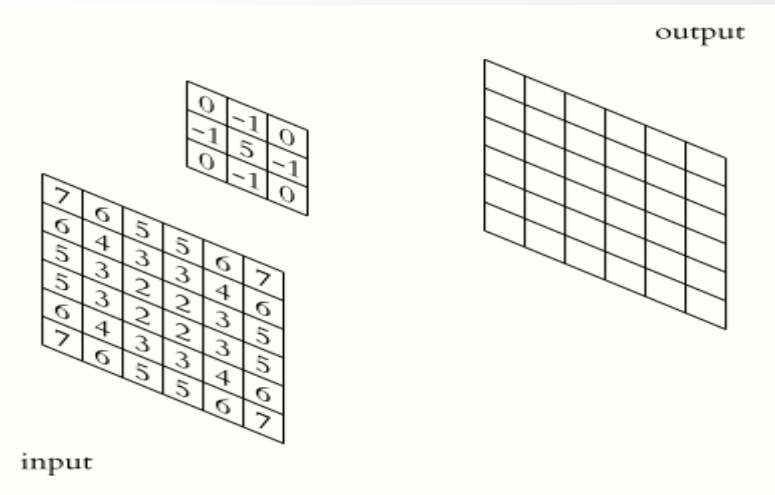


04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Le processus de convolution

2. Opération de convolution :

- Le filtre est déplacé à travers l'image d'entrée (**input**) pixel par pixel (parfois avec une certaine stride) et une opération de multiplication élément par élément est effectuée entre les valeurs des pixels de l'image et les valeurs du filtre.
- Une stride est le nombre d'étapes qu'un kernel suit de gauche à droite, puis de haut en bas pour trouver une nouvelle valeur pour la carte des caractéristiques.
- Ces produits sont ensuite sommés pour obtenir une valeur unique qui représente la réponse du filtre à cette position particulière de l'image.



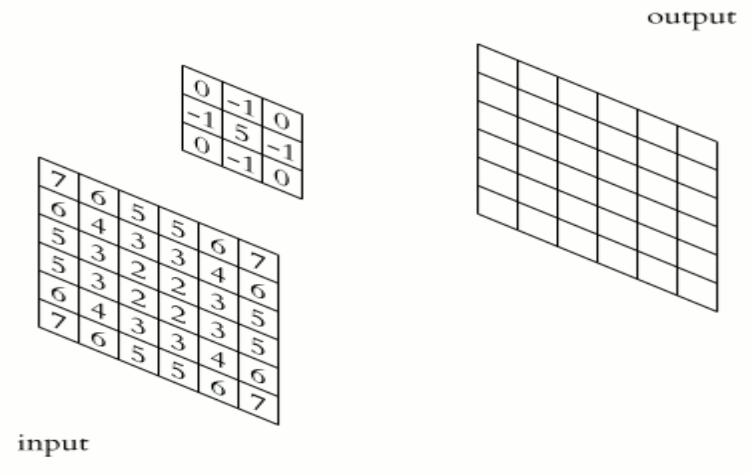
04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Le processus de convolution

3. Carte des caractéristiques (Feature Map)

:

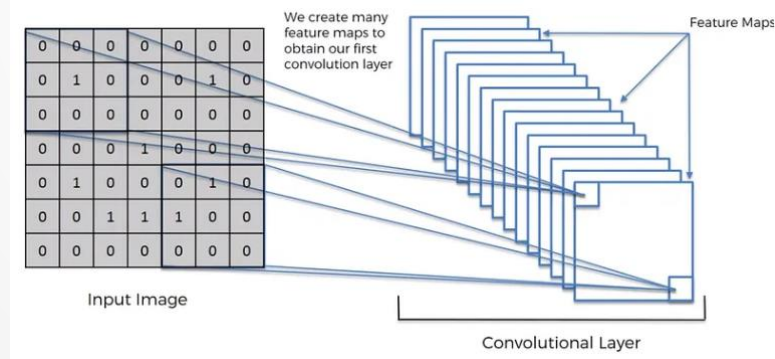
- L'opération de convolution produit une carte des caractéristiques, également appelée carte de sortie (**output**).
- Cette carte met en évidence la présence ou l'absence de certaines caractéristiques dans l'image d'entrée.
- Plusieurs filtres peuvent être appliqués simultanément pour extraire différentes caractéristiques.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Le processus de convolution

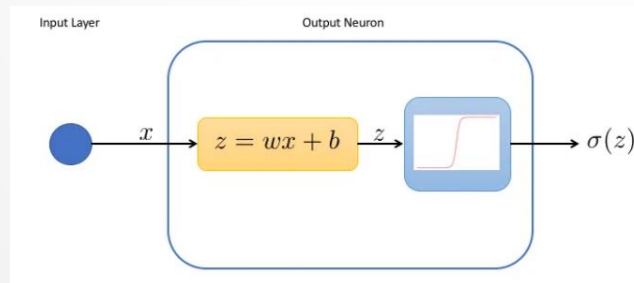
- Dans la réalité il y a plusieurs kernels (ou feature detector).
- Chaque kernel est spécialisé dans la détection d'un aspect particulier de l'image.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

L'opération ReLu (fonction d'activation)

- Après la convolution, une fonction d'activation est souvent appliquée à la carte des caractéristiques pour introduire de la non-linéarité dans le réseau neuronal.
- En effet, le processus de convolution nous donne des valeurs linéaires. Mais ReLu est conçu pour introduire de la non-linéarité dans l'image.
- La fonction d'activation la plus couramment utilisée est la fonction ReLU (Rectified Linear Unit), mais d'autres comme la fonction sigmoïde ou la tangente hyperbolique peuvent également être utilisées.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

L'opération ReLu (fonction d'activation)

- Pendant l'opération ReLu, toutes les valeurs négatives de la Feature Map sont remplacées par 0. Les positives restent inchangées.
- Supposons que nous ayons la Feature Map de gauche.
- Quand la fonction d'activation ReLu est appliquée sur elle, on obtient une nouvelle Feature Map (celle de droite).

-4	2	1	-2
1	-1	8	0
3	-3	1	4
1	0	1	-2

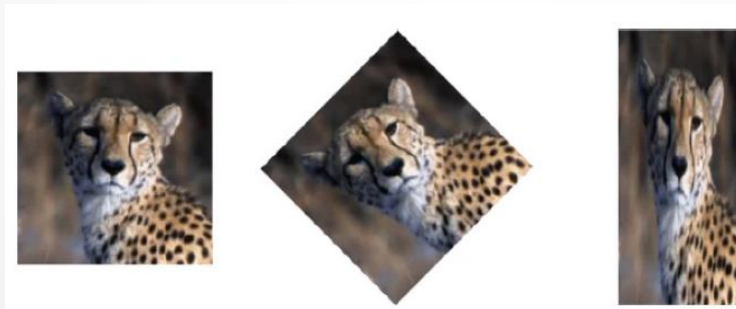


0	2	1	0
1	0	8	0
3	0	1	4
1	0	1	0

04 – Réseaux de neurones convolutifs : classification d'images (CNN)

L'opération Pooling

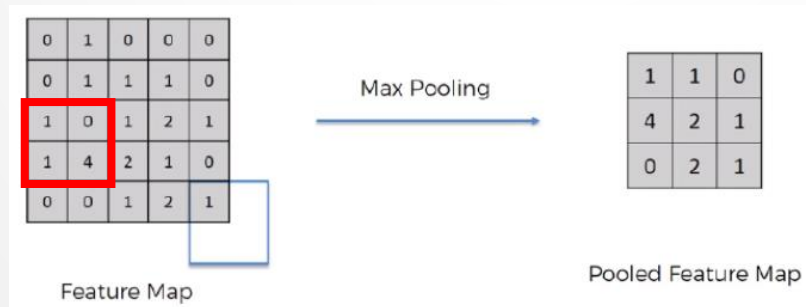
- L'opération Pooling est toujours réalisée après le processus de convolution et l'opération ReLu.
- Le rôle du Pooling est d'introduire de l'invariance spatiale dans la Feature Map (il réduit aussi les dimensions).
- L'invariance spatiale permet de stabiliser la reconnaissance de l'image même si elle subit une rotation ou une distorsion.
- Dans l'exemple ci-dessous, on reconnaît toujours l'image d'un guépard.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

L'opération Pooling

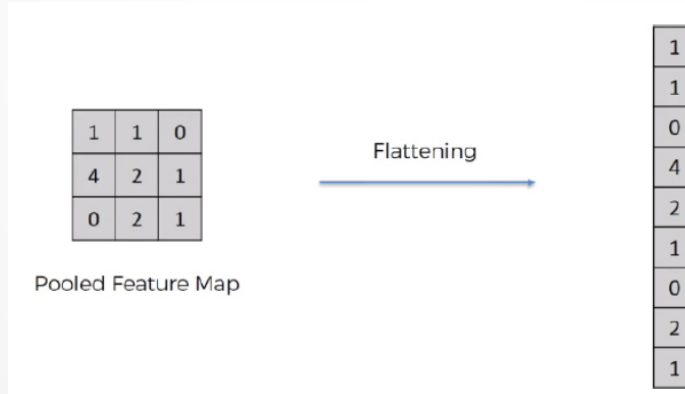
- Ici, nous avons une Feature Map et la matrice Pool peut être de taille variable.
- On passe le filtre de Pooling sur la Feature Map (max pooling, min pooling, average pooling).
- Après le Pooling, on obtient une image plus petite car on a réduit la dimension spatiale de la Feature Map.
- Par exemple, dans la Feature Map en rouge, le max pooling retiendra parmi ces pixels, la valeur la plus élevée 4.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Flattening et couches entièrement liées

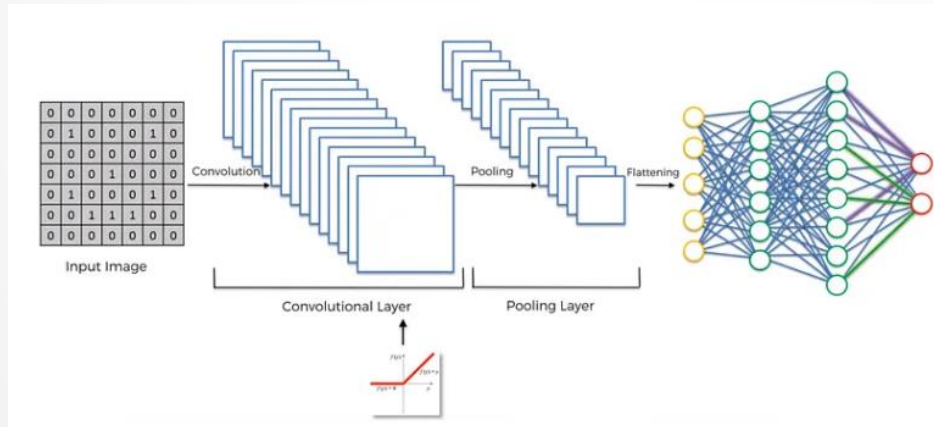
- Afin de trouver encore plus de Features à partir de l'image (input), les matrices issues du Pooling (pooled feature maps) sont aplaties pour former un vecteur à une dimension.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Flattening et couches entièrement liées

- Ce vecteur unidimensionnel est ainsi utilisé comme entrée pour les couches du réseau de neurones.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 1 (convolution)

- Au départ nous avons une image (input).
- On a un kernel 3×3 chargé de détecter des aspects spécifiques de l'image.

0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	1	0	0	0	0	1	0
0	0	1	1	1	0	0	0
0	0	0	0	0	0	0	0

Input Image

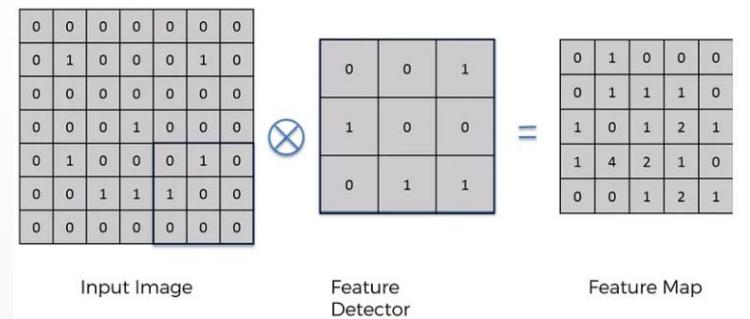
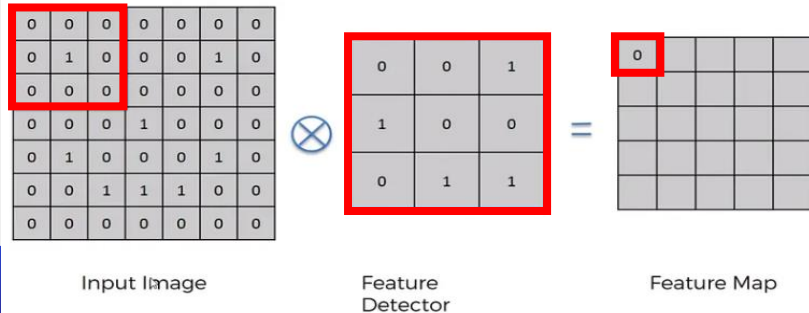
0	0	1
1	0	0
0	1	1

Feature
Detector

04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 1 (convolution)

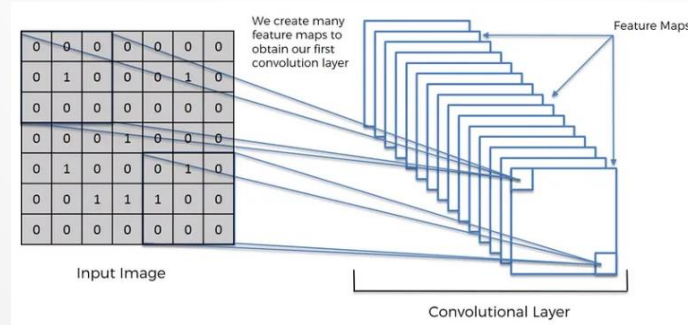
- Ensuite on multiplie chaque valeur du kernel par le premier lot de pixels de l'image.
- On additionne les résultats.
- Le résultat est soit 0 ou 1. Dans le cas précis, on obtient 0.
- L'opération se poursuit sur les lignes suivantes.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 1 (convolution)

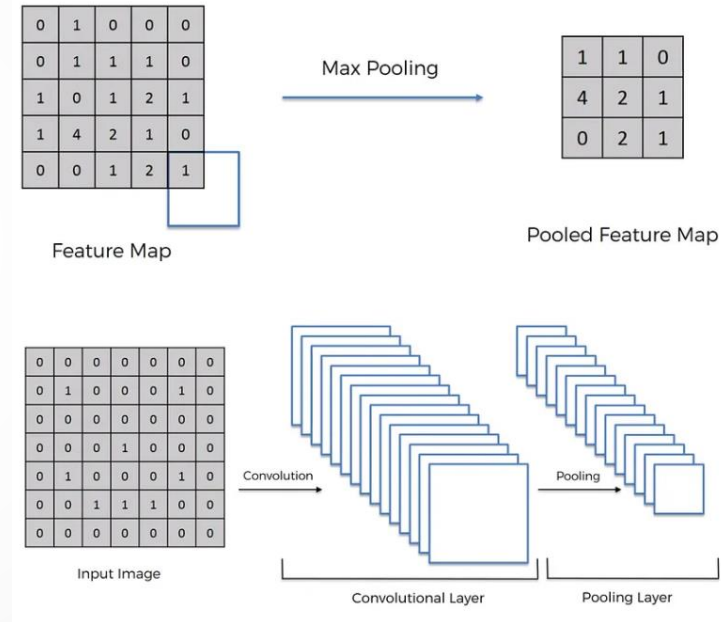
- En réalité, l'ordinateur ne parcourt pas chaque pixel, mais plutôt des lots de pixels.
- Comme chez l'humain, il ne regarde pas chaque détail (pixel), mais des features locales telles que le nez, le chapeau, les rayures du tigre...
- En entreprise, il y a de nombreuses fonctionnalités à détecter.
- On utilise donc plusieurs Feature Map pour détecter où se trouve une caractéristique spécifique sur l'image.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 2 (max pooling)

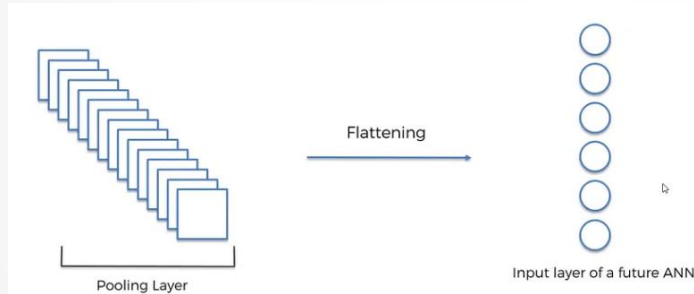
- Le réseau neuronal recherche exactement une certaine caractéristique.
- Par exemple, une caractéristique distinctive du guépard est les larmes qui coulent sur son visage provenant des yeux ou les ombres qui ressemblent à des larmes.
- Le motif qui va de ses yeux jusqu'aux côtés de son nez, qui ressemble à des larmes, est une caractéristique distinctive du guépard.
- Grâce au pooling, le réseau de neurones sera robuste car il va introduire la notion d'invariance spatiale.
- Cette notion permettra d'identifier l'animal quelque soit l'image, sa rotation ou son aplatissement.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 3 (flattening)

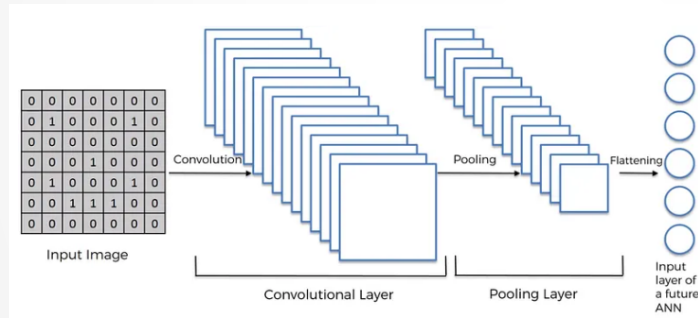
- Le flattening consiste à prendre la matrice issue du processus de Pooling (pooled feature map) pour l'étirer de façon à obtenir un vecteur colonne.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 3 (flattening)

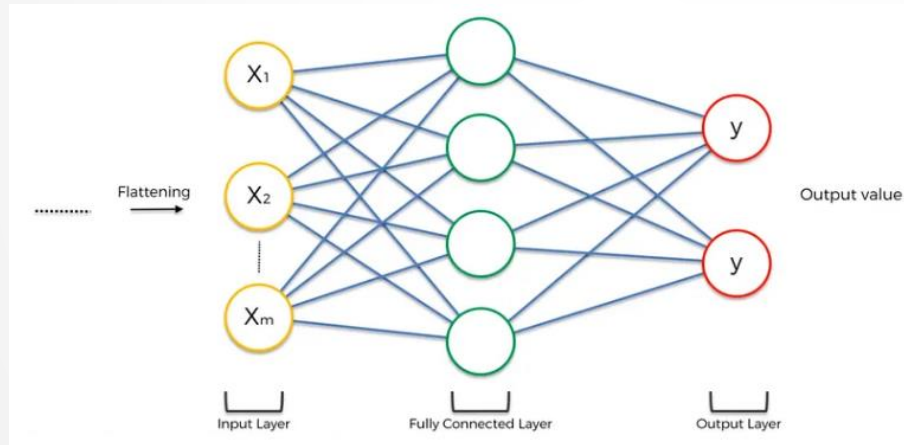
- Si on dispose de plusieurs couches (pooling Layer), on aplatit séquentiellement leurs éléments, les uns après les autres, de façon à former un très grand vecteur colonne.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 4 (réseau neuronal)

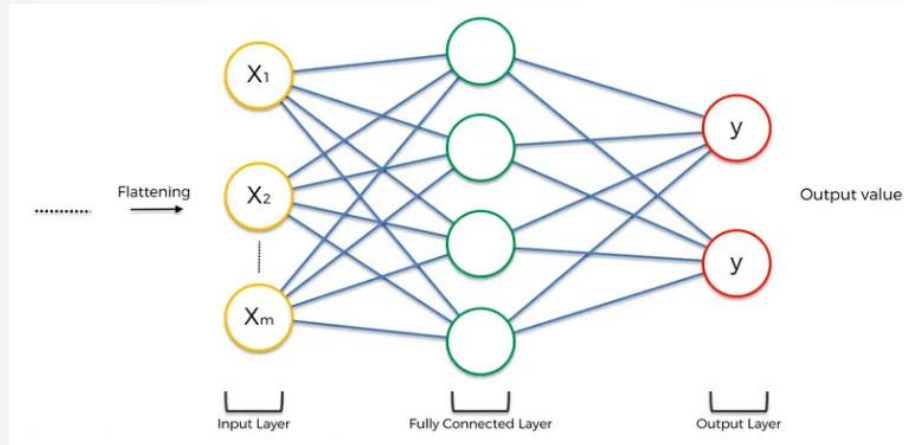
- À cette étape, on ajoute tout un réseau de neurones artificiels à notre réseau de neurones convolutifs.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 4 (réseau neuronal)

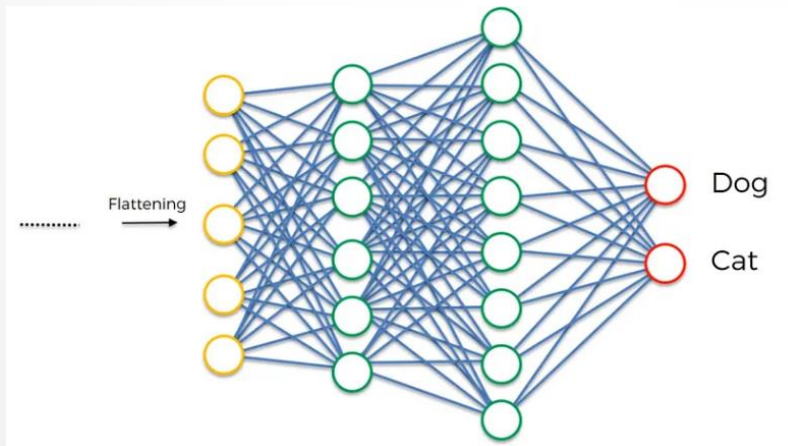
- À cette étape, on ajoute tout un réseau de neurones artificiels à notre réseau de neurones convolutifs.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 4 (réseau neuronal)

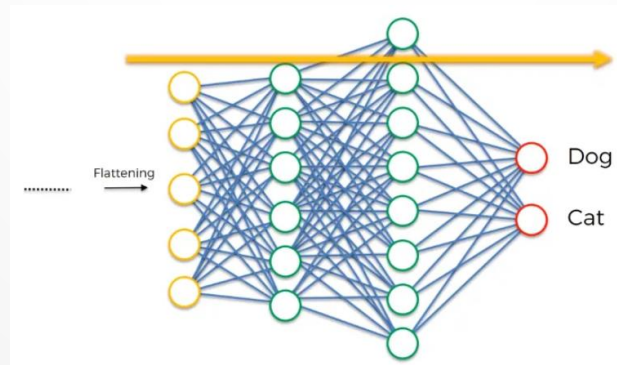
- Ici par exemple, on a :
 - 5 attributs désignés en input
 - 6 neurones dans la première couche cachée.
 - 8 neurones dans la seconde couche cachée.
 - 2 output : l'une pour prédire CHIEN et l'autre CHAT (classification)



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 4 (réseau neuronal)

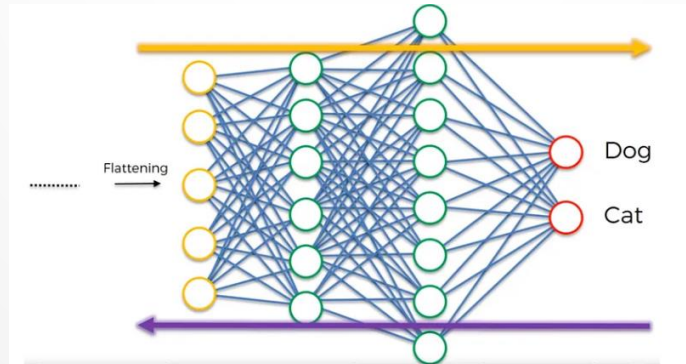
- Regardons ce qui s'y passe :
 - L'image est traitée
 - Convoluée
 - Regroupée
 - Aplatie, puis envoyée dans le réseau de neurones
 - Une prédiction est ainsi faite.
 - Puis une erreur est calculée sur la base de la fonction de coût (entropie croisée).



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 4 (réseau neuronal)

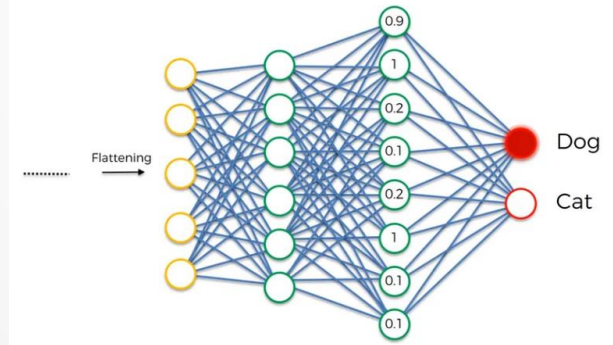
- Une fois l'erreur calculée :
 - Elle est propagée à travers le réseau
 - Elle est repropagée et on réajuste les poids le long du réseau pour optimiser les performances.
 - Les kernels sont également ajustés durant ce processus de rétropropagation.
- Ce processus est réitéré plusieurs fois.
- C'est ainsi que le réseau s'optime et qu'il s'entraîne sur les données.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 4 (réseau neuronal)

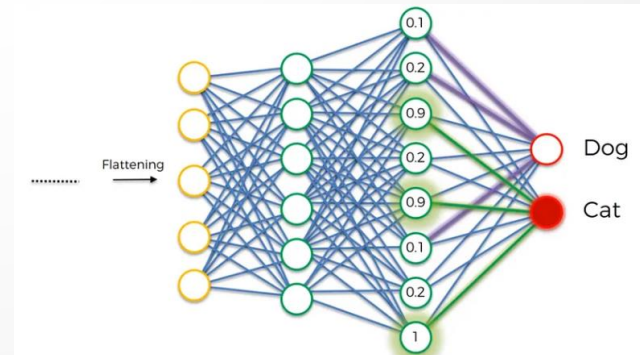
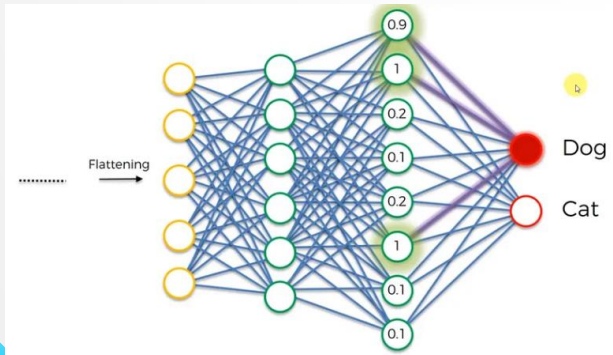
- Comment cela se passe t-il au niveau de l'output ?
 - Supposons que nous avons des poids (random)
 - Pour simplifier l'exemple, prenons des poids variants entre 0 et 1.
 - 0 signifie que le neurone n'a pas trouvé la caractéristique recherchée.
 - 1 signifie que le neurone a trouvé la caractéristique recherchée



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 4 (réseau neuronal)

- Dans le cas de la prédiction CHIEN, ces trois neurones s'activent.
- Disons que ces trois neurones traversent ce processus itératif avec de très nombreux échantillons et de très nombreuses époques.
- Grâce à de nombreuses itérations, ce neurone de chien a appris que ce neurone des sourcils, ce neurone du gros nez et ce neurone des oreilles tombantes, semblent tous vraiment très bien contribuer à la classification de ce qu'il recherche et de ce qu'est un chien.
- Le même processus est réalisé pour la prédiction CHAT.



04 – Réseaux de neurones convolutifs : classification d'images (CNN)

Résumé : étape 4 (réseau neuronal)

- Au final, les prédictions sont correctement réalisées pour identifier un chien ou un chat à partir d'images.

